

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Институт информационных технологий

Факультет компьютерных технологий

Кафедра информационных систем и технологий

К защите допустить:

Заведующий кафедрой ИСиТ

_____ А.И. Парамонов

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту
на тему

**КРОССПЛАТФОРМЕННОЕ МОБИЛЬНОЕ ПРИЛОЖЕНИЕ
УЧЕТА И АНАЛИЗА ПЕРСОНАЛЬНЫХ РАСХОДОВ**

БГУИР ДП 1-40 01 01 01 218 ПЗ

Студент А.С. Томчук

Руководитель В.А. Сицко

Консультанты:
от кафедры ИСиТ В.А. Сицко
по экономической части С.М. Климов

Нормоконтролер И.А. Середа

Рецензент

Минск 2026

РЕФЕРАТ

КРОССПЛАТФОРМЕННОЕ МОБИЛЬНОЕ ПРИЛОЖЕНИЕ УЧЕТА И АНАЛИЗА ПЕРСОНАЛЬНЫХ РАСХОДОВ: дипломный проект / А.С. Томчук. – Минск: БГУИР, 2026, – п.з. –126 с., чертежей (плакатов) – 6 л. формата А1.

Предметом дипломного проекта является разработка кроссплатформенного мобильного приложения для учёта персональных финансов.

Объектом проектирования выступает программное средство, реализующее клиент-серверную архитектуру для управления и анализа финансовых данных пользователя.

Цель дипломного проекта — создание удобного и функционального мобильного приложения, автоматизирующего процессы учёта доходов и расходов, формирования аналитических отчётов и планирования бюджета на устройствах с различными операционными системами.

В ходе работы были изучены современные подходы к мобильной разработке, проведён анализ существующих решений в сфере финансового учёта, а также определён набор ключевых функций, необходимых для эффективного использования приложения.

Для реализации программного средства были выбраны и обоснованы наиболее подходящие технологии, такие как: Flutter Framework (Dart 3.0+) для кроссплатформенной разработки под Android и iOS, архитектурный подход Clean Architecture и паттерн BLoC для управления состоянием. В качестве систем хранения данных используются SQLite. Для визуализации аналитических данных применяются библиотеки Syncfusion Charts и flutter_svg, а для обеспечения безопасности — SHA-256 хеширование.

Разработанное приложение включает модули аутентификации, учёта транзакций, аналитики, бюджетирования и управления категориями. Система поддерживает локальное хранение данных, что гарантирует конфиденциальность и автономность работы пользователя.

В разделе технико-экономического обоснования были произведены расчеты затрат, связанных с реализацией проекта, а также рентабельность разработки проекта, которое показывает, что создание данного приложения является целесообразным и востребованным решением для широкого круга пользователей, стремящихся к эффективному управлению своими финансами и финансовой дисциплине.

Разработанное мобильное приложение может быть использовано для личного финансового контроля, а также интегрировано в более крупные системы управления финансами.

Министерство образования Республики Беларусь
Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ
И РАДИОЭЛЕКТРОНИКИ**
Институт информационных технологий

Факультет	КТ	Кафедра	ИСиТ
Специальность	1-40 01 01	Специализация	01

УТВЕРЖДАЮ

А.И.Парамонов

« 23 » сентября 2025 г.

ЗАДАНИЕ

по дипломному проекту студента

Томчук Анастасии Сергеевны

(фамилия, имя, отчество)

1. Тема проекта: **Кроссплатформенное мобильное приложение учета и анализа персональных расходов**

утверждена приказом по университету от « 23 » сентября 2025 г. № 130-и

2. Срок сдачи студентом законченной работы 22 декабря 2025 года

3. Исходные данные к проекту Тип операционной системы – ОС Windows 10 и выше;
Язык программирования – Flutter (Dart); кроссплатформенная разработка для Android и iOS. Перечень выполняемых функций: Добавление и редактирование записей о расходах и доходах с возможностью выбора категории, Ведение учёта в привязке к дате и валют, Генерация аналитических отчётов (диаграммы и графики по категориям и периодам), Настройка финансового бюджета и уведомления при его превышении, Поддержка нескольких языков (локализация интерфейса), Авторизация с использованием пароля, PIN-кода или биометрии (Face/Touch ID).

Назначение разработки: Создание удобного и кроссплатформенного инструмента для автоматизации процессов учёта и анализа персональных финансов пользователя, позволяющего контролировать расходы, планировать бюджет и получать наглядную аналитику.

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов):

Введение

1 Анализ предметной области

2 Моделирование предметной области

3 Проектирование и разработка программного средства

4 Тестирование программного средства

5 Руководство по эксплуатации программного средства

6 Техничко-экономическое обоснование разработки программного средства

Заключение

Список использованных источников
Приложение А. Исходный код программного средства
5. Перечень графического материала (с точным указанием наименования) и обозначения вида и типа материала):
Диаграмма вариантов использования. Плакат – формат А1, лист 1.
Диаграмма деятельности. Плакат – формат А1, лист 1.
Диаграмма классов программного средства. Плакат – формат А1, лист 1.
Добавление транзакции и пересчет бюджета. Схема алгоритма – формат А1, лист 1.
Агрегация расходов по категориям. Схема алгоритма – формат А1, лист 1.
Алгоритмы безопасности. Схема алгоритма – формат А1, лист 1.
6. Содержание задания по технико-экономическому обоснованию
Расчет экономической эффективности от внедрения программного средства

Консультанты по дипломному проекту (с указанием разделов, по которому они консультируют):

Сицко В. А. – разделы 1-5;

Климов С. М. – раздел 6 (технико-экономическое обоснование);

Середа И.А. – нормоконтроль.

ПРИМЕРНЫЙ КАЛЕНДАРНЫЙ ГРАФИК ВЫПОЛНЕНИЯ ДИПЛОМНОГО ПРОЕКТА

Наименование этапов дипломного проекта	Объём готовности проекта в %	Срок выполнения этапа	Примечание
Первая опроектовка (введение, разделы 1-3)	30%	15.09.25 – 26.10.25	Консультант от кафедры
Вторая опроектовка (разделы 3-5)	60%	27.10.25 – 29.11.25	Руководитель проекта
Третья опроектовка (разделы 5-6, заключение, список использованных источников)	90%	30.11.25 – 13.12.25	Руководитель проекта
Консультации по оформлению графического материала и пояснительной записки, нормоконтроль	–	С 01.10.25 (согласно графику)	Нормоконтролёр
Итоговая проверка готовности дипломного проекта на заседании рабочей комиссии кафедры ИСиТ и допуск к защите в ГЭК	100%	С 20.12.25 (согласно графику)	Председатель рабочей комиссии
Рецензирование дипломного проекта	100%	С 22.12.25 (согласно распоряжению)	Рецензент
Защита дипломного проекта	100%	С 19.01.26 (согласно приказу)	ГЭК

Дата выдачи задания 15 сентября 2024 г. Руководитель _____ / В.А. Сицко/

Задание принял к исполнению _____ / А.С. Томчук /

СОДЕРЖАНИЕ

Введение	7
1 анализ предметной области	9
1.1 Описание предметной области	9
1.2 Обоснование актуальности разработки программного средства	9
1.3 Анализ методов, способов, подходов, методик и технологий	11
1.4 Анализ существующих аналогов и прототипов	13
1.5 Выбор и обоснование инструментов разработки программного средства	21
1.6 Спецификация требований	22
2 Моделирование предметной области	25
2.1 Функциональная модель разрабатываемого программного средства	25
2.2 Разработка функциональной спецификации требований к программному средству	31
3 Проектирование и разработка программного средства	34
3.1 Проектирование архитектуры программного средства	34
3.2 Проектирование и разработка алгоритмов работы программы	35
3.3 Проектирование интерфейса приложения	42
3.4 Разработка модели базы данных	47
4 Тестирование программного средства	51
4.1 Выбор и обоснование видов тестирования	51
4.2 Результаты тестирования	52
4.3 Вывод тестирования	56
5 Руководство по эксплуатации программного средства	58
6 Технико-экономическое обоснование разработки программного средства	71
6.1 Характеристика программного средства, разрабатываемого для реализации на рынке	71
6.2 Расчет инвестиций в разработку для его реализации на рынке	72
6.3 Расчет экономического эффекта от реализации программного средства на рынке	77
6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	79
Заключение	81
Список использованных источников	82
Приложение А (обязательное) Исходный текст программы	84

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В данной работе применяются следующие термины с соответствующими определениями:

Кроссплатформенное приложение – программное обеспечение, работающее на разных операционных системах без изменения исходного кода.

Flutter – фреймворк для кроссплатформенной разработки мобильных приложений, использующий язык программирования Dart и обеспечивающий нативную производительность и гибкий интерфейс.

Dart – объектно– ориентированный язык программирования, применяемый для создания клиентских приложений на платформе Flutter.

Clean Architecture – архитектурный подход, основанный на разделении приложения на уровни (Entity, Use Case, Interface Adapter, Frameworks), обеспечивающий модульность, масштабируемость и удобство тестирования.

BLoC (Business Logic Component) – шаблон управления состоянием, позволяющий отделить бизнес – логику от пользовательского интерфейса и реализовать реактивное взаимодействие компонентов.

Клиент– серверная архитектура – модель взаимодействия, при которой клиентская часть (мобильное приложение) обращается к серверу для получения, хранения и обработки данных.

SQLite – встроенная реляционная база данных, предназначенная для локального хранения структурированных данных в мобильных приложениях.

Hive – хранилище данных для Flutter, обеспечивающее быстрое чтение и запись без необходимости использования SQL – запросов.

Syncfusion Charts – библиотека для Flutter, предназначенная для визуализации аналитических данных в виде графиков и диаграмм.

PIN – код – персональный идентификационный номер для упрощённой аутентификации пользователя в приложении.

Биометрическая аутентификация – способ идентификации пользователя по биометрическим параметрам (отпечаток пальца, распознавание лица).

SHA – 256 (Secure Hash Algorithm 256 – bit) – криптографический алгоритм хеширования, используемый для защиты паролей и PIN – кодов от несанкционированного доступа.

REST API (Representational State Transfer) – архитектурный стиль взаимодействия клиентских и серверных компонентов через HTTP – запросы.

JSON (JavaScript Object Notation) – текстовый формат обмена данными между клиентом и сервером, обеспечивающий удобство сериализации структурированных данных.

Бюджет – установленный пользователем лимит расходов на определённый период или категорию с целью контроля финансовых затрат.

Уведомление – сообщение, генерируемое системой для информирования пользователя о событиях.

ВВЕДЕНИЕ

Современный уровень цифровизации общества требует эффективных и доступных инструментов для управления личными финансами. Быстрое развитие электронных платёжных систем, мультивалютные расходы и рост финансовой активности пользователей создают потребность в надёжных и удобных приложениях для учёта и анализа персональных доходов и расходов. Кроссплатформенное мобильное приложение, разработанное в рамках данного проекта, призвано автоматизировать процесс управления личным бюджетом, обеспечивая точность расчётов, визуализацию данных и безопасность хранения финансовой информации.

Актуальность темы обусловлена необходимостью в современных решениях, сочетающих функциональность, кроссплатформенность и высокую степень защиты персональных данных. Разработка приложения на основе Flutter Framework позволяет реализовать единый программный продукт для операционных систем Android и iOS с использованием единой кодовой базы, что повышает эффективность разработки и упрощает последующее сопровождение. Применение Material 3 дизайн – системы обеспечивает современный и адаптивный интерфейс, поддерживающий как светлую, так и тёмную темы, а встроенная локализация делает приложение удобным для широкой аудитории.

Назначение разработки – создание безопасного и многофункционального инструмента для учёта и анализа персональных финансов. Приложение должно обеспечить возможность ведения подробной истории доходов и расходов, формирования аналитических отчётов, настройки бюджетов и получения уведомлений при их превышении, а также гарантировать безопасный доступ к данным пользователя.

Целью данной работы является разработка клиент–серверного кроссплатформенного мобильного приложения для учёта и анализа персональных расходов, обеспечивающего интуитивно понятный интерфейс с применением современных архитектурных и технологических решений.

Для достижения поставленной цели решаются следующие задачи:

- провести анализ предметной области и существующих аналогов мобильных финансовых приложений;
- сформулировать требования к функционалу, интерфейсу и безопасности приложения;
- обосновать выбор технологического стека;
- спроектировать структуру приложения, обеспечивающую разделение бизнес– логики, данных и пользовательского интерфейса;
- разработать функциональные модули: авторизация, управление транзакциями, аналитика, бюджеты, категории и повторяющиеся операции;
- реализовать алгоритмы аналитики и безопасности, а также систему хранения данных и механизм для ускорения доступа и кэширования;

– провести тестирование функциональности и удобства использования, а также подготовить руководство пользователя.

Исходные требования включают поддержку ОС Windows 7 и выше для разработки и тестирования, клиент– серверную архитектуру, кроссплатформенную реализацию на Flutter/Dart, поддержку нескольких языков, работу с различными валютами и безопасное хранение данных.

Ожидаемым результатом является полнофункциональное мобильное приложение, позволяющее пользователю: быстро фиксировать операции, формировать наглядную аналитику в виде диаграмм и графиков, настраивать бюджеты с уведомлениями о превышении, а также безопасно входить в систему с помощью пароля, PIN-кода или биометрии. Практическая значимость проекта заключается в создании современного цифрового инструмента, который способствует формированию финансовой дисциплины и упрощает процесс принятия экономических решений.

Пояснительная записка содержит шесть разделов.

Первый раздел – «Анализ предметной области» – включает в себя описание предметной области, обоснование актуальности разработки, анализ методов, способов, подходов, методик и технологий, существующих в рассматриваемой предметной области, анализ существующих аналогов и разработка первичных спецификации требований к разрабатываемому приложению.

Второй раздел – «Моделирование предметной области» – включает в себя функциональные модели предметной области и разработку укрупненной подробной функциональной спецификации требований к разрабатываемому приложению.

Третий раздел – «Проектирование и разработка программного средства» – включает в себя проектирование и разработку архитектуры приложения, проектирование алгоритмов, разработка общего дизайна и структуры приложения.

Четвертый раздел – «Тестирование программного средства» – содержит описание процесса проверки работоспособности приложения, разработанные тесты и сведения о результатах, проведенного в процессе разработки тестирования, выявленных по результатам тестирования ошибках и предпринятых действиях чтобы их устранить.

Пятый раздел – «Руководство по эксплуатации программного средства» – содержит иллюстрированное руководство пользователя по эксплуатации приложения с описанием использования основных функций и инструкцией по запуску приложения.

Шестой раздел – «Технико–экономическое обоснование разработки программного средства» – содержит расчеты технико–экономическое обоснования разработки приложения.

Данный дипломный проект выполнен мной лично, проверен на заимствования в системе «Антиплагиат», оригинальность пояснительной записки составляет xx%.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Управление личными финансами сегодня выходит за рамки разовых расчётов: это повседневная привычка, которая влияет на качество жизни и способность достигать долгосрочных целей. Рост числа безналичных операций, изменение цен и необходимость учитывать расходы в разных валютах делают задачу контроля бюджета более сложной [1] [3]. Многие люди стремятся понять, куда уходит их деньги, но сталкиваются с избытком данных и недостатком удобных инструментов для их анализа.

Традиционные способы учёта – записи в блокноте или таблицы – часто оказываются громоздкими и требуют много времени. Они не дают быстрого визуального представления о расходах, не умеют автоматизировать повторяющиеся операции и не умеют предупреждать о превышении установленных лимитов. Это приводит к тому, что учет бросают или ведут фрагментарно, а значит – теряется реальная картина финансового поведения.

Предметом исследования в данной работе является процесс сбора, хранения, анализа и визуализации персональных финансовых данных. В фокусе – операции по доходам и расходам, их категоризация, учёт по датам и валютам, а также инструменты планирования бюджета и уведомления. Объектами автоматизации выступают: регистрация и хранение транзакций; система категорий; модуль расчёта и агрегации данных; генератор отчётов и визуализаций; механизмы авторизации и защиты данных.

Проектируемая система призвана помочь обычному пользователю – человеку, который хочет спокойнее относиться к своим деньгам и принимать более обоснованные решения. Такой пользователь должен уметь быстро фиксировать операцию, отнести её к категории, увидеть динамику расходов за неделю или месяц и получить понятную визуальную сводку. Кроме того, полезна возможность задать бюджет и получать предупреждения при его приближении или превышении.

Использование мобильного клиент– серверного приложения делает решение доступным из любой точки и обеспечивает надежное локальное хранение с опцией синхронизации. Полнота записей обеспечивает гибкость анализа: можно смотреть обобщённые и детализированные графики по конкретным периодам или категориям. Наличие понятных отчётов и уведомлений снижает нагрузку на память пользователя и делает процесс управления финансами менее рутинным и более осмысленным.

1.2 Обоснование актуальности разработки программного средства

Управление личными финансами становится одной из ключевых задач современного человека. Экономическая нестабильность, рост инфляции и

постоянные изменения в ценах заставляют людей внимательнее подходить к планированию расходов. При этом значительная часть населения по-прежнему ведёт учёт вручную или вовсе не анализирует собственные траты, что приводит к финансовым трудностям, невозможности накоплений и неосознанным расходам.

По данным аналитических агентств, более 70 % пользователей смартфонов хотя бы раз пробовали приложения для контроля бюджета, однако только треть продолжает использовать их регулярно. Основные причины – сложный интерфейс, избыточный функционал, отсутствие локализации и ограниченные возможности анализа. Эти данные подтверждают необходимость создания нового поколения простых, понятных и визуально привлекательных приложений, адаптированных под реальные потребности пользователя.

Современные цифровые технологии позволяют сделать процесс управления личными финансами более точным, удобным и безопасным. Применение мобильных приложений позволяет пользователю вести учёт расходов и доходов в реальном времени, анализировать свои финансовые привычки и принимать решения на основе наглядных данных. Визуализация информации в виде графиков и диаграмм делает результаты анализа интуитивно понятными и помогает пользователю осознанно управлять своим бюджетом.

Особое значение имеет кроссплатформенность. Создание единого приложения, работающего на устройствах под управлением Android и iOS, снижает затраты на разработку и поддержку, ускоряет внедрение новых функций и обеспечивает единый пользовательский опыт. Использование современного фреймворка Flutter и языка Dart позволяет достичь нативной производительности и плавной анимации, сохраняя при этом универсальность кода. Выбор технологии Flutter обоснован её высокой производительностью и поддержкой со стороны Google [2], что позволяет реализовать качественное кроссплатформенное решение при оптимальных затратах ресурсов.

С точки зрения архитектуры, использование Clean Architecture и паттерна BLoC обеспечивает модульность, гибкость и удобство тестирования приложения. Такие решения позволяют масштабировать проект, добавляя новые функции – например, автоматическую категоризацию расходов, работу с несколькими валютами или экспорт отчётов – без нарушения целостности системы. Подобный подход повышает устойчивость к изменениям и упрощает развитие программного продукта.

Немаловажен и аспект безопасности. Пользователи всё чаще обращают внимание на конфиденциальность финансовых данных. Поэтому применение криптографических алгоритмов (в частности, SHA-256) и биометрической аутентификации обеспечивает высокий уровень защиты при сохранении удобства. Возможность локального хранения данных с опцией синхронизации также повышает доверие к приложению и делает его

привлекательным для широкой аудитории.

Финансовая грамотность остаётся актуальной социальной задачей. Разны приложения для учёта расходов помогают пользователям лучше понимать структуру своих трат, формировать сбережения и избегать импульсивных покупок. Наглядная аналитика и возможность самостоятельно задавать финансовые цели формируют более ответственное отношение к деньгам и снижают уровень финансовой уязвимости населения [4].

С точки зрения рынка, сегмент FinTech и персональных финансовых приложений демонстрирует устойчивый рост [5]. По данным исследовательских агентств, ежегодный прирост пользователей таких приложений составляет 10–15 %, а их доля в структуре мобильных сервисов стабильно увеличивается. Пользователи ожидают от подобных решений не просто учёта, а анализа, рекомендаций и персонализированных уведомлений. Следовательно, разработка приложения с функциями аналитики и прогнозирования расходов отвечает современным трендам и потребностям рынка.

С профессиональной стороны проект также имеет образовательную ценность. Его реализация позволяет изучить современные подходы к разработке мобильных приложений, освоить принципы построения клиент–серверной архитектуры, управление состоянием и безопасность данных. Это сочетает теоретические знания с практическими навыками, необходимыми для работы в сфере мобильной и финансовой разработки.

Создание кроссплатформенного мобильного приложения учёта и анализа персональных расходов отражает актуальные тенденции в цифровой экономике. Оно объединяет в себе социальную значимость, технологическую новизну и практическую ценность – как для конечных пользователей, так и для разработчиков, стремящихся создавать современные, надёжные и полезные программные решения.

1.3 Анализ методов, способов, подходов, методик и технологий

Современная разработка программных систем опирается на проверенные методы и подходы, обеспечивающие высокое качество, масштабируемость и удобство сопровождения программного продукта. Для создания кроссплатформенного мобильного приложения учёта и анализа персональных расходов целесообразно рассмотреть архитектурные решения, подходы к проектированию пользовательского интерфейса, организации бизнес–логики и хранения данных.

В области проектирования программного обеспечения применяются различные методологии: каскадная (Waterfall), инкрементальная, спиральная, Agile и Scrum, что делает данные подходы особенно популярными при разработке мобильных приложений [6]. Для разработки мобильных приложений наилучшим образом подходит гибкий подход Agile, который предполагает итеративное создание продукта, постоянное

тестирование и быстрое внесение изменений. Такой подход позволяет адаптировать приложение под потребности пользователя и оперативно улучшать функциональность без длительных циклов обновления.

При проектировании архитектуры мобильных систем особое внимание уделяется модульности, разделению ответственности и управляемости изменениями. Эти принципы лежат в основе архитектурных шаблонов, которые применяются в современных кроссплатформенных проектах.

Среди наиболее распространённых архитектурных моделей можно выделить MVC (Model– View– Controller), MVP (Model– View– Presenter), MVVM (Model– View– ViewModel) и Clean Architecture. Последняя представляет собой усовершенствованный подход, направленный на чёткое разделение уровней приложения:

- Entity – бизнес– логика и основные модели данных;
- Use Case – сценарии взаимодействия между компонентами;
- Interface Adapter – преобразование данных для отображения;
- Frameworks and Drivers – реализация пользовательского интерфейса и работы с внешними источниками.

Использование Clean Architecture позволяет легко масштабировать приложение, добавлять новые функции и поддерживать высокий уровень тестируемости кода [7]. Для проекта по учёту и анализу персональных расходов данный подход является оптимальным, поскольку обеспечивает независимость бизнес– логики от интерфейса и внешних сервисов.

Одной из ключевых задач при разработке мобильных приложений является организация корректного обмена данными между компонентами. В экосистеме Flutter для этого применяется шаблон BLoC (Business Logic Component), который разделяет бизнес– логику и пользовательский интерфейс. Он основан на принципах реактивного программирования и использует потоки данных (Streams), что обеспечивает стабильную работу и лёгкое тестирование модулей.

BLoC– подход также способствует повышению читаемости кода и повторному использованию компонентов, что важно при создании сложных приложений с несколькими экранами и состояниями [8].

Существует три основных направления разработки мобильных приложений:

- нативная разработка – создание отдельных версий приложения под android и ios (на языках kotlin и swift соответственно). преимущества: высокая производительность и глубокая интеграция с ос. недостаток: увеличение времени и стоимости разработки.
- гибридная разработка – создание приложений с использованием веб– технологий (html, css, javascript) и упаковкой их в мобильную оболочку. недостатки: ограниченные возможности и зависимость от веб– движка.
- кроссплатформенная разработка – создание единого кода, компилируемого под разные платформы. этот подход сочетает

производительность, близкую к нативной, и эффективность разработки.

Выбор в пользу Flutter обусловлен рядом преимуществ: высокая скорость работы, выразительный пользовательский интерфейс, встроенная поддержка Hot Reload, развитая экосистема виджетов и активная поддержка сообщества. Flutter компилируется в нативный код, что обеспечивает стабильную работу и плавную анимацию даже на устройствах со средними характеристиками. Кроме того, язык Dart сочетает простоту синтаксиса с возможностями строгой типизации, что снижает количество ошибок при разработке.

Для хранения данных в мобильных приложениях используются как реляционные, так и нереляционные СУБД. Наиболее популярными являются SQLite, Hive и SharedPreferences.

- SQLite применяется для хранения структурированных данных и удобна при работе с табличными зависимостями (доходы, расходы, категории, валюты) [9].

- Hive представляет собой nosql– хранилище, обеспечивающее быструю запись и чтение данных, что особенно актуально для небольших по объёму, но часто изменяемых структур.

- SharedPreferences используется для хранения пользовательских настроек и локальных параметров интерфейса.

Для обмена данными между клиентом и сервером используется архитектурный стиль REST API с форматами JSON для передачи структурированных данных. Этот подход обеспечивает совместимость и лёгкость интеграции с внешними сервисами, такими как источники валютных курсов или облачные хранилища.

Для повышения наглядности анализа используются библиотеки визуализации, например Syncfusion Charts, позволяющие строить интерактивные графики, диаграммы и отчёты [10]. Это повышает информативность приложения и способствует лучшему восприятию данных пользователем.

Анализ современных подходов и технологий показывает, что использование Flutter в сочетании с Clean Architecture, BLoC– паттерном, SQLite/Hive, а также REST API и библиотеками визуализации данных является оптимальным решением для реализации проекта. Такой стек технологий обеспечивает кроссплатформенность, производительность, безопасность и гибкость, что соответствует целям разработки программного средства для учёта и анализа персональных расходов.

1.4 Анализ существующих аналогов и прототипов

Для определения оптимальной функциональности, интерфейса и пользовательских сценариев разрабатываемого приложения важно изучить существующие решения, уже реализующие похожие задачи. Анализ аналогов позволяет выявить сильные стороны и недостатки существующих

систем, определить удобные механики и функции, а также найти пробелы, которые можно устранить в создаваемом программном средстве.

В ходе анализа были рассмотрены три популярных приложения, доступные пользователям App Store: YNAB (You Need A Budget), Учёт расходов – Бюджет ОК и Расходы – Личный бюджет. Данные продукты представляют разные концепции финансового менеджмента – от сложного системного планирования до простого ежедневного учёта расходов.

Приложение YNAB является одним из наиболее известных и функционально насыщенных инструментов для управления личными финансами. Его ключевая особенность заключается в концепции «каждому доллару – задача», подразумевающей полное распределение доходов по категориям и контроль над каждой статьёй расходов. Пример отображения расходов представлен на рисунке 1.1.



Рисунок 1.1 – Добавления расхода

YNAB поддерживает синхронизацию с облачным хранилищем, предоставляет широкий набор инструментов для анализа и построения бюджета, а также наглядные графики и отчёты, что позволяет пользователю видеть полную картину своих финансов. Пример отчета расходов по категориям за август представлен на рисунке 1.2.



Рисунок 1.2 – Отчет по категориям

Приложение реализовано в кроссплатформенном формате и доступно для iOS, Android и Web, что обеспечивает удобство использования на различных устройствах. Однако, несмотря на высокую функциональность, система имеет ряд ограничений: отсутствует локализация на русский язык, интерфейс требует времени на освоение, а платная модель использования снижает привлекательность для широкой аудитории. Кроме того, отсутствие возможности локального хранения данных и зависимости от облака может восприниматься как недостаток с точки зрения приватности.

В отличие от YNAB, приложение «Финансы» ориентировано на простоту и удобство повседневного использования. Оно предоставляет пользователю возможность быстро вносить операции, группировать их по категориям, а также отслеживать статистику в виде диаграмм и графиков. Интерфейс полностью локализован и адаптирован под русскоязычную аудиторию, поддерживается работа с несколькими валютами. Пример добавления операции виден на рисунке 1.3.

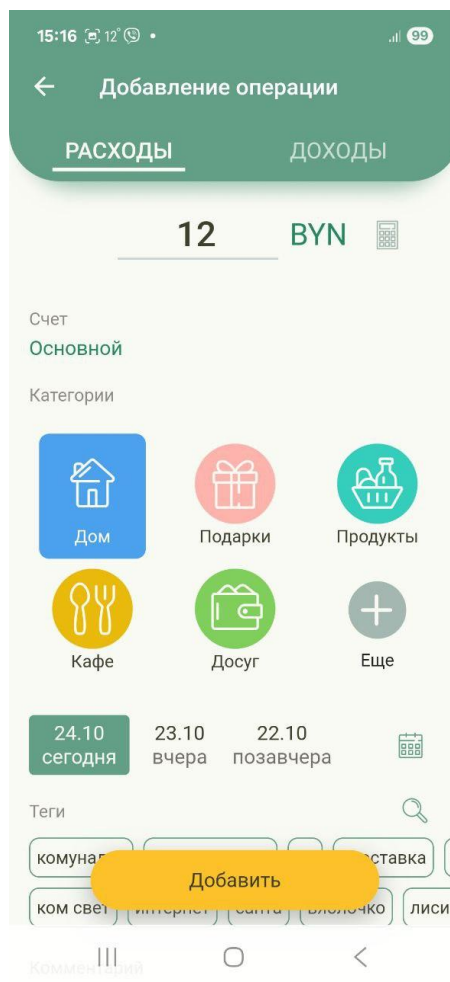


Рисунок 1.3 – Добавление операции

Основное преимущество данного решения заключается в его простоте и интуитивности. Пользователь может быстро вносить данные о своих расходах и доходах, указывая категорию, дату и сумму. Интерфейс выполнен лаконично и не перегружен лишними элементами, что особенно важно для пользователей, не имеющих опыта работы с финансовыми инструментами. Кроме того, приложение поддерживает работу с несколькими валютами и позволяет создавать отдельные счета или цели, что делает его удобным для тех, кто ведёт расходы в разных валютах или планирует крупные покупки. Отдельного внимания заслуживает реализация визуальной аналитики: пользователь может просматривать диаграммы и графики, отражающие структуру расходов по категориям, а также динамику доходов и затрат за выбранный период. Такая наглядность помогает формировать финансовую дисциплину и способствует осознанному управлению личными средствами. Ещё одним преимуществом является наличие бесплатной версии, которая предоставляет достаточный набор функций для базового финансового учёта, что делает приложение доступным для широкой аудитории. Пример сводного отчета по категориям представлен на рисунке 1.4.

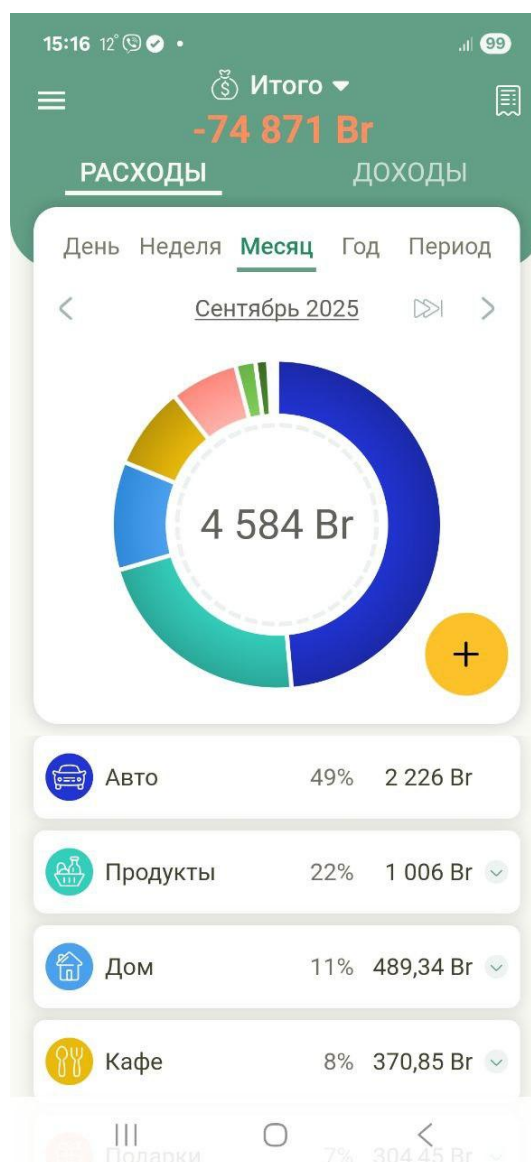


Рисунок 1.4 – Отчет по категориям

Однако при всех достоинствах система имеет ряд ограничений. Одним из наиболее существенных недостатков является отсутствие механизмов защиты данных – в приложении, как правило, не реализована авторизация с использованием PIN– кода, пароля или биометрической аутентификации, что снижает уровень безопасности при хранении персональной финансовой информации. Кроме того, не предусмотрена синхронизация данных между устройствами: информация сохраняется локально, и при смене телефона пользователь рискует потерять накопленные данные. Это снижает гибкость работы с системой и делает её менее удобной по сравнению с кроссплатформенными решениями, поддерживающими облачное хранение.

Кроме того, несмотря на наличие версии для Android, приложение не является кроссплатформенным, и пользователи устройств под

управлением iOS не имеют возможности использовать его с тем же функционалом. Это ограничивает потенциальную аудиторию и снижает удобство для тех, кто пользуется несколькими устройствами. С точки зрения локализации, интерфейс переведён на русский язык и адаптирован под региональные особенности, что можно отнести к положительным сторонам, однако визуальная и функциональная гибкость интерфейса остаётся ограниченной – отсутствует возможность настройки категорий, цветовых схем или индивидуальных иконок.

Приложение «Расходы – Личный бюджет» занимает промежуточное положение между простыми и более сложными решениями. Его функционал направлен на быстрый учёт доходов и расходов с минимальными действиями со стороны пользователя. Система позволяет задавать категории, использовать разные валюты и контролировать выполнение бюджета за выбранный период. Пример отображения представлен на рисунке 1.5.

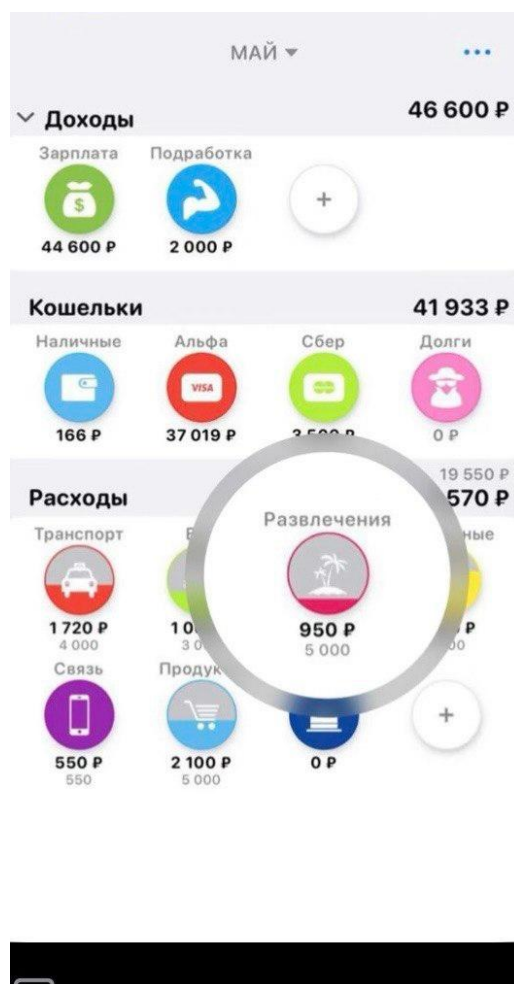


Рисунок 1.5 – Главный экран

В приложении «Расходы – Личный бюджет» пользователь начинает работу с созданием или выбором счёта (например, наличные, карта,

электронный кошелёк), после чего имеет возможность добавлять операции расходов или доходов вручную. Обычно это делается при помощи кнопки «+» или аналогичной иконки – пользователю предлагается ввести сумму, выбрать категорию (например, «Питание», «Транспорт», «Развлечения»), указать. Такой подход обеспечивает контроль над каждым расходом и позволяет фиксировать данные сразу после совершения операции, что повышает актуальность учёта. Создание записи о расходе представлено на рисунке 1.6.

The screenshot shows a mobile application interface for recording an expense. At the top, there is a title bar with the word "Расход" (Expense) and a close button. Below the title bar, the "Сумма" (Sum) field is set to "2 560 Р". Underneath, the "Кошелек" (Wallet) is set to "Наличные" (Cash) with a balance of "96 500 Р". The "Категория" (Category) is set to "Транспорт" (Transport). Below this, the "Подкатегория" (Subcategory) is set to "без" (None). The "Дата" (Date) is set to "28 декабря" (December 28). A blue "Сохранить" (Save) button is located below the date. At the bottom of the screen is a numeric keypad with digits 1-9, 0, and symbols for clearing, division, multiplication, subtraction, and addition.

Рисунок 1.6 – Описание расхода

После того как записано достаточное количество операций, приложение формирует отчёт по категориям, в котором расходы распределены по категориям за выбранный период. В частности, одним из основных видов визуализации используется круговая диаграмма: каждое цветовое «сектор» диаграммы соответствует одной категории расходов, и размер сектора показывает долю этой категории в общей сумме расходов за период. Пользователь может одним взглядом увидеть, на что он тратит больше всего – например, если сектор «Питание» занимает 35 % расходов, а «Развлечения» – 10 %, это сразу видно. Диаграмма часто сопровождается списком категорий с указанием суммы и процента, что позволяет выбрать категорию для более детального анализа. Пример диаграммы представлен на рисунке 1.7. Данный подход имеет несколько важных преимуществ. Во– первых, удобная ручная фиксация операций обеспечивает

интуитивный ввод: пользователь не перегружён сложными настройками, быстро получает результат. Во– вторых, визуализация через круговую диаграмму делает структуру расходов наглядной и легко воспринимаемой, помогает пользователю осознать.



Рисунок 1.7 – Круговая диаграмма.

Простота интерфейса и лёгкость работы делают приложение удобным для повседневного применения. Однако его возможности ограничиваются базовыми функциями: аналитика представлена в упрощённом виде, отсутствует синхронизация между устройствами, не поддерживаются уведомления о превышении бюджета и интеграция с внешними сервисами. Также приложение не реализует авторизацию и не использует биометрические средства защиты, а данные хранятся открыто на устройстве без шифрования. Отдельным недостатком можно считать отсутствие кроссплатформенных технологий – пользователь не имеет возможности продолжить работу на другом устройстве, сохранив данные и настройки.

Проведённый анализ показывает, что существующие приложения для учёта персональных финансов условно можно разделить на две категории. К первой относятся системы с расширенным функционалом и глубокой аналитикой, обладающие высокой мощностью, но сложные для освоения и требующие постоянного подключения к сети. Ко второй – простые и удобные трекеры, отличающиеся простотой, но ограниченные по аналитике, безопасности и возможностям кастомизации.

Разрабатываемое приложение должно объединить достоинства рассмотренных аналогов, сохранив простоту интерфейса и доступность, но дополнив их современными возможностями анализа и планирования. Предусматривается полная локализация интерфейса, поддержка нескольких валют, уведомления о превышении бюджета, а также механизмы защиты данных с использованием PIN– кода и биометрической аутентификации. Особое внимание уделяется кроссплатформенности, которая обеспечивается использованием фреймворка Flutter. Это позволит поддерживать единый код для Android и iOS, гарантируя пользователям одинаковый функционал и визуальное оформление независимо от устройства.

1.5 Выбор и обоснование инструментов разработки программного средства

Для реализации кроссплатформенного мобильного приложения учёта и анализа персональных расходов был проведён анализ современных технологий и инструментов, применяемых в области мобильной разработки. Основным требованием при выборе технологического стека являлась возможность создания единого приложения, одинаково корректно работающего на устройствах под управлением Android и iOS, с сохранением высокой производительности, простоты сопровождения и современного пользовательского интерфейса.

В качестве основной платформы разработки выбран фреймворк Flutter, разработанный компанией Google. Он позволяет создавать нативные по производительности мобильные приложения на едином языке программирования Dart, что существенно сокращает время и затраты на разработку и поддержку приложения для разных операционных систем. Flutter обеспечивает гибкие возможности по кастомизации интерфейса, богатый набор встроенных виджетов, поддержку анимаций и адаптивной верстки, что делает его оптимальным выбором для разработки современного и визуально привлекательного пользовательского интерфейса. Кроме того, Flutter активно поддерживается сообществом и развивается в рамках официальной поддержки Google, что гарантирует его стабильность и совместимость с новыми версиями операционных систем.

В качестве архитектурного подхода выбрана Clean Architecture, обеспечивающая модульность, читаемость и расширяемость кода. Этот подход позволяет отделить бизнес– логику от пользовательского интерфейса, что упрощает тестирование, сопровождение и дальнейшее развитие проекта. Для управления состоянием приложения используется паттерн BLoC (Business Logic Component), позволяющий реализовать реактивное взаимодействие между слоями интерфейса и логики, обеспечивая предсказуемость поведения и удобство при масштабировании.

Для хранения данных выбрана локальная база данных SQLite,

обеспечивающая надёжное и эффективное хранение структурированных данных непосредственно на устройстве пользователя. В качестве альтернативы и для упрощённого взаимодействия с данными может использоваться хранилище Hive, которое отличается высокой скоростью работы и отсутствием необходимости в написании SQL– запросов. Такой подход позволяет пользователю работать с приложением в офлайн–режиме без необходимости постоянного интернет– соединения, а при наличии синхронизации – автоматически обновлять данные.

Для визуализации аналитической информации (диаграмм и графиков) используется библиотека Syncfusion Charts, которая предоставляет широкий набор инструментов для построения интерактивных диаграмм, включая круговые, линейные и столбчатые графики. Это позволяет наглядно представить пользователю структуру расходов и доходов по категориям и периодам.

Для обеспечения безопасности пользовательских данных применяется алгоритм SHA– 256, используемый для хеширования паролей и PIN– кодов, что предотвращает несанкционированный доступ к конфиденциальной информации. Для аутентификации пользователей предусмотрено использование PIN– кода и биометрических данных (Face ID или Touch ID), что делает процесс входа в приложение быстрым и безопасным.

В процессе разработки используются современные инструменты и среды: Android Studio и Visual Studio Code – для написания и отладки кода, а также встроенная система управления пакетами pub.dev, предоставляющая доступ к библиотекам и компонентам, необходимым для реализации дополнительных функций.

Выбор именно этих инструментов обусловлен их надёжностью, широкими возможностями интеграции, активным развитием и соответствием современным стандартам мобильной разработки. Такое сочетание технологий обеспечивает высокое качество, удобство сопровождения и дальнейшего масштабирования программного продукта.

1.6 Спецификация требований

Разрабатываемое программное средство представляет собой кроссплатформенное мобильное приложение для учёта и анализа персональных расходов, предназначенное для автоматизации процессов ведения личного бюджета. Система обеспечивает пользователю возможность фиксировать доходы и расходы, анализировать финансовые потоки, формировать отчёты и визуализировать данные в виде графиков и диаграмм. Приложение предназначено для индивидуального использования и ориентировано на пользователей, стремящихся контролировать личные финансы, повышать финансовую грамотность и рационально планировать бюджет.

Система должна выполнять следующие основные функции:

- добавление, редактирование и удаление записей о доходах и расходах с возможностью выбора категории, даты и валюты;
- отображение финансовых операций в виде списка с возможностью фильтрации по дате, категории или типу операции;
- ведение учёта в нескольких валютах с автоматическим пересчётом сумм при необходимости;
- генерация аналитических отчётов с визуализацией данных (круговые диаграммы, графики расходов по периодам и категориям);
- настройка индивидуального бюджета с указанием лимита на определённый период и уведомления при его превышении;
- реализация авторизации пользователя с использованием пароля, pin-кода или биометрической аутентификации (face/touch id);
- поддержка локализации интерфейса (многоязычный интерфейс);
- возможность локального хранения данных с опцией синхронизации и резервного копирования;
- настройка параметров приложения и отображаемых категорий;
- предоставление пользователю аналитических данных по динамике доходов, расходов и накоплений за выбранный период.
- система принимает следующие входные данные:
- сумма операции (в валюте пользователя);
- тип операции (доход или расход);
- категория операции (питание, транспорт, развлечения и др.);
- дата и время совершения операции;
- комментарий (необязательно);
- параметры пользователя (логин, пароль, pin-код, биометрические данные при авторизации).

Система генерирует следующие выходные данные:

- сформированные отчёты и диаграммы, отображающие распределение расходов и доходов по категориям и периодам;
- уведомления о превышении бюджета или приближении к лимиту;
- аналитические показатели (процентное соотношение расходов, общие итоги за период, динамика изменений);
- экспортируемые данные (возможность выгрузки отчёта или резервной копии).

Аппаратные требования:

- мобильное устройство под управлением Android 10.0 и выше или iOS 13.0 и выше;
- оперативная память – не менее 2 ГБ;
- свободное место на устройстве – не менее 100 МБ;
- поддержка модуля Face ID или Touch ID (для аутентификации, при наличии).

Программные требования:

- операционная система – Windows 7 и выше (для разработки);

- фреймворк – Flutter (версия 3.x и выше);
- язык программирования – Dart;
- система управления базами данных – SQLite или Hive;
- дополнительная библиотека для визуализации – Syncfusion Charts;
- инструментальная среда разработки – Android Studio или Visual Studio Code.

Система должна обеспечивать корректное сохранение данных при любых сбоях в работе устройства. Все операции записываются в локальное хранилище с возможностью автоматического резервного копирования. В случае непредвиденного завершения работы приложения данные не должны теряться.

Для обеспечения безопасности пользовательских данных должны быть реализованы следующие меры:

- авторизация с использованием PIN– кода или биометрических данных;
- хранение паролей в зашифрованном виде с применением алгоритма SHA– 256;
- ограничение доступа к данным приложения только зарегистрированным пользователям;
- возможность блокировки приложения при бездействии пользователя.

Приложение должно быть простым и удобным в использовании. Интерфейс должен оставаться понятным и аккуратным на устройствах с разными размерами экранов. Время отклика при добавлении или просмотре данных не должно превышать одной секунды, чтобы обеспечить комфортную работу пользователя. Приложение должно одинаково корректно работать на устройствах под управлением Android и iOS, сохраняя единый внешний вид и функциональность. Интерфейс должен поддерживать как минимум русский и английский языки, чтобы приложение было доступно более широкой аудитории. Архитектура программы должна быть построена таким образом, чтобы при необходимости можно было легко добавлять новые функции без переработки существующей структуры.

2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Функциональная модель разрабатываемого программного средства

Моделирование предметной области – процесс, направленный на изучение и представление ключевых аспектов изучаемой предметной области, которые необходимо учитывать при разработке программного обеспечения. Цель моделирования – это создание абстрактного представления системы, позволяющее понять структуру, связи и процессы в рамках конкретной задачи. Данный этап позволяет лучше понять требования к приложению и определить его структуру и функциональность. При моделировании предметной области разрабатываются концептуальные модели, которые формируют основу для архитектуры программного обеспечения, пользовательских интерфейсов и механики взаимодействия.

Предметная область приложения для учёта и анализа персональных расходов представляет собой комплекс взаимосвязанных процессов, связанных с управлением личными финансами пользователя. Она включает взаимодействие таких элементов, как доходы, расходы, категории затрат, бюджеты, валюты и аналитические отчёты. Моделирование предметной области финансового приложения требует учёта взаимосвязей между этими компонентами и построения системы, которая позволит пользователю эффективно контролировать своё финансовое состояние.

Основная задача приложения заключается в упрощении и автоматизации учёта личных финансов, предоставляя пользователю удобный цифровой инструмент для регистрации доходов и расходов, анализа финансовых потоков и планирования бюджета. Приложение должно не только фиксировать данные, но и визуализировать их, помогая пользователю принимать более осознанные финансовые решения.

Для начала моделирования предметной области определяются основные сущности, с которыми взаимодействует пользователь. Ключевыми элементами являются:

- пользователь, который взаимодействует с системой, вводит данные и получает результаты анализа;
- финансовые операции – отдельные записи о доходах и расходах, содержащие сумму, категорию, дату, валюту и комментарий;
- категории – группы операций (например, питание, транспорт, развлечения), позволяющие структурировать расходы;
- бюджет – установленный пользователем лимит средств на определённый период или категорию;
- аналитические отчёты и диаграммы, отображающие статистику и динамику финансов;
- валюты – используемые денежные единицы, с возможностью пересчёта и ведения мультивалютного учёта.

Пользователь является центральной сущностью предметной области. Именно он инициирует все основные действия: добавление и редактирование операций, настройку бюджета, просмотр отчётов и анализ своих расходов. Система, в свою очередь, обеспечивает надёжное хранение данных, визуализацию информации и уведомления о превышении установленных лимитов.

Такое моделирование предметной области позволяет определить структуру данных и взаимосвязи между сущностями, что является основой для дальнейшего проектирования архитектуры приложения. В результате создаётся программное средство, которое помогает пользователю управлять личными финансами быстро, удобно и безопасно.

Ещё одной важной частью предметной области является финансовая структура пользователя, включающая категории расходов, источники доходов, валюты и бюджеты. Эти сущности взаимодействуют между собой, формируя общую картину финансового состояния. Моделирование этой структуры позволяет создать систему, которая не только хранит данные, но и помогает пользователю анализировать их и принимать решения на основе полученной информации.

Категории расходов и доходов являются ключевыми элементами системы. Каждая операция в приложении должна быть отнесена к определённой категории (например, питание, транспорт, жильё, развлечения), что обеспечивает структурирование данных и возможность последующего анализа. Моделирование категорий предусматривает возможность их добавления, редактирования и удаления пользователем, а также использование предустановленных шаблонов для удобства работы.

Неотъемлемым элементом является и система бюджета, которая позволяет пользователю устанавливать финансовые лимиты на определённый период или категорию. Такая функция помогает контролировать траты и своевременно получать уведомления о превышении установленных ограничений. Реализация этой сущности требует учёта взаимосвязей между бюджетом, категориями и операциями, а также формирования отчётов о выполнении бюджета.

Важную роль играет работа с валютами. Пользователь может вести учёт в разных валютах, что особенно актуально для людей, получающих доход или совершающих покупки в иностранных денежных единицах. Моделирование валютной системы должно предусматривать хранение курса обмена и возможность пересчёта сумм, что позволит формировать корректные отчёты и аналитику.

Одним из ключевых компонентов является аналитический модуль, который формирует отчёты и визуализирует данные в виде графиков и диаграмм. Он обеспечивает пользователю наглядное представление финансовой информации, распределение расходов по категориям и динамику изменений за различные периоды. Такая визуализация помогает лучше понимать структуру расходов и доходов, выявлять тенденции и

принимать более обоснованные финансовые решения.

После определения основных сущностей и связей между ними осуществляется логическое моделирование – процесс построения абстрактной структуры данных, которая отражает взаимосвязи между пользователем, операциями, категориями, валютами и бюджетами. Логическое моделирование позволяет определить, какие данные необходимо хранить, как они связаны между собой и каким образом будут использоваться в работе приложения.

Цель логического моделирования – создание универсальной и понятной модели данных, которая станет основой для проектирования архитектуры и разработки приложения. Такая модель должна обеспечивать целостность, удобство обновления и расширяемость данных, что особенно важно при создании кроссплатформенного финансового приложения с возможностью дальнейшего масштабирования и добавления новых функций.

Для визуализации результатов логического моделирования обычно используются диаграммы на языке UML (Unified Modeling Language). Первично происходит разработка диаграммы вариантов использования, которая представляет из себя графическое представление модели данных.

Диаграмма вариантов использования (use case) – диаграмма, отражающая отношения между акторами и прецедентами и являющаяся составной частью модели прецедентов, позволяющей описать систему на концептуальном уровне.

Актор (актёр) – множество логически связанных ролей, исполняемых при взаимодействии с прецедентами или сущностями (система, подсистема или класс). Актором может быть человек или другая система, подсистема или класс, которые представляют нечто вне сущности.

Прецедент – спецификация последовательностей действий в унифицированном языке моделирования, которые может осуществлять система, подсистема или класс, взаимодействуя с внешними действующими лицами.

Основное назначение диаграммы – описание функциональности и поведения, позволяющие заказчику, конечному пользователю и разработчику совместно обсуждать проектируемую систему.

Диаграмма вариантов использования представлена на рисунке 2.1, в более удобном виде см. ГУИР.281071.001 ПЛ.

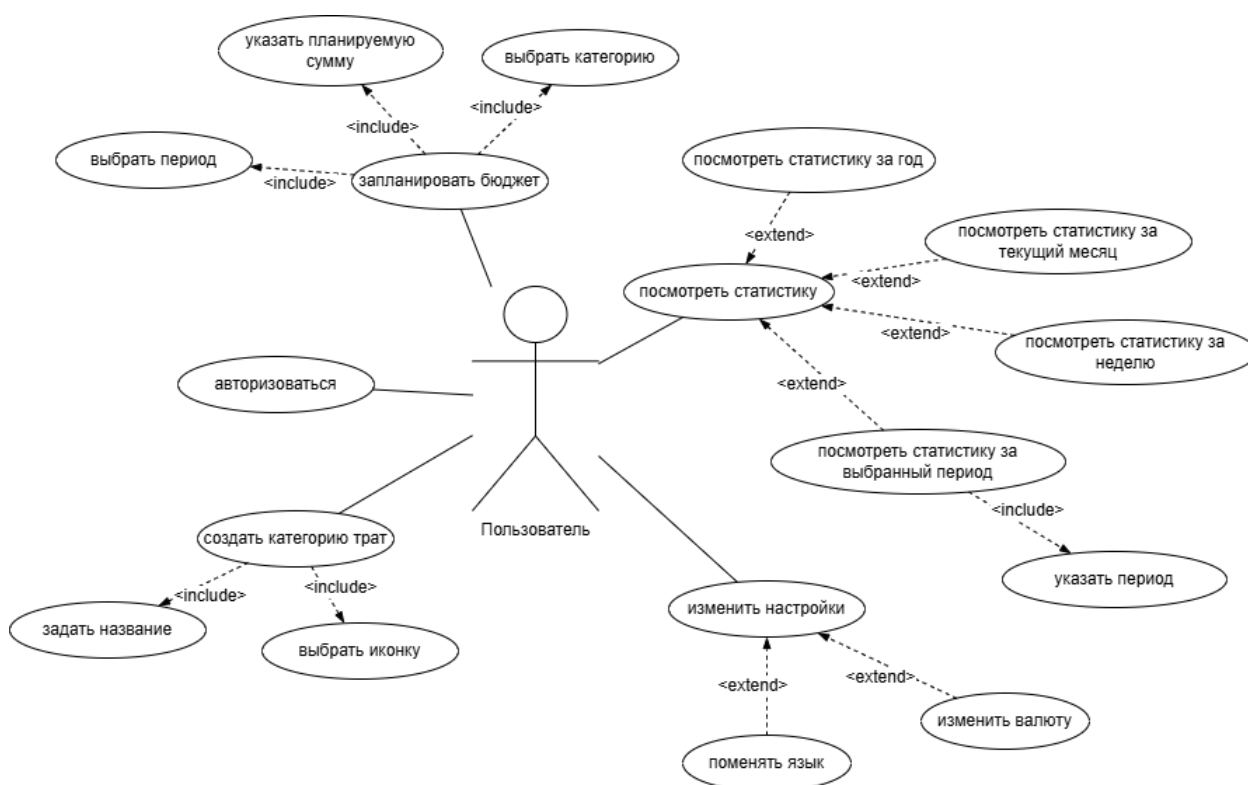


Рисунок 2.1 – Диаграмма вариантов использования

Основным актором системы является пользователь, который взаимодействует с приложением для ведения личного бюджета, анализа расходов и контроля финансовых показателей.

Диаграмма включает следующие ключевые варианты использования:

- авторизация в системе – пользователь проходит аутентификацию с использованием pin– кода, пароля или биометрических данных для получения доступа к персональным данным;
- добавление записи о расходах или доходах – пользователь вводит сумму, категорию, дату и валюту операции;
- редактирование и удаление операций – пользователь может изменять или удалять ранее добавленные записи;
- просмотр истории операций – система отображает список всех транзакций с возможностью фильтрации по категориям, дате и типу операции;
- просмотр аналитических отчётов, пользователь может анализировать свои расходы и доходы с помощью диаграмм и графиков;
- настройка бюджета – пользователь устанавливает лимит расходов на определённый период или категорию;
- получение уведомлений – система информирует пользователя о превышении бюджета или приближении к установленным лимитам;
- настройка категорий, пользователь может добавлять и редактировать категории расходов и доходов;
- управление валютами – система позволяет работать с несколькими

валютами и автоматически пересчитывать суммы при необходимости;

– изменение настроек приложения – пользователь может изменять язык интерфейса, тему оформления и параметры безопасности.

Все варианты использования связаны с пользователем, так как именно он инициирует выполнение всех ключевых действий в системе. Вариант «Авторизация в системе» является обязательным условием для доступа к остальным функциям приложения, обеспечивая безопасность данных и персонализацию пользовательского интерфейса.

Построенная диаграмма вариантов использования позволяет наглядно представить основные сценарии взаимодействия пользователя с системой, выделить приоритетные функции для реализации и определить границы взаимодействия между пользователем и приложением. Она служит основой для проектирования интерфейса, архитектуры и дальнейшего описания функциональных требований.

Диаграмма деятельности (Activity Diagram) используется для моделирования динамических аспектов работы программной системы и отображает последовательность действий, выполняемых пользователем и системой. В отличие от диаграмм классов, которые описывают структуру, диаграмма деятельности показывает поток операций и взаимосвязи между ними. Это делает её эффективным инструментом для проектирования логики взаимодействия в мобильных приложениях.

В рамках разработки приложения учёта и анализа персональных расходов диаграмма деятельности позволяет наглядно представить последовательность действий пользователя при работе с системой – от авторизации и добавления финансовых операций до формирования аналитических отчётов и получения уведомлений. Такое моделирование помогает понять логику взаимодействия между компонентами приложения, определить зависимость между процессами и выявить возможные точки улучшения интерфейса.

Диаграмма деятельности также полезна для анализа различных сценариев использования приложения. Она позволяет отразить ветвления логики, например, когда пользователь превышает лимит бюджета и система генерирует уведомление, или когда операция сохраняется локально без подключения к сети. Моделирование таких процессов помогает обеспечить устойчивость и предсказуемость поведения приложения при разных условиях.

Кроме того, диаграмма помогает определить возможные узкие места и уточнить требования к функциональности приложения. Благодаря пошаговому отображению процессов можно заранее выявить логические несоответствия и устранить их до этапа программной реализации. Диаграмма деятельности является важным инструментом при проектировании кроссплатформенного мобильного приложения, так как позволяет описать потоки действий пользователя, взаимодействие модулей и логику обработки данных. Применение диаграммы деятельности на этапе проектирования

помогает избежать ошибок, выявить узкие места и создать эффективную архитектуру программной системы.

Разработанная для приложения диаграмма деятельности представлена на рисунке 2.2, в более удобном виде см. ГУИР. 281071.002 ПЛ.

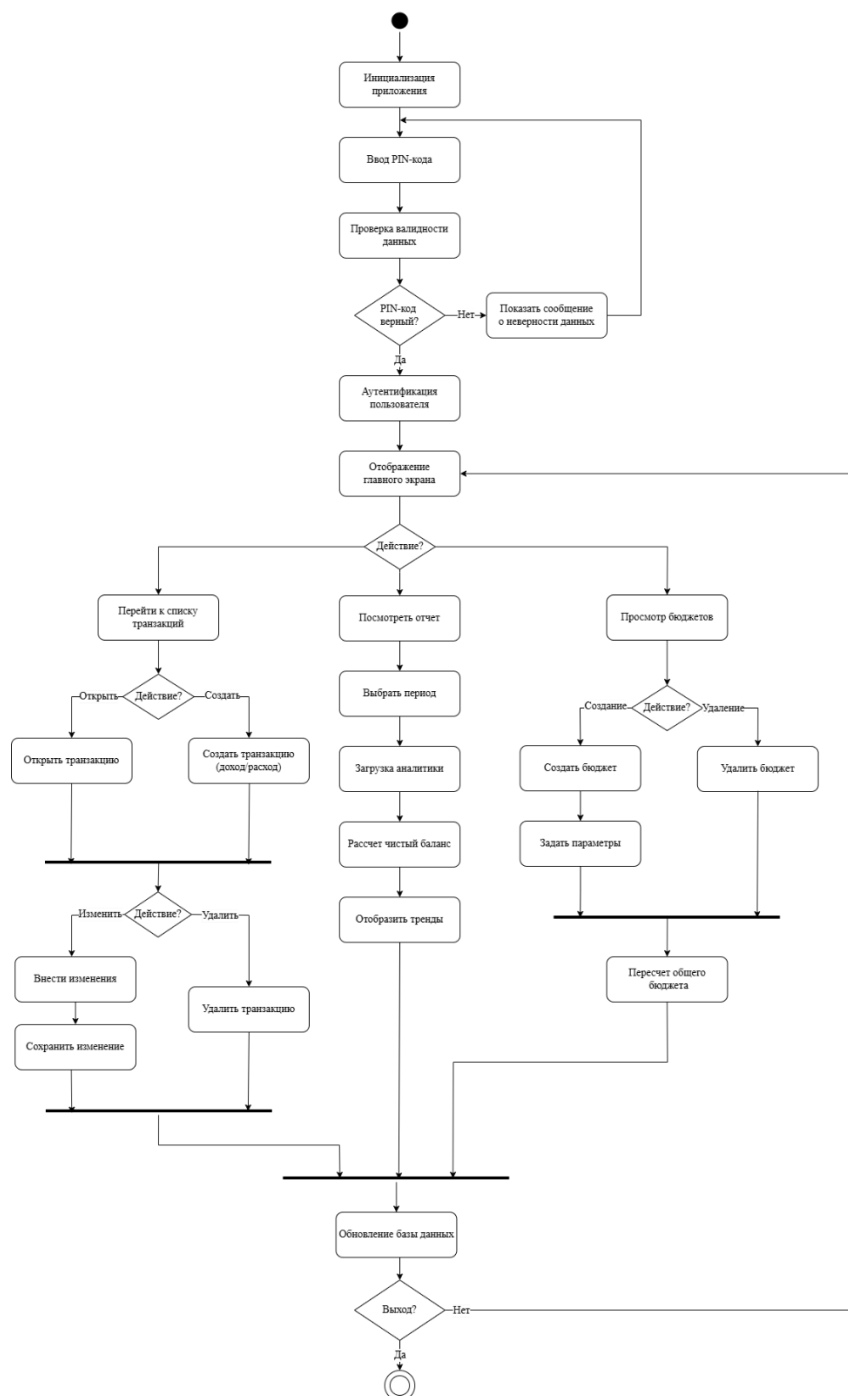


Рисунок 2.2 – Диаграмма деятельности

После запуска приложения происходит инициализация основных компонентов системы – загрузка интерфейса, проверка сохранённых данных и настроек пользователя, а также инициализация локальной базы данных. Если пользователь ранее авторизовался, выполняется автоматический вход, в

противном случае предлагается ввод PIN– кода или использование биометрической аутентификации.

После успешной аутентификации пользователь попадает на главный экран, где отображается текущий баланс, статистика расходов и доходов, а также краткий обзор финансового состояния за выбранный период. На этом этапе поток действий может развиваться в нескольких направлениях в зависимости от выбора пользователя.

При добавлении новой финансовой операции система открывает форму ввода, где указываются тип операции (доход или расход), сумма, категория, дата и комментарий. После подтверждения данные сохраняются в локальное хранилище (например, Hive или SQLite), а общий баланс пересчитывается. Пользователь может редактировать или удалять существующие операции, после чего приложение обновляет визуализацию данных и возвращает пользователя на главный экран.

Если выбирается раздел аналитики, приложение инициализирует модуль визуализации данных и отображает статистику расходов по категориям в виде круговой диаграммы, а также динамику доходов и расходов за различные периоды. Пользователь может изменять диапазон анализа, фильтровать данные по категориям или экспортировать отчёт. После завершения просмотра управление возвращается на главный экран.

Также доступен раздел настроек, где можно изменить валюту, задать лимиты бюджета, выбрать способ аутентификации или включить уведомления о превышении расходов. Все изменения сохраняются локально и применяются сразу без необходимости перезапуска приложения.

Диаграмма деятельности отражает ключевые сценарии взаимодействия пользователя с системой: запуск и инициализацию, аутентификацию, добавление и редактирование операций, анализ данных и настройку параметров. Каждый из процессов завершает свой цикл, возвращая пользователя к обновлённому главному экрану с актуальной финансовой информацией.

2.2 Разработка функциональной спецификации требований к программному средству

По завершению моделирования предметной области необходимо выделить функциональные требования, предъявляемые к разрабатываемому программному средству. Функциональные требования (functional requirements) – описание требуемого поведения системы в определенных условиях. Функциональные требования определяют, каким должно быть поведение продукта в тех или иных условиях. Они определяют, что разработчики должны создать, чтобы пользователи смогли выполнить свои задачи (пользовательские требования) в рамках бизнес– требований.

Мобильное приложение для учёта и анализа персональных расходов предназначено для автоматизации процессов управления личными

финансами. Оно помогает пользователю фиксировать доходы и расходы, контролировать бюджет и получать наглядную аналитику.

Основная цель разработки – создание простого, интуитивно понятного и кроссплатформенного инструмента, обеспечивающего удобный учёт финансов с возможностью анализа и планирования расходов.

Функциональные требования:

- учёт финансовых операций – приложение должно позволять пользователю добавлять, редактировать и удалять записи о расходах и доходах с указанием суммы, категории, даты и комментария;

- категоризация и аналитика – все операции должны автоматически распределяться по категориям, а пользователь должен иметь возможность просматривать расходы в виде диаграмм и графиков по выбранному периоду;

- бюджетирование. пользователь может устанавливать лимиты по категориям или общему бюджету и получать уведомления при их превышении;

- мультивалютность – приложение должно поддерживать выбор валюты и отображение конвертации при необходимости;

- уведомления. реализуется возможность отправки напоминаний о превышении бюджета, регулярных платежах или необходимости внесения новых данных;

- безопасность данных – для защиты информации предусматривается авторизация с использованием pin– кода, пароля или биометрической аутентификации (face/touch id). данные пользователей хранятся в зашифрованном виде с использованием алгоритма sha– 256;

- работа без подключения к сети– приложение должно сохранять данные локально и синхронизироваться с сервером при наличии интернет–соединения;

- многоязычность интерфейса – интерфейс приложения должен поддерживать минимум два языка – русский и английский;

- экспорт данных – пользователь должен иметь возможность экспортировать отчёты в виде таблиц или графиков для последующего анализа.

Приложение должно быть совместимо с операционными системами Android 8.0+ и iOS 13+, обеспечивать быстрое время отклика (не более 1 секунды при взаимодействии с интерфейсом) и корректно отображаться на экранах разных размеров. Интерфейс должен быть простым, логичным и визуально аккуратным.

Хранение данных реализуется с использованием Hive или SQLite, визуализация аналитики – на базе библиотеки Syncfusion Charts. Архитектура приложения строится по принципам Clean Architecture с использованием шаблона BLoC для управления состоянием.

Разработанные требования позволяют чётко определить объём и качество создаваемого программного обеспечения, а также установить критерии его готовности к использованию.

Данные функциональные требования обеспечивают основной функционал приложения для учета персональных финансов, и позволяет четко определить готовность и качество разрабатываемого приложения.

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Проектирование архитектуры программного средства

Проектирование архитектуры программного средства направлено на определение структуры, функционального назначения и взаимосвязей между ключевыми компонентами системы. Архитектура мобильного приложения для учёта и анализа персональных расходов основана на принципах модульности, масштабируемости, разделения ответственности и независимости слоёв. Такой подход обеспечивает гибкость системы, её устойчивость к изменениям и возможность дальнейшего развития.

Программное средство использует многослойную архитектуру с чётким разграничением уровней пользовательского интерфейса, бизнес – логики, управления состоянием, доступа к данным и интеграции с внешними сервисами. В рамках проектирования были выделены следующие основные компоненты системы:

Пользовательский интерфейс (UI слой) реализован средствами Flutter и содержит набор экранов, обеспечивающих визуализацию данных, навигацию по приложению и взаимодействие пользователя с системой. Интерфейс включает экраны транзакций, аналитики, бюджетов, настроек и экран добавления транзакций. UI работает в связке со слоем BLoC и получает из него уже подготовленное состояние.

Слой бизнес– логики и управления состоянием (BLoC) отвечает за обработку событий, поступающих от пользовательского интерфейса, реализацию сценариев работы приложения и подготовку данных для отображения. Каждый функциональный модуль приложения представлен отдельными BLoC–компонентами: для транзакций, аналитики, бюджетов, пользователей и настроек. Такой подход обеспечивает изоляцию логики и её независимость от интерфейса.

Слой данных и репозитория служит промежуточным звеном между бизнес–логикой и источниками данных. Репозитории инкапсулируют операции чтения, записи, фильтрации, поиск и преобразования данных, обеспечивая единый интерфейс для BLoC–компонентов. Это позволяет гибко изменять внутренние механизмы сохранения данных без влияния на остальные уровни системы.

Локальная база данных построена на основе Hive и SQLite и используется для хранения транзакций, категорий, бюджетов, настроек пользователя, курсов валют и информации о повторяющихся операциях. Hive обеспечивает высокую скорость работы с объектами, а SQLite применяется для выборки аналитики и агрегирующих запросов. Такое сочетание позволяет достичь максимальной эффективности при работе с разнородными данными.

Сервис аналитики обеспечивает выполнение агрегирующих

запросов, формирование статистики по доходам, расходам, категориям и временным периодам. Аналитический модуль использует локальную базу данных для выполнения выборок и предоставляет BLoC– компонентам обобщённые данные для построения графиков и отчётов.

Сервис повторяющихся транзакций отвечает за обработку периодических операций. Он рассчитывает даты следующих списаний, формирует новые транзакции согласно заданному интервалу и обеспечивает корректную синхронизацию с остальными модулями системы.

Сервис синхронизации используется для обновления курсов валют и обмена данными с внешними источниками. При наличии сети он взаимодействует с удалённым API, обновляет курсы валют, а также при необходимости может участвовать в синхронизации пользовательских данных.

Модуль аутентификации и безопасности включает механизмы локальной защиты данных пользователя: установку PIN– кода, биометрическую аутентификацию, хранение конфиденциальной информации в защищённом хранилище и контроль доступа к чувствительным разделам приложения.

Взаимодействие компонентов осуществляется через чётко определённые внутренние интерфейсы: UI взаимодействует с BLoC, бизнес– логика – с репозиториями, а те – с локальным хранилищем и сетевыми сервисами. Обмен данными между модулями производится в формате JSON, а при работе с внешними ресурсами – посредством HTTPS– запросов.

Проектирование архитектуры обеспечивает модульную, гибкую и устойчивую структуру программного средства, которая поддерживает расширение функциональности, удобство сопровождения и эффективное выполнение всех пользовательских задач.

3.2 Проектирование и разработка алгоритмов работы программы

Проектирование и разработка алгоритмов работы приложения для учёта и анализа персональных расходов является ключевым этапом создания надёжного и эффективного программного продукта. Логика работы системы должна обеспечивать корректное взаимодействие между пользователем, базой данных и аналитическими модулями, обеспечивая стабильность, точность расчётов и удобство работы.

Алгоритмы определяют, как приложение реагирует на действия пользователя: добавление, редактирование или удаление финансовых операций, настройку бюджета, фильтрацию данных и построение аналитических отчётов. Правильное проектирование последовательности этих процессов позволяет обеспечить согласованность данных, точность

отображаемой информации и предсказуемое поведение приложения.

Разработка алгоритмов включает описание логики взаимодействия между компонентами приложения: пользовательским интерфейсом (UI), бизнес– логикой (BLoC), системой хранения данных (Hive/SQLite) и модулем визуализации аналитики. Каждый этап должен выполняться последовательно – от получения данных до их анализа и отображения пользователю.

Основные задачи проектирования алгоритмов в рамках создания приложения заключаются не только в реализации отдельных функций, но и в обеспечении их согласованной и надёжной работы. В первую очередь необходимо обеспечить корректную обработку пользовательских данных: каждая транзакция, бюджет или настройка должны сохраняться безошибочно и учитываться в общих расчётах.

Также важным аспектом является разработка устойчивого и предсказуемого механизма обновления и хранения информации. Алгоритмы должны гарантировать, что данные остаются актуальными, даже при возможных сбоях сети, перезапуске приложения или изменении пользовательских параметров.

Отдельное внимание уделяется автоматизации расчётов – бюджеты, аналитические показатели, тренды расходов и доходов должны формироваться автоматически, без участия пользователя. Это позволяет значительно повысить удобство эксплуатации и уменьшить риск ошибок в ручных вычислениях.

Не менее важной задачей является реализация алгоритмов формирования визуальных отчётов и уведомлений. Система должна не только отображать графики и статистику в понятном виде, но и своевременно предупреждать пользователя о превышении лимитов или приближении к установленным бюджетным границам.

Итогом проектирования становится комплекс взаимосвязанных алгоритмов, которые обеспечивают целостность данных, удобное и интуитивное взаимодействие пользователя с приложением, а также стабильную и надёжную работу системы при различных сценариях использования.

На рисунке 3.1 алгоритм отражает общую логику функционирования кроссплатформенного мобильного приложения для учёта и анализа персональных расходов, демонстрируя последовательность основных действий пользователя и внутренних процессов системы. Он позволяет понять, как именно приложение взаимодействует с пользователем, обрабатывает данные и обеспечивает их сохранность и анализ.

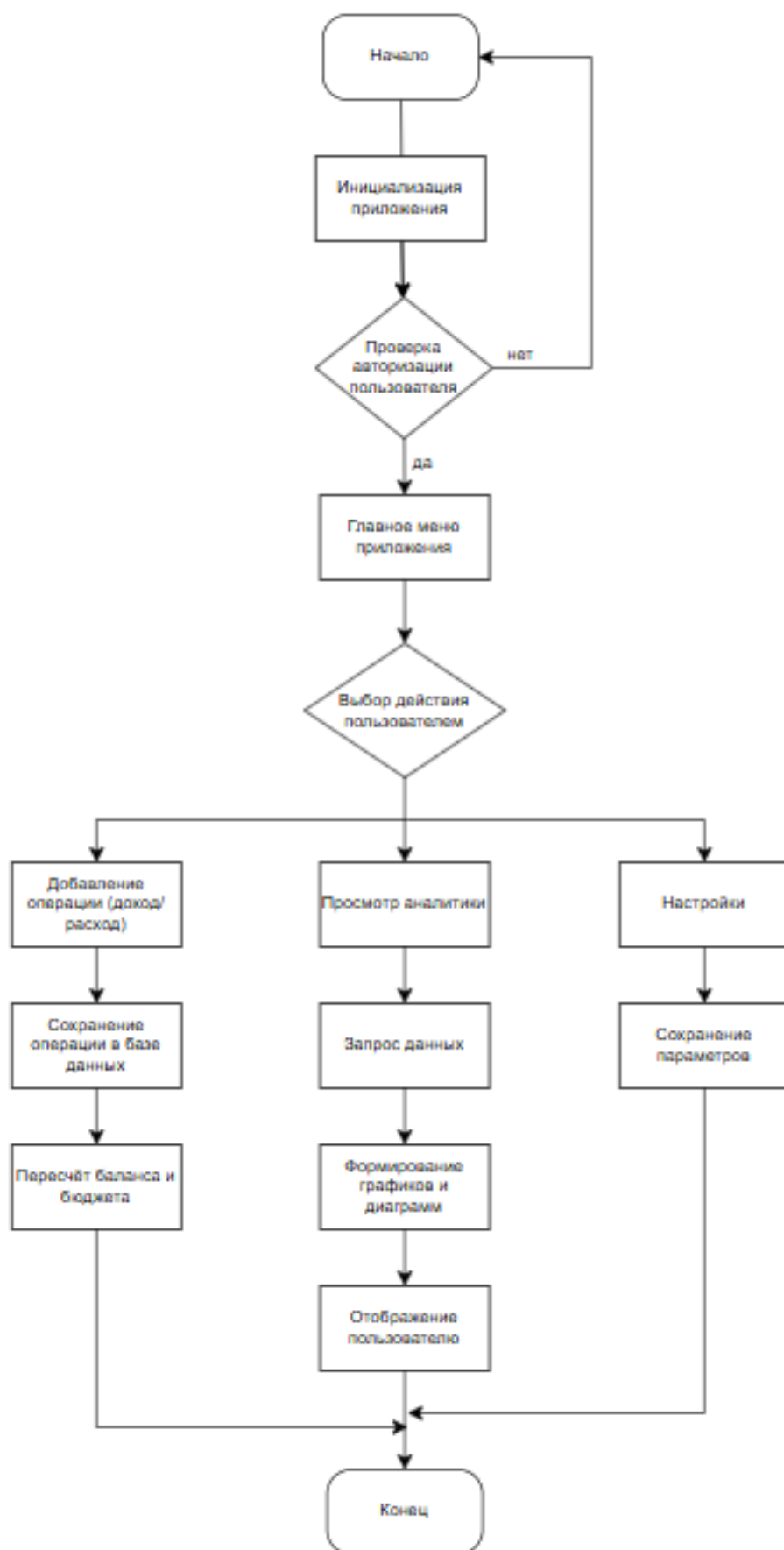


Рисунок 3.1 – Общий алгоритм работы приложения

После запуска мобильного приложения выполняется поэтапная инициализация всех ключевых компонентов системы. На данном этапе происходит загрузка пользовательских настроек, проверка целостности и доступности локальной базы данных, а также подготовка пользовательского интерфейса к дальнейшему взаимодействию. Эти действия обеспечивают стабильную работу приложения и корректную обработку пользовательских данных на протяжении всей сессии.

Для обеспечения точного учёта персональных доходов и расходов, а также для поддержания актуального состояния бюджетов в приложении реализован комплексный алгоритм добавления финансовых транзакций. Данный алгоритм является универсальным и применяется как для операций дохода, так и для операций расхода. В ходе его выполнения осуществляется проверка корректности и полноты вводимых пользователем данных, включая сумму операции, выбранную категорию, дату и тип транзакции. После успешной валидации информация сохраняется в локальной базе данных, что гарантирует целостность и сохранность финансовой истории пользователя.

Следующим этапом работы алгоритма является перерасчёт аналитических показателей. На основе добавленной транзакции обновляются агрегированные данные, используемые для построения отчётов, графиков и аналитических сводок. Параллельно выполняется перерасчёт общего бюджета пользователя, а также бюджетов, заданных для отдельных категорий расходов. В случае если добавленная транзакция приводит к превышению установленного лимита, система фиксирует данный факт и помечает соответствующий бюджет как превышенный. При этом транзакция сохраняется без изменений, что позволяет избежать потери данных и сохранить достоверность финансовых записей.

Дополнительно алгоритм предусматривает формирование уведомлений для пользователя о превышении бюджетных ограничений, что способствует повышению финансовой осознанности и позволяет оперативно реагировать на изменения в структуре расходов. Завершающим этапом является обновление аналитических данных, обеспечивающее актуальность отчётов и визуальных представлений финансовой информации.

Последовательность выполнения всех этапов алгоритма, а также логика принятия решений на ключевых шагах процесса представлены на рисунке 3.2.

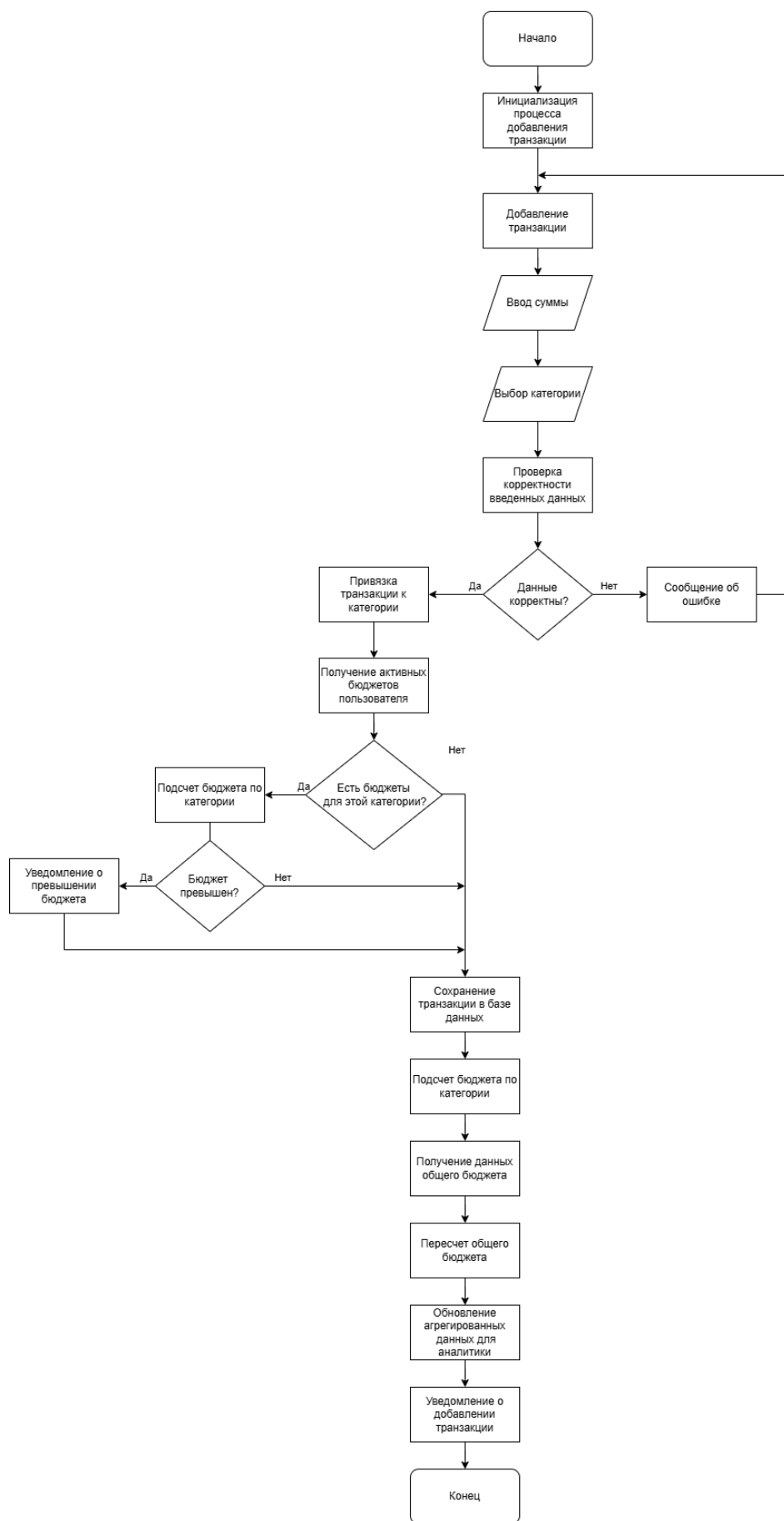


Рисунок 3.2 – Алгоритм добавления транзакции и пересчет бюджета

Процесс начинается с инициации авторизации, когда пользователь вводит свои учетные данные, включая имя пользователя и пароль. Система выполняет проверку корректности введенного PIN- кода. В случае успешной проверки пользователь получает доступ к системе, и процесс завершается. Если же пароль некорректен, система выводит сообщение об ошибке и прерывает процесс авторизации.

Алгоритм построен таким образом, чтобы минимизировать риски несанкционированного доступа, обеспечивая надежную проверку учетных данных и безопасный вход в систему.

После успешной аутентификации пользователь попадает в главное меню, которое служит центральной точкой доступа ко всем основным функциям приложения. Здесь доступны следующие разделы:

- добавление операции – позволяет пользователю внести новую запись о доходе или расходе, указав категорию, сумму, дату и комментарий. после ввода данные сохраняются в локальной базе (например, `sqlite` или `hive`), что обеспечивает мгновенный доступ без необходимости подключения к сети.

- просмотр аналитики – пользователь может просмотреть финансовую статистику в виде диаграмм и графиков (например, с использованием библиотеки `syncfusion charts`). данный модуль позволяет анализировать структуру расходов, определять наиболее затратные категории и отслеживать динамику бюджета.

- настройки – предоставляет возможность изменения параметров приложения, таких как валюта, лимиты бюджета, способы аутентификации и выбор темы интерфейса.

Каждая операция, добавленная пользователем, проходит проверку на превышение установленных лимитов бюджета. Если лимит по категории или общему бюджету превышен, приложение уведомляет пользователя о необходимости корректировки расходов.

После внесения данных система проверяет наличие интернет-соединения. При его наличии выполняется синхронизация данных с сервером, что обеспечивает резервное копирование информации и возможность восстановления данных при смене устройства. В случае отсутствия сети данные сохраняются локально и будут синхронизированы автоматически при следующем подключении.

В завершении цикла работы, после выполнения всех необходимых действий – добавления операции, просмотра аналитики или изменения настроек – пользователь может завершить сеанс, и система корректно выходит из активного состояния, сохраняя все изменения.

Такой алгоритм обеспечивает логичную, интуитивную и безопасную последовательность действий, направленную на удобное управление личными финансами. Он гарантирует корректность работы приложения, целостность данных и стабильное взаимодействие между интерфейсом, бизнес-логикой и хранилищем данных.

Для обеспечения корректной обработки финансовых данных и

формирования аналитической отчётности в мобильном приложении важным этапом является проектирование алгоритмов агрегации расходов по категориям (рисунок 3.3). Такой алгоритм позволяет автоматически объединять все транзакции пользователя, относящиеся к определённым категориям, вычисляя суммарные значения расходов за выбранный период. Реализация данной логики необходима для построения диаграмм, аналитических сводок, отображения «топ» категорий расходов и формирования общей картины финансового поведения пользователя.

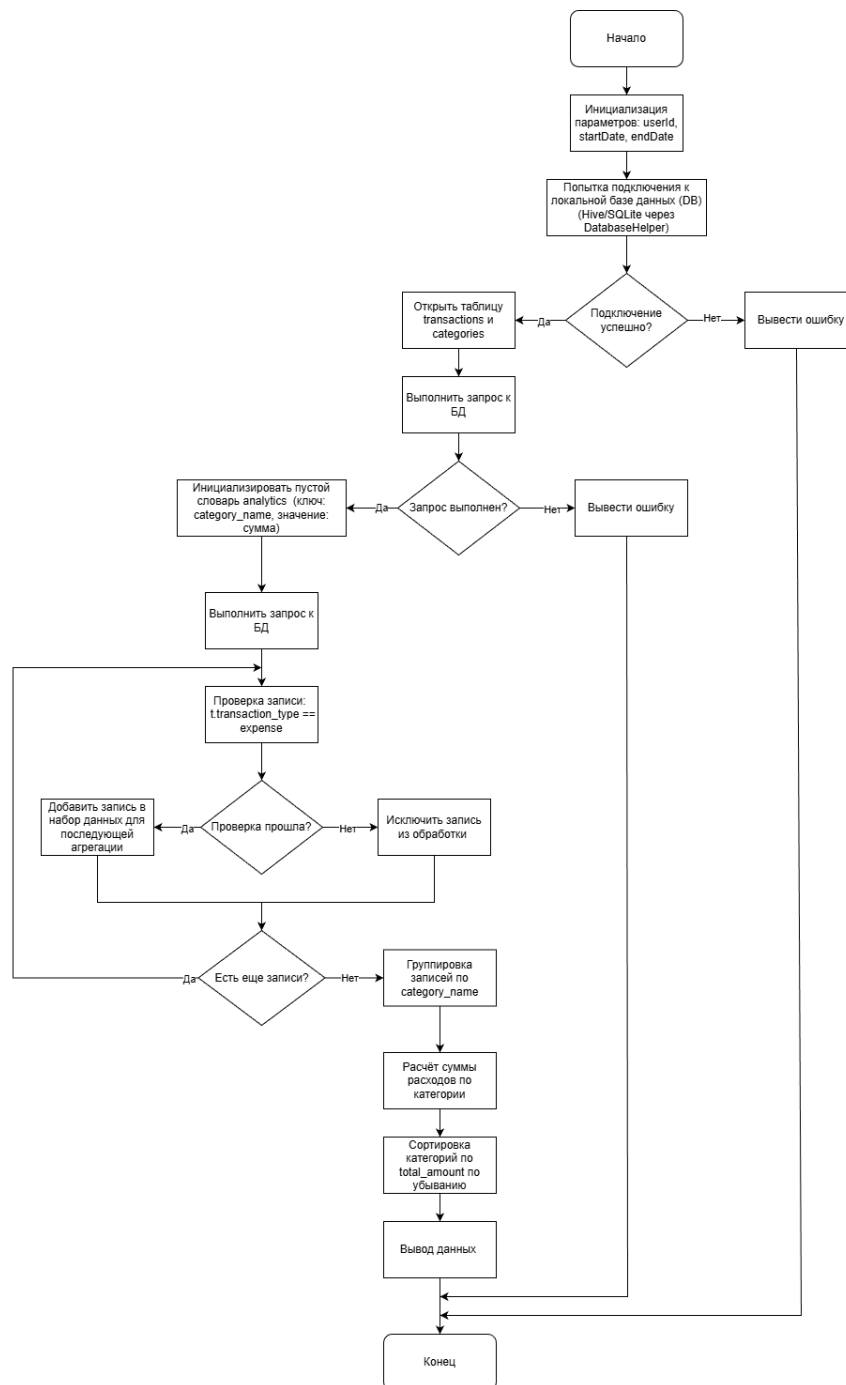


Рисунок 3.2 – Алгоритм агрегации по категориям

Алгоритм агрегации основан на обращении к локальной базе данных, выборке всех транзакций типа «расход», их группировке по категориям и вычислении итоговых сумм. На этапе проектирования важно определить последовательность выполнения всех операций: от инициализации параметров и проверки доступности базы данных до обработки результата SQL-запроса, формирования структурированных данных и передачи итоговой информации в модуль аналитики. Блок-схема данного алгоритма демонстрирует логику обработки данных, ветвления, проверок и циклов, обеспечивая наглядность работы механизма анализа расходов внутри приложения.

Представленные алгоритмы формируют ключевую функциональность мобильного приложения, обеспечивая корректное добавление, обработку, поиск и анализ финансовых данных пользователя. Они отвечают за точность формирования отчётов, стабильность работы модулей аналитики, корректное управление бюджетами и безопасную авторизацию. Реализация данных алгоритмов позволяет обеспечить удобство использования, надёжность хранения информации и высокий уровень защиты персональных данных, что является критически важным для финансовых приложений. Диаграммы деятельности наглядно отображают логику работы каждого алгоритма, демонстрируя последовательность операций, условия ветвления и возможные результаты выполнения процессов.

3.3 Проектирование интерфейса приложения

Проектирование пользовательского интерфейса является одним из ключевых этапов разработки мобильного приложения для учёта и анализа персональных расходов. Интерфейс обеспечивает взаимодействие пользователя с функциональной частью программы, поэтому от его качества напрямую зависит удобство, скорость и точность работы с приложением.

Основная цель проектирования интерфейса – создание понятной и интуитивной системы управления, которая позволит пользователю без труда вести учёт доходов и расходов, просматривать статистику и анализировать свои финансовые привычки. Удобный интерфейс делает процесс работы с приложением не только эффективным, но и приятным, повышая вовлечённость и лояльность пользователей.

При разработке интерфейса были учтены особенности целевой аудитории: пользователи, ведущие личный или семейный бюджет, предпочитают простые решения без лишних функций, с акцентом на наглядность данных. Поэтому основной упор сделан на визуальную структуру, минимализм и интуитивность интерфейса.

Перед проектированием интерфейса был проведён анализ аналогичных приложений для определения удачных дизайнерских решений и выявления их недостатков. На основе этого анализа были выработаны следующие требования к интерфейсу создаваемого приложения:

- пользователь должен с первого запуска понимать, как добавлять операции, просматривать категории и анализировать траты. все основные действия должны выполняться в один–два шага;
- основные финансовые показатели должны быть представлены в виде графиков и диаграмм, позволяющих быстро оценить структуру расходов;
- интерфейс должен быть адаптирован под экраны мобильных устройств, с рациональным размещением элементов и удобной навигацией;
- использование нейтральных цветов, читаемых шрифтов и лаконичных иконок делает приложение визуально приятным и не отвлекает от основной информации;
- интерфейс разрабатывается с учётом одинакового пользовательского опыта как на устройствах с android, так и с ios. использование фреймворка flutter позволяет добиться единого дизайна и поведения элементов интерфейса на обеих платформах.

Проектирование интерфейса позволило определить ключевые экраны приложения, такие как главная панель с балансом, страница добавления операции, анализ расходов по категориям, графики и диаграммы, а также раздел настроек и отчётов. Логика навигации построена по принципу минимального количества переходов между разделами, что обеспечивает плавное и логичное взаимодействие пользователя с системой.

Визуализация интерфейса на этапе проектирования позволила протестировать структуру и убедиться в удобстве взаимодействия. Это значительно снизило риски появления ошибок на стадии разработки и помогло создать эргономичный, современный и функциональный интерфейс, полностью соответствующий целям приложения – упрощение управления личными финансами.

После составления требований к интерфейсу приложения была проведена его разработка. На рисунке 3.4 изображен макет интерфейса главного экрана.



Рисунок 3.4 – Макет интерфейса главного экрана

Окно системы заметок будет состоять из основной информации и функций, кнопки «+» «-» и переходов на другие страницы. По нажатию на кнопку добавить будет создаваться новая запись транзакции. По нажатию на кнопки меню будет происходить переход на другие страницы. На рисунке 3.5 представлен макет интерфейса окна добавление транзакции.

Добавить операцию

+

добавить доход

-

добавить расход

сумма

еда

образование

развлечения

здоровье

транспорт

другое

главная	операции	отчеты	бюджет	настройки
---------	----------	--------	--------	-----------

Рисунок 3.5 – Макет интерфейса создания транзакции

Далее представлено окно с просмотром транзакций (рисунок 3.6).

Все транзакции

Все

еда

развлечения

здоровье

Все

доходы

расходы

кафе

5.44

парк

3.99

кино

21.45

главная	операции	отчеты	бюджет	настройки
---------	----------	--------	--------	-----------

Рисунок 3.6 – Макет интерфейса просмотра транзакций

И так же меню профиля с настройками (рисунок 3.7).

Настройки				
Pin-Code		изменить		
Язык русский		изменить		
Валюта бел.руб.		изменить		
Обновить курсы валют				
Включить уведомления				
главная	операции	отчеты	бюджет	настройки

Рисунок 3.7 – Макет интерфейса просмотра транзакций

Проектирование интерфейса играет важнейшую роль в процессе создания мобильного приложения для учёта и анализа личных финансов, формируя основу для дальнейших этапов разработки. На этом этапе определяется логика взаимодействия пользователя с системой, структура экранов и принципы отображения информации. Грамотное проектирование интерфейса помогает заранее продумать, как пользователь будет добавлять операции, просматривать расходы, анализировать статистику и управлять своими финансовыми данными.

Разработка интерфейсных решений на ранних стадиях позволяет избежать множества ошибок и неудобств, которые могли бы возникнуть при создании приложения без предварительного визуального планирования. Это также помогает выявить потенциальные трудности в навигации и взаимодействии с элементами управления, оптимизировать компоновку экранов и обеспечить логичную последовательность действий.

Проектирование интерфейса на данном этапе обеспечивает целостное представление о будущем продукте и позволяет создать удобную, интуитивно понятную систему, способствующую эффективному ведению личного бюджета и повышению финансовой грамотности пользователей.

3.4 Разработка модели базы данных

Опираясь на ранее проведённый анализ предметной области и функциональные требования, информационное обеспечение программного средства предлагается реализовать в виде реляционной базы данных, приведённой к третьей нормальной форме (3НФ).

Реляционная база данных (БД) – это совокупность взаимосвязанных таблиц, каждая из которых хранит структурированные данные об отдельных сущностях приложения. С точки зрения пользователя, база данных представляет собой набор связанных таблиц, в которых каждая запись (строка) соответствует отдельному объекту предметной области, а столбцы – его атрибутам.

Использование реляционной модели позволяет эффективно организовать хранение и обработку данных, исключить дублирование информации и обеспечить логическую целостность. Благодаря наличию общих ключевых полей таблицы связываются между собой, что позволяет объединять данные из разных таблиц с помощью запросов.

Связи между таблицами в реляционной базе данных могут быть различных типов:

- «один–к–одному» – каждой записи одной таблицы соответствует одна запись другой таблицы;
- «один–ко–многим» – одной записи главной таблицы соответствуют несколько записей подчинённой;
- «многие–ко–многим» – записи из обеих таблиц могут быть взаимно связаны между собой.

Приведение базы данных к третьей нормальной форме (3НФ) позволяет устранить избыточность данных и исключить аномалии при их обновлении. Согласно требованиям 3НФ, все неключевые атрибуты таблицы должны быть функционально независимы и зависеть только от первичного ключа.

В проектируемой базе данных предусмотрено разделение таблиц на справочные и операционные.

- справочные таблицы содержат информацию, используемую в качестве классификаторов или категорий (например, категории расходов и доходов);
- операционные таблицы включают данные, которые постоянно обновляются пользователем – записи о финансовых транзакциях, бюджетах, счетах и т. д.

В состав проектируемой базы данных входят следующие таблицы:

- categories – справочная таблица, содержащая перечень категорий доходов и расходов;
- transactions – операционная таблица, в которой фиксируются все финансовые операции пользователя;
- accounts – справочная таблица, в которой хранятся данные о

различных счетах пользователя (например, наличные, карта, депозит);

- budgets – таблица, содержащая сведения о планируемых лимитах и целях пользователя;

- statistics – вспомогательная таблица, в которой агрегируются аналитические данные для отображения отчётов и диаграмм.

Состав информации для операционных и справочных таблиц приведен далее в таблицах 3.1– 3.5.

Таблица 3.1 – Структура таблицы «Categories»

Наименование поля	Тип данных	Ключевое поле / Обязательное	Описание
Id	Integer	Да / Да	Уникальный идентификатор категории
Name	Text	Нет / Да	Название категории (например, «Продукты», «Зарплата»)
Type	Text	Нет / Да	Тип категории: income или expense

Таблица 3.2 – Структура таблицы «Accounts»

Наименование поля	Тип данных	Ключевое поле / Обязательное	Описание
id	INTEGER	Да / Да	Уникальный идентификатор счёта
name	TEXT	Нет / Да	Название счёта (например, «Кошелёк», «Карта»)
balance	REAL	Нет / Да	Текущий баланс счёта
currency	TEXT	Нет / Да	Валюта счёта (например, BYN, USD, EUR)
created_at	TEXT	Нет / Да	Дата создания счёта

Таблица 3.3 – Структура таблицы «Transactions»

Наименование поля	Тип данных	Ключевое поле / Обязательное	Описание
id	INTEGER	Да / Да	Уникальный идентификатор транзакции
account_id	INTEGER	Нет / Да	Ссылка на счёт, с которого произведена операция

Продолжение таблицы 3.3

category_id	INTEGER	Нет / Да	Ссылка на категорию операции
amount	REAL	Нет / Да	Сумма операции
type	TEXT	Нет / Да	Тип операции: income или expense
note	TEXT	Нет / Нет	Комментарий к операции
date	TEXT	Нет / Да	Дата и время совершения операции

Таблица 3.4 – Структура таблицы «Budgets»

Наименование поля	Тип данных	Ключевое поле / Обязательное	Описание
id	INTEGER	Да / Да	Уникальный идентификатор бюджета
category_id	INTEGER	Нет / Да	Ссылка на категорию, для которой установлен лимит
limit_amount	REAL	Нет / Да	Установленный лимит расходов
period_start	TEXT	Нет / Да	Дата начала бюджетного периода
period_end	TEXT	Нет / Да	Дата окончания бюджетного периода

Таблица 3.5 – Структура таблицы «Statistics»

Наименование поля	Тип данных	Ключевое поле / Обязательное	Описание
id	INTEGER	Да / Да	Уникальный идентификатор записи
category_id	INTEGER	Нет / Да	Ссылка на категорию
total_income	REAL	Нет / Нет	Общая сумма доходов по категории
total_expense	REAL	Нет / Нет	Общая сумма расходов по категории
period	TEXT	Нет / Да	Период, за который собрана статистика (месяц, неделя и т.д.)

Для обеспечения целостности данных и логической взаимосвязанности

таблицы объединены связями типа «один– ко– многим»:

- таблица categories связана с таблицей transactions по полю category_id;
- таблица accounts связана с таблицей transactions по полю account_id;
- таблица categories связана с таблицей budgets по полю category_id;
- таблица transactions связана с таблицей statistics по полю transaction_id.

Такое проектное решение обеспечивает логическую целостность данных, удобство выборки и гибкость при добавлении новых функций, например, учёта многовалютности или планирования расходов по периодам.

База данных разрабатываемого мобильного приложения реализована с использованием SQLite, встроенной системы управления базами данных, оптимизированной для мобильных устройств. SQLite не требует отдельного сервера, обеспечивает высокую скорость работы и надёжность хранения данных, что делает её идеальным выбором для мобильных приложений.

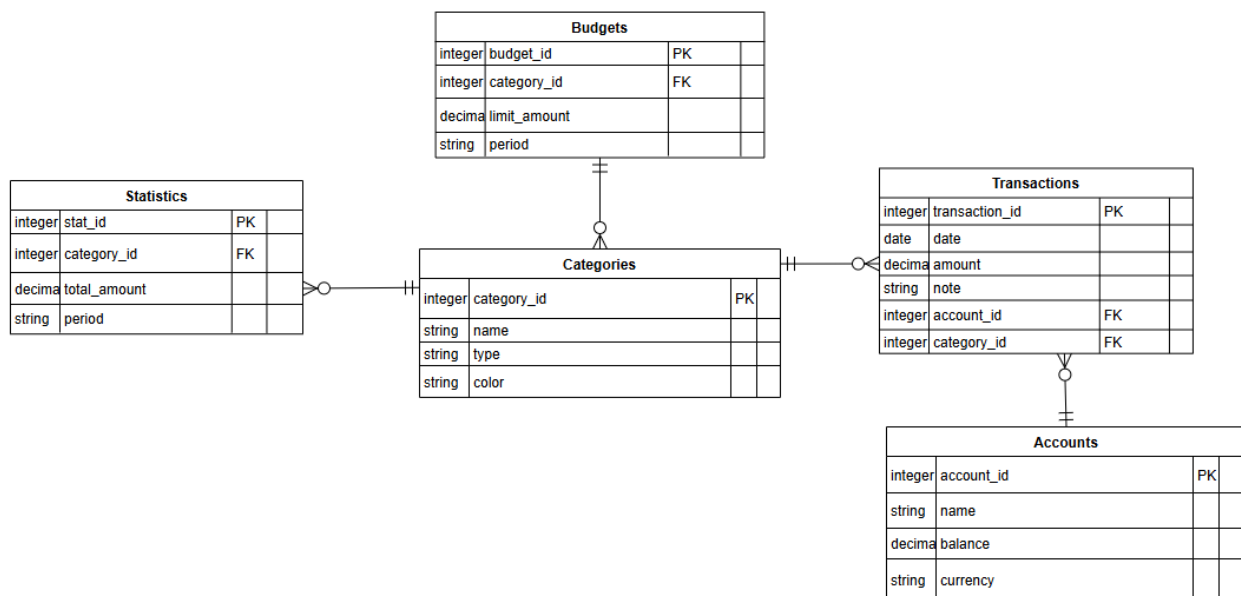


Рисунок 3.8 – Диаграмма базы данных

Для наглядного представления структуры базы данных была построена ER– диаграмма (Entity– Relationship Diagram), отражающая основные таблицы, их поля и взаимосвязи между ними. На рисунке 3.7 диаграмма демонстрирует логическую организацию данных приложения, взаимосвязь между категориями, транзакциями, счетами и бюджетами, а также показывает, как обеспечивается целостность и согласованность информации при работе системы.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

4.1 Выбор и обоснование видов тестирования

Тестирование представляет собой систематическую деятельность по проверке соответствия работы программного продукта предъявленным требованиям, поиску ошибок и подтверждению готовности приложения к эксплуатации. Для кроссплатформенного мобильного приложения учёта и анализа персональных расходов выбран комплекс видов тестирования, обеспечивающий проверку как функциональной полноты, так и надёжности, безопасности и удобства использования. Обоснование выбора типов тестирования обусловлено спецификой предметной области (работа с персональными финансовыми данными), технической архитектурой (клиент – Flutter, локальное хранилище и синхронизация с сервером) и требованиями к кроссплатформенности и офлайн– работоспособности.

Основная задача тестирования – выявление ошибок и несовершенств, в работе приложения. Тестирование включает несколько направлений, каждое из которых имеет свои задачи.

Функциональное тестирование направлено на проверку соответствия реализованного функционала требованиям технического задания. Необходимо убедиться, что все ключевые сценарии выполняются корректно: добавление/редактирование/удаление операций, присвоение категорий, учёт по счетам, установка и отслеживание бюджетов, формирование отчётов и диаграмм, экспорт/импорт данных. Тесты охватывают позитивные и негативные сценарии (корректные данные, пустые поля, некорректные форматы и др.). Выбор функционального тестирования обусловлен критической важностью корректности расчётов и учёта финансовых данных для конечного пользователя.

Интеграционное тестирование проверяет взаимодействие модулей приложения между собой и с внешними компонентами. Особое внимание уделяется корректности сериализации/десериализации данных, обработке конфликтов при синхронизации, консистентности при параллельных изменениях и работе механизмов отката при ошибках. Интеграционные проверки необходимы в силу многослойной архитектуры и наличия нескольких точек возможных ошибок при передаче данных.

Тестирование производительности и нагрузочное тестирование важно при формировании аналитики, построении диаграмм и массовой обработке операций (например, импорт большой истории транзакций). Необходимо оценить время отклика UI при запросах к локальной БД, время построения графиков, использование памяти и CPU на устройствах разного уровня. Нагрузочное тестирование (симуляция большого числа операций и длительной работы) позволит выявить узкие места и предотвратить зависания или утечки памяти. Для мобильного приложения критично обеспечить приемлемую производительность на типичных для

целевой аудитории устройствах.

UI/UX тестирование (интерфейсное тестирование) проверяет корректность отображения элементов интерфейса на различных экранах и разрешениях, удобство навигации, читаемость текстов, расположение элементов для управления одной рукой. Тесты включают проверку адаптивности макетов, корректности локализации и соответствия гайдлайнам платформ (Android / iOS). UI/UX тестирование направлено на снижение порога входа для пользователей и повышение удовлетворённости от использования.

Поскольку приложение обрабатывает конфиденциальную финансовую информацию, приоритетом является проверка механизмов аутентификации (PIN/биометрия), шифрования локальных данных (хранение паролей/PIN в зашифрованном виде, проверка хеширования паролей), корректности работы с правами доступа, защитой при экспорте/импорте данных и безопасной синхронизацией с сервером. Тестирование включает проверку на уязвимости уровней хранения и передачи данных, а также тесты на устойчивость к простым атакам (например, попыткам доступа к файлам приложения).

Для структурирования тестирования разрабатываются чек– листы и детальные тест– кейсы. Чек– лист служит быстрым инструментом для контрольных проверок релизов, а тест– кейс описывает входные условия, последовательность действий и ожидаемый результат для конкретного сценария. Совокупность тест– кейсов формирует тест– сьюты, используемые при функциональных, регрессионных и интеграционных проверках. Чек– лист и тест– кейс дополняют друг друга и являются удобными инструментами тестирования, которые позволяют охватить и структурировать весь процесс тестирования приложения при этом разделив его на последовательные легко обрабатываемые блоки тестов.

Выбор указанных видов тестирования обеспечивает всестороннюю проверку приложения: от корректности финансовых расчётов и целостности данных до удобства интерфейса и безопасности пользовательских данных. Комплексный подход обеспечивает минимизацию операционных рисков при выводе продукта на рынок и повышает доверие конечных пользователей к функционалу приложения.

4.2 Результаты тестирования

Тест–кейсы в разработке программного обеспечения – это детализированные сценарии, предназначенные для проверки определенных аспектов функционирования программы. Они содержат пошаговое руководство по выполнению тестов, предусловия, ожидаемые результаты и критерии успешности выполнения каждой операции. Цель тест–кейсов убедиться, что программа работает в соответствии с установленными требованиями и спецификациями.

Каждый тест– кейс обычно включает следующие элементы:

- название;
- краткое описание сути теста;
- идентификатор;
- предусловия;
- шаги выполнения;
- ожидаемый результат;
- фактический результат;
- статус;
- комментарии.

Тестирование программы проводилось в процесс разработки и по завершению разработки приложения было проведено итоговое тестирование. Для разрабатываемого приложения был создан чек– лист, представленный в таблице 4.1.

Таблица 4.1 – Чек– лист

Корректность работы авторизации
Корректность работы главного окна приложения
Корректность работы модуля учёта операций
Корректность работы модуля категорий
Корректность работы модуля аналитики
Корректность работы модуля бюджетов

На основании разработанного чек– листа были разработаны тест– кейсы для каждого модуля приложения.

В таблице 4.2 представлен тест– кейс для авторизации с помощью валидных данных

Таблица 4.2 – Тест– кейс авторизации с помощью валидных данных

Тест	Ожидаемый результат	Результат
Авторизация 1. Открыть приложение 2. Проверить элементы на странице 3. Ввести валидную информацию в поля «Введите PIN– код» 4. Кликнуть кнопку «Ввод»	1. Открывается страница авторизации. 2. Отображаются следующие элементы: форма авторизации (заголовок, поле для ввода PIN– кода, кнопка «Забыли пароль», кнопка, отправляющая форму). 3. В полях «Введите PIN– код» не отображается введенная информация в целях конфиденциальности. 4. Отображается главная страница приложения.	Успех

В таблице 4.3 представлен тест– кейс для главного окна приложения.

Таблица 4.3 – Тест– кейс для главного окна приложения

Тест	Ожидаемый результат	Результат
Корректность работы главного окна 1. Запустить программу 2. Проверить корректность отображение элементов управления 3. Проверить корректность адресации кнопок меню	1. Открывается главное меню. 2. Отображаются кнопки вызова модулей приложения. 3. При нажатии на кнопку: – «Операции» открывается окно с отображением операций, – «Отчеты» открывается модуль отображения отчетов, – «Бюджет» открывается модуль создание бюджета, – «Настройки» открывается окно настроек.	Успех

Следующим был разработан тест– кейс для модуля учёта операций (см. таблица 4.4). И при помощи разработанного тест– кейса был протестирован модуль.

Таблица 4.4 – Тест– кейс для модуля учёта операций

Тест	Ожидаемый результат	Результат
Корректность работы модуля учёта операций 1. Перейти на вкладку Транзакции 2. Проверить корректность отображение элементов управления 3. Создать новую транзакцию нажав на элемент «+» 4. Удалить транзакцию нажав на элемент «Корзина» 5. Изменить существующую транзакцию	1. Открывается модуль учёта операций 2. Корректно отображаются следующие элементы: – список транзакций; – список категорий; – кнопка для создания транзакции, – поля для ввода данных о транзакции. 3. При нажатии на кнопку «+» отображается интерфейс редактора транзакции. При внесении параметров и нажатии кнопки сохранить создается новая транзакция, которая сохраняется после закрытий модуля и отображается в общем списке транзакций.	Успех

После был разработан тест– кейс для тестирования работы модуля категорий. Разработанный тест– кейс представлен в таблице 4.5.

Таблица 4.5 – Тест– кейс для модуля категории

Тест	Ожидаемый результат	Результат
Корректность работы модуля категории 1. Перейти на вкладку Транзакции 2. Проверить корректность отображение элементов управления 3. Создать новую транзакцию нажав на элемент «+» 4. Создать новую категорию нажав на элемент «Добавить категорию»	1. Открывается модуль бюджетов 2. Корректно отображаются следующие элементы: – название категории; – тип категории; – кнопка для добавления иконки, – кнопка «Отмена» и «Добавить». 3. При нажатии на кнопку «Добавить категорию» отображается интерфейс добавления категории. При внесении параметров и нажатии кнопки сохранить создается новая категория, которая сохраняется после закрытий модуля и отображается в общем списке категорий.	Успех

Для проведения тестирования инструментов генерации аналитики в приложении был разработан тест– кейс. Данный тест– кейс представлен в таблице 4.6.

Таблица 4.6 – Тест– кейс для модуля аналитики

Тест	Ожидаемый результат	Результат
Корректность работы модуля аналитики 1. Перейти на вкладку Аналитика 2. Выбрать период 3. Проверить корректность отображение элементов управления	1. Открывается модуль аналитики 2. Корректно отображаются следующие элементы: – выбор периода; – основные блоки аналитики; – круговая диаграмма; – тренды доходов и расходов; – топ категорий. 3. При нажатии на кнопку выбора периода отображается интерфейс аналитики данных за этот период.	Успех

Затем разработан тест–кейс для тестирования работы модуля бюджетов, который показывает работоспособность системы относительно создания бюджетов по категориям расходов. Разработанный тест– кейс представлен в таблице 4.7.

Таблица 4.7 – Тест– кейс для модуля бюджетов

Тест	Ожидаемый результат	Результат
Корректность работы модуля бюджетов 1. Перейти на вкладку Бюджеты 2. Проверить корректность отображение элементов управления 3. Создать новую бюджет нажав на элемент «+»	1. Открывается модуль бюджетов 2. Корректно отображаются следующие элементы: – добавление бюджета; – обзор общего бюджета; – бюджеты категорий, – кнопка «Корзина». 3. При нажатии на кнопку «+» отображается интерфейс добавления бюджетов. При внесении параметров и нажатии кнопки сохранить создается новая бюджет, которая сохраняется после закрытий модуля и отображается в общем списке бюджетов.	Успех

В ходе тестирования использовались разработанные тест–кейсы, которые помогли выявить ряд ошибок в работе приложения. Функциональное тестирование показало, что все основные механики программного средства работают согласно спецификации требований. Выявленные ошибки были исправлены сразу же после обнаружения.

4.3 Вывод тестирования

В процессе разработки кроссплатформенного мобильного приложения для учёта и анализа персональных расходов особенно важно не только реализовать все предусмотренные функции, но и обеспечить их стабильную и корректную работу. Надёжность функционирования является ключевым фактором, поскольку приложение оперирует персональными финансовыми данными пользователей и должно работать безошибочно в любой ситуации.

Для достижения высокого качества итогового продукта тестирование проводилось на различных этапах разработки. Проверка отдельных модулей выполнялась параллельно с их созданием, что позволило своевременно выявлять логические ошибки, неточности в интерфейсе и несоответствия функциональных требований. После завершения реализации всех компонентов было выполнено комплексное тестирование всей системы, включающее проверку интеграции модулей и взаимодействия между ними.

В ходе тестирования были обнаружены и устранены следующие типовые ошибки:

– Некорректное отображение списка операций затрудняло просмотр полной финансовой истории, так как элементы списка расходов и доходов располагались вне видимой области из-за ошибки в адаптивной верстке.

– Обнаруженные ошибки в аналитическом модуле приводили к неверной визуализации статистики. Изменения временного периода не учитывались при построении диаграмм расходов, что также было исправлено через корректировку логики фильтрации.

– Проблемы адаптивности интерфейса затрудняли навигацию на устройствах с малым разрешением, так как часть элементов не помещалась на экран. Оптимизация интерфейса и переработка размеров отдельных элементов позволили решить эту задачу.

Тестирование охватило несколько направлений: функциональность, корректность обработки данных, качество интерфейса, стабильность работы при длительной эксплуатации, а также кроссплатформенность. Приложение проверялось на устройствах разных форм-факторов и под управлением различных версий Android и iOS, что позволило убедиться в корректной работе на всех целевых платформах.

Последовательное устранение выявленных ошибок и повторные циклы проверки позволили усовершенствовать систему и добиться того, чтобы каждый модуль приложения функционировал чётко и предсказуемо.

5 РУКОВОДСТВО ПО ЭКСПЛУАТАЦИИ ПРОГРАММНОГО СРЕДСТВА

Руководство по эксплуатации программного средства является важным элементом технической документации, обеспечивающим пользователя всей необходимой информацией для корректной и безопасной работы с приложением. Оно описывает требования к устройству, порядок установки и первоначальной настройки, что позволяет избежать ошибок на начальном этапе использования. Кроме того, руководство включает разъяснение функциональных возможностей программы и принципы работы основных модулей, благодаря чему пользователь легче осваивает приложение и может эффективно применять его в повседневной деятельности.

В документ также включены рекомендации по устранению наиболее распространённых неполадок. Это позволяет пользователю самостоятельно решать типовые проблемы, не прибегая к помощи специалистов. Отдельное внимание уделяется мерам безопасности и правилам эксплуатации, что снижает риск неправильного использования и потери данных.

Наличие руководства по эксплуатации способствует соблюдению нормативных требований и обеспечивает приложение официальным документальным сопровождением, которое может быть использовано при возникновении спорных ситуаций. Таким образом, данный документ играет ключевую роль в обеспечении стабильной, безопасной и удобной работы программного средства пользователями.

Руководство по эксплуатации предназначено для пользователей кроссплатформенного мобильного приложения учёта и анализа персональных расходов и включает инструкции по установке, первоначальной настройке и дальнейшему использованию программного средства.

Для корректной работы приложения устройство пользователя должно соответствовать следующим минимальным требованиям:

Операционная система:

- Android 8.0 и выше;
- iOS 13 и выше.

Процессор:

- двухъядерный процессор с тактовой частотой от 1,4 ГГц или аналогичный.

Оперативная память:

- не менее 2 ГБ.

Свободное место на устройстве:

- от 150 МБ для установки и хранения пользовательских данных.

Экран устройства:

- разрешение не ниже 720×1280 пикселей.

Интернет– подключение:

- желательно для синхронизации данных и загрузки обновлений,

скорость от 1 Мб/с.

Эти требования обеспечивают стабильную работу мобильного приложения и корректное отображение всех компонентов пользовательского интерфейса.

После установки и запуска мобильного приложения начинается автоматическая инициализация всех необходимых модулей. На экране отображается короткий загрузочный экран.

В процессе загрузки приложение выполняет проверку корректности сохранённых данных и конфигурационных параметров, включая наличие локальных записей о расходах, категорий трат и сохранённых настроек пользователя. При первом запуске автоматически создаются базовые системные таблицы (категории расходов, журнал транзакций, настройки пользователя), необходимые для дальнейшей работы приложения.

После завершения всех инициализационных процессов происходит автоматическое создание локального профиля пользователя. Система загружает пользовательские настройки и подготавливает интерфейс для дальнейшей работы.

На рисунке 5.1 представлено отображение главного экрана приложения:

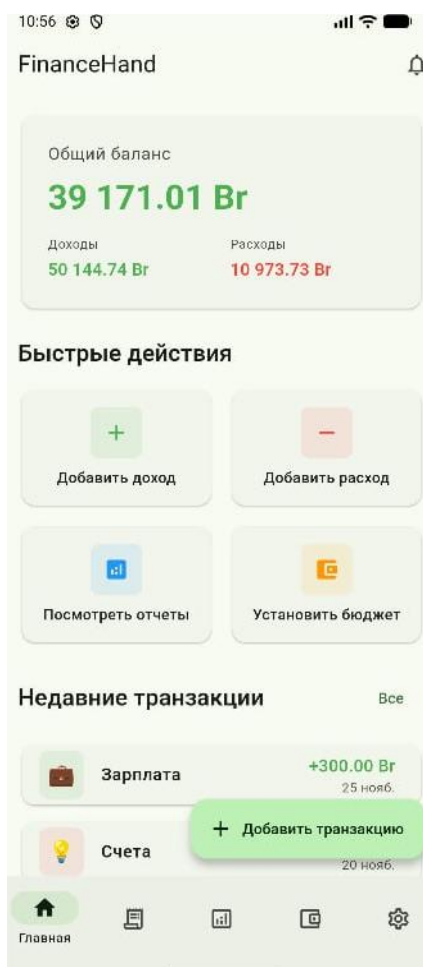


Рисунок 5.1 – Главное меню

После успешной инициализации на экране отображается главное окно приложения, содержащее текущий баланс расходов за период, быстрый доступ к добавлению новой транзакции и элементы навигации по основным модулям. Приложение полностью готово к использованию.

Главный экран является основной рабочей областью и содержит верхнюю панель отображает текущий общий баланс, баланс доходов и расходов, центральную область содержит быстрые действия, при взаимодействии с которыми можно совершать основную деятельность в приложении, также здесь отображаются, недавние добавленные транзакции и нижняя навигационная панель служит основным средством перемещения между разделами приложения и включает несколько значимых элементов управления. С её помощью пользователь может быстро переходить к ключевым экранам: просмотру всех транзакций, разделу аналитики, модулю бюджетирования и странице настроек.

При открытии вкладки «Транзакции» пользователю отображается полный список всех добавленных операций, упорядоченных по дате – от самых новых к более ранним. Интерфейс вкладки «Транзакции», представлен на рисунке 5.2.



Рисунок 5.2 – Интерфейс вкладки «Транзакции»

В верхней части экрана расположен быстрый фильтр, позволяющий мгновенно отсортировать транзакции по выбранным категориям. Дополнительно справа в верхнем углу размещена кнопка расширенного фильтра. Этот инструмент предоставляет возможности более тонкой настройки отображения данных: выбор конкретного периода, просмотр только доходов или только расходов, а также детализация по отдельным категориям.

Интерфейс вкладки «Транзакции» обеспечивает удобную и гибкую работу с финансовыми данными, позволяя пользователю быстро находить нужную информацию и анализировать свои операции.

Интерфейс расширенного («умного») фильтра, позволяющего настраивать период, типы операций и набор категорий, представлен на рисунке 5.3. Этот инструмент обеспечивает более гибкую работу с транзакциями и помогает пользователю выводить только действительно нужные данные.

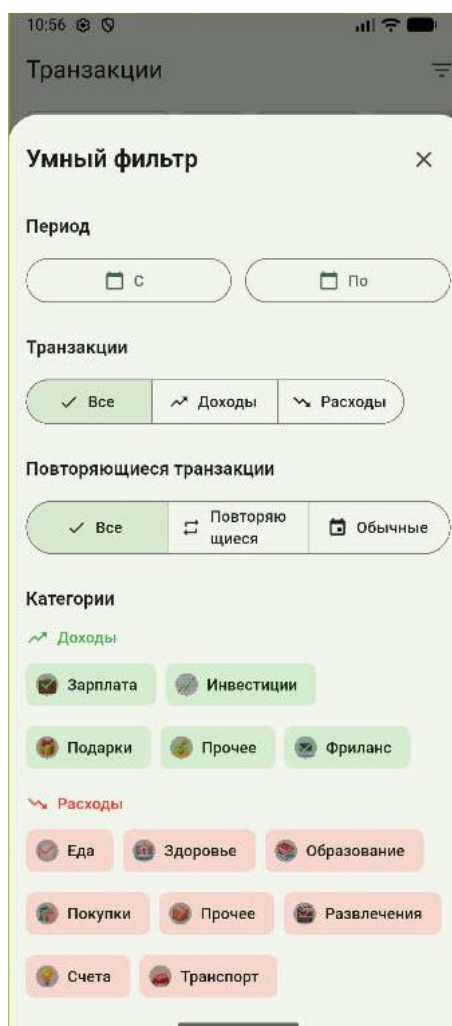


Рисунок 5.3 – Интерфейс расширенного фильтра

Экран добавления новой транзакции, представленный на рисунке 5.4, обеспечивает удобный и интуитивно понятный процесс внесения финансовых операций. Пользователь начинает с выбора типа транзакции – доход или расход. В зависимости от выбранного типа автоматически отображается соответствующий набор категорий, сформированных ранее при создании категорий. Это позволяет избежать ошибок при классификации операций и обеспечивает структурированность финансовых данных. Отображение соответствующих категорий продемонстрировано на следующем рисунке по порядку.

После выбора типа транзакции пользователь указывает сумму операции и выбирает необходимую валюту. Далее следует выбор категории. Если подходящей категории нет в списке, приложение позволяет создать её непосредственно из формы добавления транзакции.

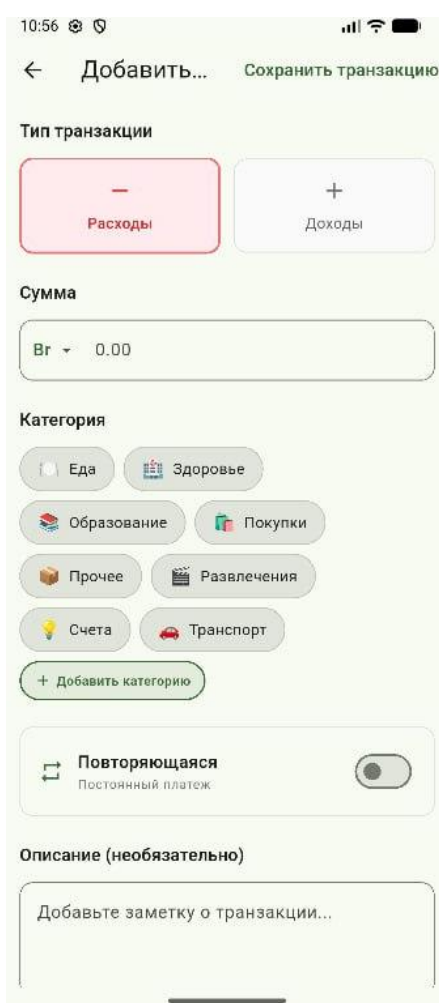


Рисунок 5.4 – Интерфейс добавления транзакции

Интерфейс создания новой категории приведён на рисунке 5.5. В этом окне можно задать название категории, определить её тип (доходная или расходная), а также выбрать визуальную иконку, которая будет использоваться в интерфейсе. Такой подход делает процесс настройки

категорий гибким и удобным, позволяя адаптировать приложение под индивидуальные особенности пользователя.

Помимо основных параметров, форма добавления транзакции предоставляет возможность отметить, является ли операция повторяющейся, добавить текстовое описание, а также указать точную дату финансового события. Все элементы интерфейса расположены последовательно и логично, что обеспечивает комфортный и быстрый процесс добавления операций, даже если пользователь ранее не работал с подобными приложениями.

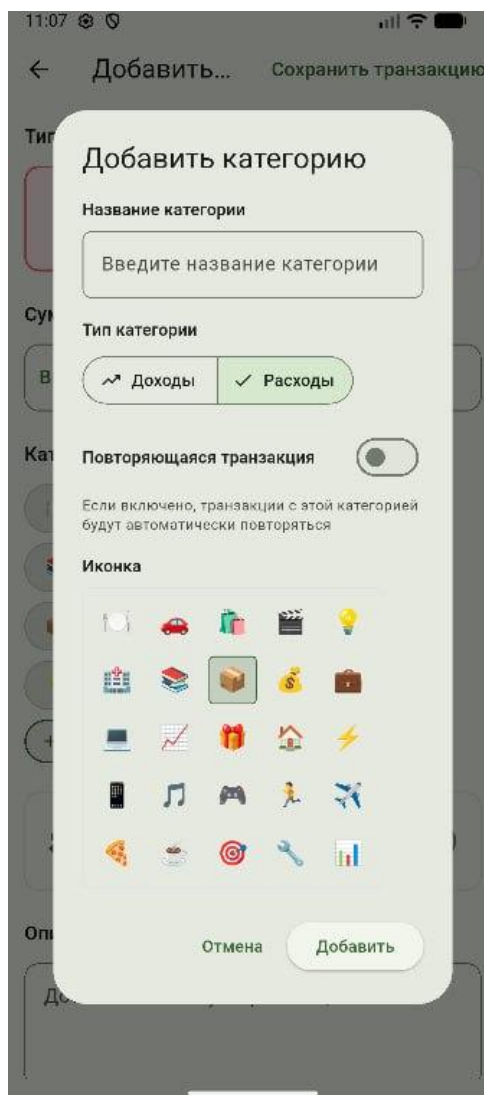


Рисунок 5.5 – Интерфейс добавления категории

Экран аналитики предназначен для наглядного отображения финансовой активности пользователя и представлен на рисунке 5.6. На верхней панели размещены ключевые графики, отражающие сумму доходов, общие расходы и величину чистого баланса. Чистый баланс представляет собой разницу между всеми полученными доходами и понесёнными расходами за выбранный период. Этот показатель позволяет пользователю

оценить, растут ли его сбережения или расходы превышают доходы.

Справа в верхней части экрана расположено меню выбора периода, изображённое на рисунке 5.6. Пользователь может сформировать отчёт за день, неделю, месяц, год или за произвольный диапазон дат. Переключение периодов моментально обновляет графики и статистические показатели, обеспечивая гибкость анализа финансовых данных.



Рисунок 5.6 – Интерфейс финансовой аналитики

Если пролистать экран ниже, становятся доступны дополнительные аналитические блоки. В разделе «Тренды» отображаются динамика доходов и расходов, что позволяет визуально увидеть изменения финансового поведения во времени. Ниже представлен раздел «Топ категорий», где перечислены категории, которые занимают наибольший удельный вес в

расходах или доходах пользователя. Это помогает определить ключевые направления, на которые тратится больше всего средств, и выявить потенциальные зоны оптимизации бюджета. Пример интерфейса этих аналитических разделов приведён на рисунке 5.7.

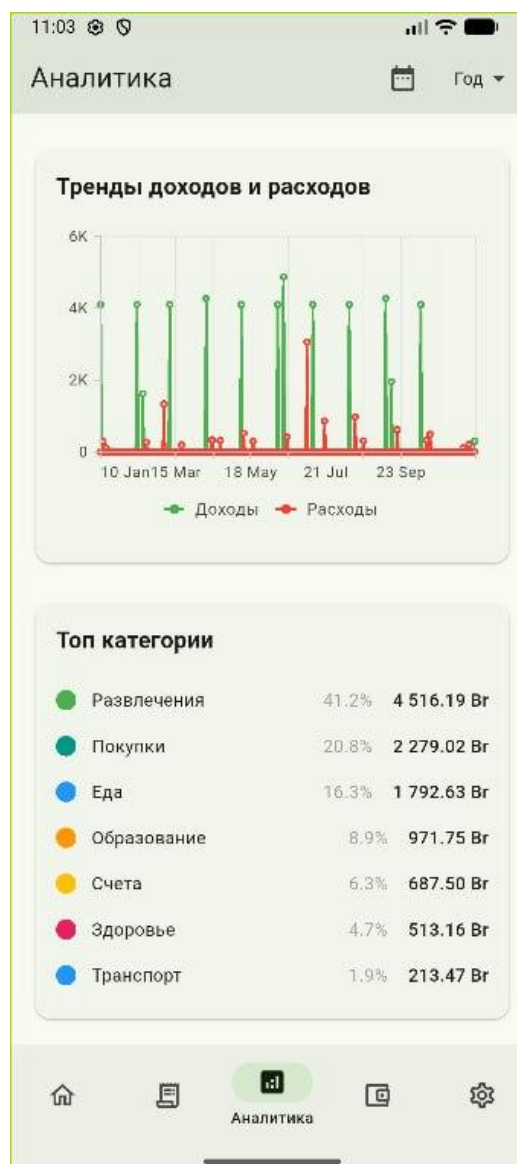


Рисунок 5.7 – Интерфейс расширенного фильтра

Доходы и расходы визуализируются с использованием диаграмм и графиков, что делает представление данных наглядным и доступным даже для пользователей без опыта в финансовом анализе.

Экран бюджетов, представленный на рисунке 5.8, предоставляет пользователю удобный инструмент для контроля запланированных расходов. В верхней части вкладки отображается обзор общего бюджета, где показана установленная сумма лимита, уже израсходованные средства и оставшийся доступный баланс. Благодаря этому пользователь может быстро оценить, насколько текущие траты соответствуют его финансовым планам.

Ниже расположен список бюджетов по категориям. Для каждой категории указывается период действия бюджета (неделя, месяц или год), установленный лимит, сумма потраченных средств и процент освоения бюджета. Визуальное представление прогресса позволяет легко определить категории, в которых расходы находятся под контролем, и те, где существует риск превышения установленного лимита. Рядом с каждой строкой размещена кнопка удаления, позволяющая оперативно редактировать структуру бюджетов в соответствии с изменяющимися потребностями пользователя.

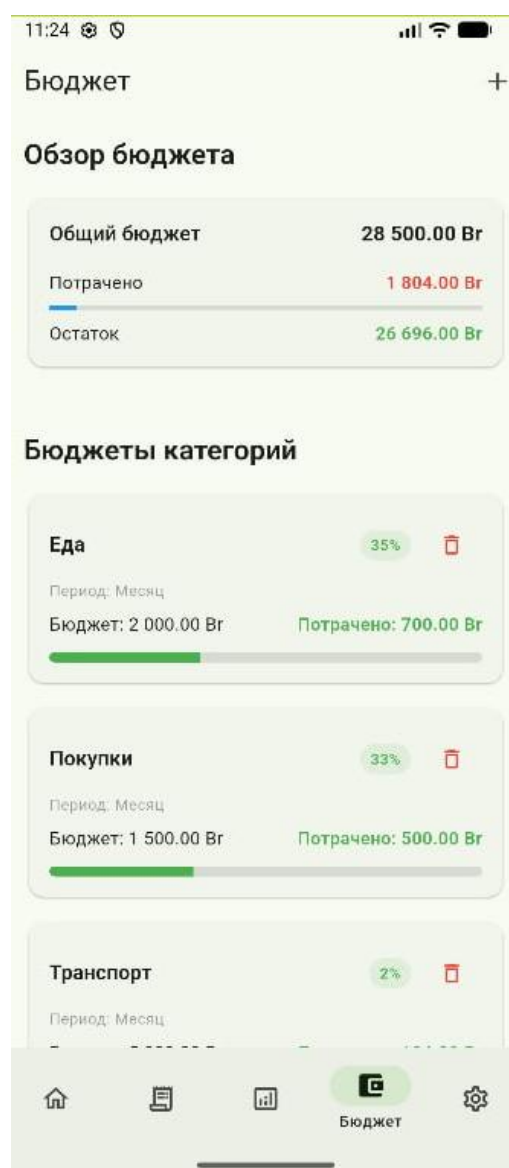


Рисунок 5.8 – Интерфейс бюджетов

Создание нового бюджета происходит через кнопку «+», размещённую в верхней части экрана. При добавлении бюджета пользователь выбирает период действия, категорию и вводит сумму лимита. Окно создания нового

бюджета приведено на рисунке 5.9. Такая структура обеспечивает гибкость и позволяет формировать как краткосрочные бюджеты, например, на неделю, так и долгосрочные – на месяц или год. Это делает систему бюджетирования универсальной и адаптивной под индивидуальные финансовые цели.

11:27

← Создать бюджет Сохранить

Период бюджета

Неделя Месяц Год

Категория

Еда Здоровье

Образование Покупки

Прочее Развлечения

Счета Транспорт

Лимит бюджета

0.00

Установите лимит расходов для выбранной категории

Создать бюджет

Рисунок 5.9 – Интерфейс создания нового бюджета

Система бюджетов является одним из ключевых инструментов финансового планирования, позволяя пользователю заранее определять допустимые пределы расходов и тем самым формировать более осознанное отношение к своим тратам. Использование бюджетов помогает создать финансовую дисциплину, поскольку пользователь постоянно видит, насколько его фактические расходы соответствуют поставленным целям.

Вкладка «Настройки» (рисунок 5.10) предоставляет пользователю доступ к основным параметрам управления приложением и позволяет

адаптировать его работу под индивидуальные предпочтения. Интерфейс раздела структурирован таким образом, чтобы даже при большом количестве функций настройки оставались интуитивно понятными и легко доступными.

Одним из ключевых элементов является возможность задать или изменить PIN– код для входа в приложение. Такая функция повышает уровень безопасности персональных финансовых данных, обеспечивая дополнительную защиту в случаях утери устройства или доступа к нему третьих лиц.

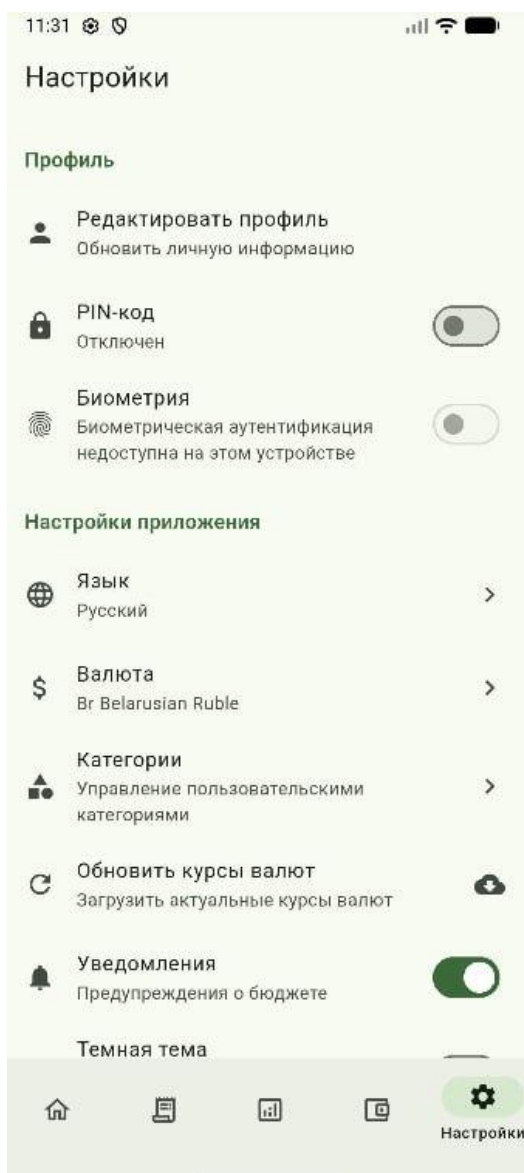


Рисунок 5.10 – Интерфейс «Настройки»

Во вкладке предусмотрена возможность управления используемой валютой. Пользователь может выбрать основную валюту учёта, а также при необходимости изменить её. После изменения основной валюты все суммы в интерфейсе автоматически пересчитываются в соответствии с актуальным курсом.

Дополнительно в разделе настроек доступно управление категориями расходов и доходов: пользователь может редактировать существующие категории, добавлять новые или удалять неактуальные. Это обеспечивает гибкость и позволяет точнее адаптировать приложение под реальный финансовый стиль пользователя.

Одним из ключевых элементов является возможность задать или изменить PIN– код для входа в приложение. Такая функция повышает уровень безопасности персональных финансовых данных, обеспечивая дополнительную защиту в случаях утери устройства или доступа к нему третьих лиц.



Рисунок 5.11– Интерфейс создания «PIN– код»

Также пользователю доступна смена языка интерфейса (рисунок 5.11 и рисунок 5.12). Приложение поддерживает несколько языков, что делает его удобным для широкой аудитории и позволяет выбрать наиболее комфортный

вариант отображения информации. Переключение языка происходит мгновенно, без необходимости перезапуска приложения.

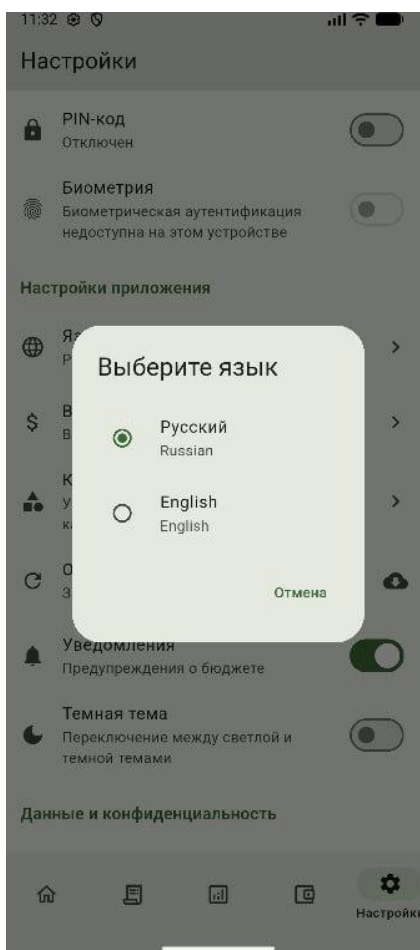


Рисунок 5.12 – Выбор языка интерфейса

Также реализована функция включения и отключения уведомлений. Пользователь может настроить получение напоминаний о расходах, поступлениях или достижении лимитов бюджета, что помогает более эффективно контролировать финансы.

Отдельного внимания заслуживает механизм обновления валютных курсов. При нажатии соответствующей кнопки приложение отправляет запрос к внешнему API и получает актуальные курсы валют. После получения данных все транзакции, бюджеты и аналитические показатели пересчитываются автоматически.

Мобильное приложение предлагает интуитивно понятный интерфейс для управления финансами без подготовки. Основные функции обеспечивают быструю навигацию. Руководство пользователя предоставляет необходимую информацию для эффективного использования приложения в повседневной финансовой деятельности.

6 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО СРЕДСТВА

6.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Мобильное приложение по учёту и анализу персональных расходов представляет собой современное программное средство, предназначенное для учёта личных доходов и расходов, анализа финансовой активности и формирования наглядной картины финансового состояния пользователя. Приложение обеспечивает удобный и интуитивный интерфейс для регулярного внесения финансовых операций, ведения категорий расходов, контроля бюджета и отслеживания динамики затрат.

Основной целью разработки данного приложения является создание доступного, функционального и лёгкого в использовании инструмента, который позволит пользователям эффективно управлять личными финансами, систематизировать расходы и принимать более взвешенные финансовые решения. Приложение направлено на повышение финансовой грамотности пользователей и упрощение процесса ежедневного учёта денежных операций. Область применения разработанного программного средства охватывает такие направления как личный финансовый учёт, планирование бюджета, анализ расходов, оптимизация повседневных финансов и повышение качества финансового планирования. Приложение может использоваться широким кругом пользователей – от студентов и молодых специалистов, желающих контролировать бюджет и снижать необоснованные траты.

Приложение автоматизирует процесс финансового учёта, позволяя пользователю оперативно вносить информацию о расходах и доходах, задавать категории операций, просматривать статистику и отслеживать изменения в течение дня, недели или месяца. Особое внимание уделено визуализации данных – пользователь получает круговые диаграммы распределения расходов по категориям, графики изменений и ключевые показатели, что существенно упрощает восприятие информации.

К основным функциям приложения относятся:

- внесение расходов и доходов с указанием категории, даты и суммы;
- ведение списка категорий с возможностью добавления собственных;
- просмотр статистики в виде диаграмм и аналитических сводок;
- фильтрация и сортировка операций;
- отслеживание динамики расходов по дням и месяцам.

Потенциальными пользователями программного средства являются:

- частные лица, желающие наладить личный финансовый контроль;
- студенты, контролирующие ограниченный бюджет;
- пользователи, стремящиеся сократить необязательные траты;
- люди, предпочитающие визуальный анализ финансовых данных.

Актуальность разработки обусловлена рядом факторов:

- рост популярности финансовых приложений в связи с цифровизацией бытовых процессов;
- увеличение потребности в инструментах контроля расходов из-за роста стоимости товаров и услуг;
- стремление пользователей к финансовой дисциплине и оптимизации бюджета;
- недостаток простых, компактных и удобных приложений без лишних функций и перегрузки интерфейса.

На рынке присутствуют различные аналоги – такие как YNAB (You Need A Budget), Учёт расходов – Бюджет ОК и Расходы – Личный бюджет и другие, однако многие из них перегружены функционалом или ориентированы на продвинутую аудиторию. Разрабатываемое приложение делает акцент на простоте, наглядности и минимализме, что делает его привлекательным для пользователей, которым нужен быстрый и удобный инструмент учёта без сложных настроек.

Экономическая модель приложения может включать:

- бесплатное распространение с возможностью добровольной поддержки разработчика;
- расширенные функции (например, экспорт данных или дополнительные диаграммы) по подписке;
- вывод ограниченной рекламы при бесплатном использовании;
- платную премиум– версию без ограничений.

На данный момент такое мобильное приложение является значимым и представляет собой актуальное, востребованное и экономически перспективное программное средство, ориентированное на широкую аудиторию и обладающее потенциалом для успешного выхода на рынок.

6.2 Расчет инвестиций в разработку для его реализации на рынке

Расчёт инвестиций, необходимых для разработки мобильного приложения для проведения настольных ролевых игр, является ключевым этапом технико– экономического обоснования проекта. Проведение данного расчёта позволяет определить экономическую целесообразность разработки, оценить общий объём финансовых вложений, спрогнозировать сроки окупаемости и потенциальную прибыль, а также минимизировать риски, связанные с выводом программного продукта на рынок.

Инвестиционные затраты включают расходы на оплату труда специалистов, принимающих участие в создании приложения: разработчиков, дизайнеров пользовательского интерфейса, тестировщиков и аналитиков. В расчёт также входят затраты на приобретение или использование необходимого оборудования, лицензированного программного обеспечения,

сторонних библиотек, облачных сервисов и хранилищ данных. Отдельной статьёй расходов выступают маркетинговые мероприятия, направленные на продвижение приложения на рынке мобильных решений.

При планировании инвестиций важно учитывать и будущие затраты, связанные с эксплуатацией и развитием приложения после его запуска. К ним относятся расходы на техническую поддержку пользователей, регулярное обновление функционала, устранение ошибок, а также адаптация программного средства под новые версии мобильных операционных систем и типы устройств.

Объём необходимых инвестиций оказывает непосредственное влияние на качество конечного продукта и его конкурентоспособность. Современный интерфейс, стабильная работа, продуманная архитектура и широкий набор инструментов для игроков и ведущих настольных ролевых игр формируют привлекательность приложения для целевой аудитории. Это способствует росту пользовательской базы и увеличению потенциальных доходов от продаж, подписочных моделей или рекламных интеграций.

Расчёт инвестиций предоставляет разработчику и потенциальным инвесторам объективную оценку финансовой стороны проекта, формируя основу для последующих расчётов экономического эффекта и показателей эффективности реализации программного продукта.

Ставки налоговых отчислений были взяты из источника [11], поскольку с момента публикации данного методического пособия они остались неизменными. Это позволяет использовать актуальные данные для расчетов и анализа, обеспечивая надежность представленной информации.

Что касается ставок по депозитам, они были получены с официального сайта Национального банка Республики Беларусь (НБ РБ) и также не подвергались изменениям с 28 июня 2023 года [12]. Использование этих данных гарантирует, что информация о доходности вкладов актуальна и соответствует реальным условиям рынка.

На 2025 год расчетная норма рабочего времени для сотрудников, работающих по пятидневной рабочей неделе, составляет 168 часов. Это основано на стандартной продолжительности рабочего дня в 8 часов, что подразумевает среднемесячную норму рабочего времени в 21 рабочий день. Эти показатели являются важными для планирования трудозатрат и расчета заработной платы, что подчеркивает их значимость в анализе.

Разработкой программного средства занимались следующие лица: Full stack программист, тестировщик, UX/UI дизайнер.

Full Stack программист обладает навыками как фронтенд, так и бэкенд– разработки, что позволяет ему разрабатывать весь стек приложения и быстро решать возникающие проблемы.

Тестировщик (QA Engineer) – отвечает за качество продукта, проводя различные виды тестирования для выявления ошибок и повышения стабильности приложения.

UX/UI дизайнер – создает интуитивно понятный интерфейс и

улучшает пользовательский опыт, разрабатывая макеты и прототипы с учетом потребностей пользователей.

Оклады данных работников были взяты в виде медианного значения с сайта rabota.by [14] и оформлены в виде таблицы 6,1.

Таблица 6.1 – месячные оклады работников

Категория исполнителя	Месячный оклад, р
Full Stack программист	3500
Тестировщик	2300
UI/UX Дизайнер	2000

Действующие на момент проведения расчетов ставки налогов и отчислений в Республике Беларусь можно указать следующим образом:

Подходный налог – ставка: 13% для резидентов и 30% для нерезидентов.

Налог на добавленную стоимость (НДС) – основная ставка: 20%.
Налог на прибыль – ставка: 20%.

Единый социальный налог – ставка: 34% от фонда оплаты труда.

Действующие ставки рефинансирования – 9.5% и ставки по долгосрочным банковским депозитам.

Разрабатываемая программное средство будет проводится для обеспечения свободной реализации на рынке ИТ, поэтому расчет затрат на разработку ПО, оценка результата (эффекта) от использования (или продажи) ПО, расчета показателей эффективности инвестиций в разработку ПО будет производится соответствующим образом. Далее в работе используются формулы, приведённые в методическом пособии по экономическому обоснованию дипломных проектов [13].

Расчет затрат на разработку ПО представлен в таблице 6.2 и показан в разрезе следующих статей:

- затраты на основную заработную плату разработчиков;
- затраты на дополнительную заработную плату разработчиков;
- отчисления на социальные нужды;
- прочие затраты (амортизационные отчисления, расходы на электроэнергию, командировочные расходы, арендная плата за офисные помещения и оборудование, расходы на управление и реализацию и т.п.).

Затраты на основную заработную плату команды разработчиков определяются исходя из состава и численности команды, размеров месячной заработной платы каждого из участников команды, а также общей трудоемкости разработки программного обеспечения. Расчет затрат на основную заработную плату Z_0 разработчиков осуществляется по формуле 6.1:

$$З_0 = K_{\text{пр}} \sum_{i=1}^n З_{\text{ч}i} \cdot t_i, \quad (6.1)$$

где $K_{\text{пр}}$ – коэффициент премий и иных стимулирующих выплат;

n – категории исполнителей, занятых разработкой;

$З_{\text{ч}i}$ – часовой оклад плата исполнителя i – й категории, руб.;

t_i – трудоёмкость работ, выполняемых исполнителем i – й категории.

Таблица 6.2 – Расчёт затрат на основную заработную плату команды.

Участник команды	Вид выполняемой работы	Месячная заработная плата, р.	Часовая заработная плата, р.	Трудоёмкость работ, ч.	Зарплата по тарифу, р.
Full Stack программист	Разработка логики приложения	3500	19,05	500	9525
Тестировщик	Тестирование приложения	2300	10,71	300	3213
UI/UX Дизайнер	Разработка визуала приложения	2000	8,93	250	2233
Всего затрат на основную заработную плату разработчиков					14971

$$З_0 = 19,05 \cdot 500 + 10,71 \cdot 300 + 8,93 \cdot 250 = 14970,5 \cdot 1,5 = 14971$$

Затраты на дополнительную заработную плату команды разработчиков включает выплаты, предусмотренные законодательством о труде (оплата трудовых отпусков, льготных часов, времени выполнения государственных обязанностей и других выплат, не связанных с основной деятельностью исполнителей), и определяется по формуле 6.2:

$$З_д = \frac{З_0 \cdot Н_д}{100}, \quad (6.2)$$

где $З_0$ – затраты на основную заработную плату, (руб.); $Н_д$ – норматив дополнительной заработной платы, рекомендуется брать в пределах 10–20% (или по согласованию с консультантом по экономическому разделу). По формуле 6.2 вычислим:

$$З_д = \frac{14971 \cdot 10}{100} = 1497,1$$

Отчисления на социальные нужды $P_{\text{соц}}$ рассчитываются в соответствии

с действующими законодательными актами по формуле 6.3:

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100}, \quad (6.3)$$

где: $H_{\text{соц}}$ – норматив отчислений на социальные нужды, %. По формуле 6.3 вычислим:

$$P_{\text{соц}} = \frac{(14971 + 1497,1) \cdot 34,6}{100} = 5697,96$$

Прочие затраты рассчитываются по формуле 6.4:

$$P_{\text{пр}} = \frac{Z_o \cdot H_{\text{пз}}}{100}, \quad (6.4)$$

где: $H_{\text{пз}}$ – норматив прочих затрат, рекомендуется брать в пределах 30 – 40%. По формуле 6.4 вычислим:

$$P_{\text{пр}} = \frac{14971 \cdot 35}{100} = 5239.85$$

Расходы на реализацию рассчитываются по формуле 6.5:

$$P_{\text{р}} = \frac{Z_o \cdot H_{\text{р}}}{100}, \quad (6.5)$$

где: $H_{\text{р}}$ – норматив расходов на реализацию, рекомендуется брать в пределах 3 – 5%. По формуле 6.5 вычислим:

$$P_{\text{р}} = \frac{14971 \cdot 3}{100} = 449,13$$

Общая сумма затрат на разработку Z_p находится по формуле 6.6:

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_{\text{р}} \quad (6.6)$$

где Z_o – основная заработная плата специалистов, руб.;

Z_d – дополнительная заработная плата специалистов, руб.;

$P_{\text{соц}}$ – отчисления на социальные нужды, руб.;

$Z_{\text{пр}}$ – прочие затраты, руб..

Используя формулу 6.6, вычислим общую сумму затрат на

разработку:

$$З_p = 14971 + 1497,1 + 5697,96 + 5239,85 + 449,13 = 27855,04$$

Результаты формул 6.1 – 6.6 представлены в таблице 6.3.

Таблица 6.3 – Затраты на разработку программного обеспечения

Статья затрат	Сумма, р.
Основная заработная плата команды разработчиков	14971
Дополнительная заработная плата команды разработчиков	1497,1
Отчисления на социальные нужды	5697,96
Прочие затраты	5239,85
Расходы на реализацию	449,13
Общая сумма затрат на разработку	27855,04

6.3 Расчет экономического эффекта от реализации программного средства на рынке

Экономический эффект от реализации мобильного приложения для управления личным бюджетом выражается в увеличении чистой прибыли, зависящей от объёма продаж, цены приложения и затрат на его разработку и последующее сопровождение.

Обоснование прогнозируемого объёма продаж (например, 10000 загрузок):

Рост интереса к персональному финансовому планированию. В последние годы наблюдается устойчивый рост спроса на приложения, позволяющие контролировать расходы, вести учёт транзакций и формировать личный бюджет. Это связано с нестабильностью экономической ситуации, ростом цен и общей необходимостью более осознанного финансового поведения. Всё больше пользователей стремятся к прозрачности собственных финансов и выбирают инструменты автоматизации учёта.

Разрабатываемое приложение содержит такие востребованные функции, как:

- учёт доходов и расходов;
- распределение финансов по категориям;
- сводные аналитические диаграммы;
- статистика за периоды;
- удобная система хранения финансовых записей.

Такие возможности позволяют продукту занять востребованную нишу в сегменте финансовых приложений.

Конкурентные преимущества в сравнении с существующими аналогами. На рынке уже существуют приложения, ориентированные на финансовый учёт, однако значительная их часть либо перегружена функционалом, либо рассчитана на подписочную модель, что снижает

доступность для широкой аудитории.

Разрабатываемое приложение отличается следующими преимуществами:

- простота пользовательского интерфейса;
- локализация под русскоязычную аудиторию;
- отсутствие обязательной подписки;
- ориентация именно на личный бюджет без лишних корпоративных инструментов;
- работа в офлайн– режиме без постоянного подключения к сети.

Эти особенности делают приложение привлекательным как для начинающих пользователей, так и для тех, кому важно иметь минималистичный, быстрый и доступный инструмент финансового учёта.

Доступная стоимость приложения (например, 10 бел. рублей). Умеренная и конкурентоспособная цена повышает вероятность покупки среди широкой аудитории, включающей:

- студентов;
- молодых специалистов;
- пользователей с ограниченным бюджетом;
- людей, которые ищут простой инструмент для ведения финансов без сложных функций.

При прогнозируемом объёме продаж в 40 000 загрузок данный показатель считается реалистичным благодаря востребованности финансовых приложений, доступной цене и грамотной маркетинговой стратегии.

Экономический эффект организации– разработчика программного обеспечения в данном случае представляет собой прибыли (чистую прибыль) от его продажи множеству потребителей.

Разработка и использование мобильного приложения по управлению бюджетом предполагает значительный экономический эффект для разработчика. Основным источником дохода является продажа приложения. Предположим, что приложение будет продаваться по цене 10 рублей, и в течение первого года будет 10000 загрузок.

Экономический эффект организации – разработчика программного средства представляет собой прирост чистой прибыли от его продажи на рынке потребителям, величина которого зависит от объема продаж, цены реализации и затрат на разработку программного средства.

Необходимо сделать обоснование предполагаемого объема продаж – ожидаемого количества копий (лицензий) программного средства, которое будет приобретено пользователями.

Прирост чистой прибыли, полученной разработчиком от реализации программного средства на рынке, можно рассчитать по формуле 6.7.

$$\Delta\P_{\text{ч}}^{\text{P}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot P_{\text{пр}} \cdot 1 - \quad (6.7)$$

где $C_{отп}$ – отпускная цена копии (лицензии) программного средства, р.; N – количество копий (лицензий) программного средства, реализуемое за год, шт.;

НДС – сумма налога на добавленную стоимость, р.

$R_{пр}$ – рентабельность продаж копий (лицензий) (по фактическим данным предприятия или на уровне 20– 40%);

$N_{п}$ – ставка налога на прибыль согласно действующему законодательству, % (по состоянию на июль 2021 г. – 18%).

По формуле 6.7 вычислим:

$$\Delta\P_{ч}^P = (10 \cdot 10000 - 16666,67) \cdot 10 \cdot 1 - \frac{18}{100} = 833333,12$$

Налог на добавленную стоимость определяется по формуле 6.8:

$$\text{НДС} = \frac{C \cdot N \cdot N_{дс}}{100\% + N_{дс}}, \quad (6.8)$$

где $N_{дс}$ – ставка налога на добавленную стоимость согласно действующему законодательству, (20%).

По формуле 6.8 вычислим:

$$\text{НДС} = \frac{10 \cdot 10000 \cdot 20\%}{100\% + 20\%} = 16666,67$$

Для оценки экономического эффекта у организации– разработчика необходимо определить цену ПО и рассчитать уровень рентабельности.

6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Сравнивая сумму общей выручки с затратами на разработку в размере 27855,04 рублей, можно увидеть, что проект обеспечит чистую прибыль в размере 833333,12 рублей, что подтверждает его экономическую целесообразность.

Расчет показателей экономической эффективности разработки и использования программного средства играет решающую роль в принятии стратегических решений компанией. Анализ полученных данных позволяет судить о финансовой целесообразности проекта, его влиянии на прибыльность.

Для определения экономической эффективности разработки программного средств необходимо сравнить с затратами на разработку программного средства и полученного экономического эффекта (годового прироста чистой прибыли). В данном случае годовой экономический

эффект больше затрат, следовательно, необходимо воспользоваться формулой рентабельности инвестиций (Return on Investments, ROI)[13]:

ROI выражает отношение прибыли от инвестиций к изначальным инвестициям. Этот показатель позволяет оценить эффективность использования ресурсов в процессе разработки и эксплуатации программного средства. Для расчёта ROI, воспользуемся формулой 6.9:

$$ROI = \frac{\Pi_{\text{ч}}^{\text{р}} - Z_{\text{р}}}{Z_{\text{р}}} \cdot 100\%, \quad (6.9)$$

где $\Pi_{\text{ч}}^{\text{р}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке, руб.;

$Z_{\text{р}}$ – затраты на разработку программного средства, руб.

Подставим значения и получим:

$$ROI = \frac{833333,12 - 27855,04}{27855,04} \cdot 100\% = 28,93$$

Проект разработки программного обеспечения будет экономически эффективным, поскольку рентабельность инвестиций (ROI) на его реализацию составила 28,93%. Таким образом, данное программное средство целесообразно как для организации– разработчика, так и для организации– заказчика, поскольку программное средство выходит на окупаемость уже в первом году. В последующие годы не будет таких крупных затрат, однако прибыль ожидается на схожем уровне. Получается, что данное вложение несет для организации– заказчика значительный потенциал.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта было разработано кроссплатформенное мобильное приложение учёта и анализа персональных расходов, предназначенное для автоматизации процесса управления личными финансами. Приложение реализовано с использованием фреймворка Flutter и языка программирования Dart, что обеспечило единый код для Android и iOS и высокую производительность.

В ходе работы проведён анализ предметной области управления персональными финансами, изучены существующие аналоги и определены их преимущества и недостатки. На основе этого анализа сформулированы функциональные и нефункциональные требования к разрабатываемому приложению.

В рамках проекта реализованы основные функции: добавление и редактирование операций доходов и расходов, категоризация, работа с несколькими валютами, визуализация данных в виде диаграмм, формирование аналитических отчётов и настройка бюджета с уведомлениями при его превышении. Для обеспечения безопасности реализована авторизация с использованием PIN-кода, а данные пользователей защищены с помощью алгоритма SHA-256.

Применение архитектурного подхода Clean Architecture и шаблона BLoC позволило достичь модульности и удобства сопровождения кода. Использование локальной базы данных SQLite (или Hive) обеспечило надёжное хранение данных и автономную работу приложения без постоянного подключения к интернету.

В процессе разработки проведено тестирование функциональных модулей, подтверждена корректность работы основных функций, устойчивость приложения и удобство пользовательского интерфейса.

Практическая ценность проекта заключается в создании современного, надёжного и простого инструмента для ведения личного бюджета. Приложение может использоваться широкой аудиторией для контроля расходов, анализа финансовых привычек и планирования бюджета.

Дальнейшее развитие проекта может включать реализацию функций автоматического импорта банковских операций, синхронизацию данных между устройствами, добавление рекомендаций по оптимизации расходов и расширение аналитических возможностей.

Разработанное программное средство соответствует поставленным целям и требованиям, демонстрирует применение современных технологий мобильной разработки и имеет потенциал для дальнейшего развития и коммерческого использования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Отчёт о развитии цифровых финансовых услуг в Республике Беларусь за 2024 год [Электронный ресурс]. – Минск : НБРБ, 2024. – Режим доступа : <https://www.nbrb.by/> (дата обращения: 23.10.2025).
2. Google for Developers. Flutter Overview: Cross– platform Development Toolkit [Электронный ресурс]. – Режим доступа : <https://developers.google.com/flutter> (дата обращения: 23.10.2025).
3. Министерство финансов Республики Беларусь. Национальная стратегия повышения финансовой грамотности населения на 2023– 2025 годы [Электронный ресурс]. – Минск : Минфин РБ, 2023. – Режим доступа : <https://www.minfin.gov.by/> (дата обращения: 23.10.2025).
4. Журавлёва Т. С., Лисовец А. В. Развитие цифровых технологий в сфере личных финансов и формирование финансовой грамотности населения [Электронный ресурс] // Экономика и банки. – 2023. – № 2. – С. 45– 52. – Режим доступа : <https://elibrary.ru/> (дата обращения: 23.10.2025).
5. Deloitte.Global FinTech Industry Report 2024 [Электронный ресурс]. – Deloitte, 2024. – Режим доступа : <https://www.deloitte.com/> (дата обращения: 23.10.2025).
6. Schwaber K., Sutherland J. The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game [Электронный ресурс]. – Scrum.org, 2020. – Режим доступа : <https://scrumguides.org/> (дата обращения: 23.10.2025).
7. Martin R. C. Clean Architecture: A Craftsman’s Guide to Software Structure and Design [Электронный ресурс]. – Boston : Pearson Education, 2018. – Режим доступа : <https://www.pearson.com/en– us/subject– catalog/p/clean– architecture> (дата обращения: 23.10.2025).
8. Flutter Documentation.State Management – BLoC Pattern [Электронный ресурс]. – Google Developers, 2025. – Режим доступа : <https://bloclibrary.dev> (дата обращения: 23.10.2025).
9. SQLite Consortium.SQLite Documentation – Lightweight Relational Database for Mobile Applications [Электронный ресурс]. – Режим доступа : <https://www.sqlite.org/docs.html> (дата обращения: 23.10.2025).
10. [Syncfusion.Flutter Charts Library Overview [Электронный ресурс]. – Syncfusion Inc., 2025. – Режим доступа : <https://help.syncfusion.com/flutter/cartesian– charts/getting– started> (дата обращения: 23.10.2025).
11. Техничко– экономическое обоснование разработки программного обеспечения: методическое пособие для студентов учреждений высшего образования / под ред. И. В. Горбуновой. – Минск : БГУИР, 2020. – 68 с.
12. Национальный банк Республики Беларусь. Основные показатели финансового рынка: процентные ставки по депозитам физических лиц [Электронный ресурс]. – Режим доступа: <https://www.nbrb.by/statistics/bankingsector/deposits> – Дата доступа: 25.11.2025.
13. В. Г. Горовой [и др.]. Экономика проектных решений:

методические указания по экономическому обоснованию дипломных проектов: учеб.– метод. пособие – Минск: БГУИР, 2021. – 107 с.

14. Подборка вакансий [Электронный ресурс]. – Режим доступа: <https://rabota.by/> – Дата доступа: 27.11.2025.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программного модуля

```
//main.dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:hive_flutter/hive_flutter.dart';

import 'l10n/app_localizations.dart';
import 'simple_main_page.dart';
import 'core/database/database_helper.dart';
import 'core/di/injection_container.dart';
import 'core/services/theme_service.dart';
import 'core/services/locale_service.dart';
import 'core/services/currency_service.dart';
import 'core/services/currency_api_service.dart';
import 'core/utils/mock_data_seeder.dart';
import 'features/transactions/data/models/transaction_model.dart';
import 'features/categories/data/models/category_model.dart';
import 'features/budget/data/models/budget_model.dart';
import 'features/auth/data/models/user_model.dart';
import 'features/recurring/data/models/recurring_transaction_model.dart';
import 'features/transactions/presentation/bloc/transactions_bloc.dart';
import 'features/categories/presentation/bloc/categories_bloc.dart';
import 'features/analytics/presentation/bloc/analytics_bloc.dart';
import 'features/budget/presentation/bloc/budget_bloc.dart';
import 'core/di/injection_container.dart';
import 'core/security/pin_auth_service.dart';
import 'core/security/biometric_auth_service.dart';

late ThemeService themeService;
late LocaleService localeService;
late CurrencyService currencyService;
late BiometricAuthService biometricAuthService;

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Initialize SharedPreferences
  final prefs = await SharedPreferences.getInstance();
```

```

// Initialize ThemeService
themeService = ThemeService(prefs);

// Initialize LocaleService
localeService = LocaleService(prefs);

// Initialize CurrencyService
final currencyApiService = CurrencyApiService(prefs);
currencyService = CurrencyService(prefs, currencyApiService);

// Initialize BiometricAuthService
biometricAuthService = BiometricAuthService(prefs);

// Initialize Hive
await Hive.initFlutter();

// Register adapters
Hive.registerAdapter(TransactionModelAdapter());
Hive.registerAdapter(CategoryModelAdapter());
Hive.registerAdapter(BudgetModelAdapter());
Hive.registerAdapter(UserModelAdapter());
Hive.registerAdapter(RecurringTransactionModelAdapter());
Hive.registerAdapter(RecurringIntervalModelAdapter());

// Initialize database
await DatabaseHelper.instance.database;

// Ensure default categories exist
await DatabaseHelper.instance.ensureDefaultCategories();

// Initialize dependency injection
await initializeDependencies(prefs);

// Load mock data on first run
if (await MockDataSeeder.shouldLoadMockData()) {
  print('🌱 First run detected - loading mock data...');
  await MockDataSeeder.seedMockData('default_user');
}

runApp(
  MultiBlocProvider(
    providers: [
      BlocProvider<TransactionsBloc>(
        create: (_) => getIt<TransactionsBloc>(),

```

```

    ),
    BlocProvider<CategoriesBloc>(
      create: (_) => getIt<CategoriesBloc>(),
    ),
    BlocProvider<AnalyticsBloc>(
      create: (_) => getIt<AnalyticsBloc>(),
    ),
    BlocProvider<BudgetBloc>(
      create: (_) => getIt<BudgetBloc>(),
    ),
  ],
  child: const FinanceHandApp(),
),
);
}

```

```

class FinanceHandApp extends StatelessWidget {
  const FinanceHandApp({super.key});

```

```

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: Listenable.merge([themeService, localeService]),
      builder: (context, child) {
        return MaterialApp(
          title: 'FinanceHand',
          theme: ThemeData(
            useMaterial3: true,
            colorScheme: ColorScheme.fromSeed(
              seedColor: const Color(0xFF2E7D32),
              brightness: Brightness.light,
            ),
          ),
          darkTheme: ThemeData(
            useMaterial3: true,
            colorScheme: ColorScheme.fromSeed(
              seedColor: const Color(0xFF2E7D32),
              brightness: Brightness.dark,
            ),
          ),
          themeMode: themeService.themeMode,
          locale: localeService.locale,
          localizationsDelegates: const [
            AppLocalizations.delegate,

```

```

        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
    ],
    supportedLocales: const <Locale>[
        Locale('ru', 'RU'),
        Locale('en', 'US'),
    ],
    home: const SimpleMainPage(),
    debugShowCheckedModeBanner: false,
  );
},
);
}
}

//app_router.dart

// import 'package:flutter/material.dart'; // Unused for now
import 'package:go_router/go_router.dart';

import '../features/auth/presentation/pages/login_page.dart';
import '../features/auth/presentation/pages/register_page.dart';
import '../features/auth/presentation/pages/pin_setup_page.dart';
import '../features/main/presentation/pages/main_page.dart';
import '../features/main/presentation/pages/home_page.dart';
import '../features/main/presentation/pages/transactions_page.dart';
import '../features/main/presentation/pages/analytics_page.dart';
import '../features/main/presentation/pages/budget_page.dart';
import '../features/main/presentation/pages/settings_page.dart';

class AppRouter {
  static final GoRouter router = GoRouter(
    initialLocation: '/login',
    routes: [
      GoRoute(
        path: '/login',
        name: 'login',
        builder: (context, state) => const LoginPage(),
      ),
      GoRoute(
        path: '/register',
        name: 'register',
        builder: (context, state) => const RegisterPage(),
      ),
    ],
  );
}

```

```

    ),
    // GoRoute(
    //   path: '/pin-setup',
    //   name: 'pin-setup',
    //   builder: (context, state) => const PinSetupPage(), // Requires
pinAuthService parameter
    // ),
    GoRoute(
      path: '/main',
      name: 'main',
      builder: (context, state) => const MainPage(),
      routes: [
        GoRoute(
          path: '/home',
          name: 'home',
          builder: (context, state) => const HomePage(),
        ),
        GoRoute(
          path: '/transactions',
          name: 'transactions',
          builder: (context, state) => const TransactionsPage(),
        ),
        GoRoute(
          path: '/analytics',
          name: 'analytics',
          builder: (context, state) => const AnalyticsPage(),
        ),
        GoRoute(
          path: '/budget',
          name: 'budget',
          builder: (context, state) => const BudgetPage(),
        ),
        GoRoute(
          path: '/settings',
          name: 'settings',
          builder: (context, state) => const SettingsPage(),
        ),
      ],
    ),
  ],
);
}

```

```

import 'dart:convert';
import 'package:crypto/crypto.dart';
import 'package:shared_preferences/shared_preferences.dart';

class PinAuthService {
  static const String _pinHashKey = 'user_pin_hash';
  static const String _pinEnabledKey = 'pin_enabled';
  static const String _maxAttemptsKey = 'pin_attempts';
  static const String _lockoutTimeKey = 'lockout_until';

  static const int _maxAttempts = 3;
  static const int _lockoutDurationMinutes = 5;

  final SharedPreferences _prefs;

  PinAuthService(this._prefs);

  /// Проверяет, настроен ли PIN-код
  bool isPinEnabled() {
    return _prefs.getBool(_pinEnabledKey) ?? false;
  }

  /// Сохраняет PIN-код (хешированный)
  Future<bool> setPin(String pin) async {
    if (!_isValidPin(pin)) {
      return false;
    }

    try {
      final hash = _hashPin(pin);
      await _prefs.setString(_pinHashKey, hash);
      await _prefs.setBool(_pinEnabledKey, true);
      _clearAttempts();
      return true;
    } catch (e) {
      return false;
    }
  }

  /// Проверяет PIN-код
  Future<PinVerificationResult> verifyPin(String pin) async {
    if (!isPinEnabled()) {
      return PinVerificationResult.error('PIN not enabled');
    }
  }

```

```

// Проверяем блокировку
if (_isLockedOut()) {
    final lockoutUntil = _getLockoutUntil();
    if (lockoutUntil != null) {
        return PinVerificationResult.lockout(lockoutUntil);
    } else {
        return PinVerificationResult.error('Lockout error');
    }
}

// Проверяем формат PIN
if (!_isValidPin(pin)) {
    return PinVerificationResult.invalidFormat();
}

try {
    final storedHash = _prefs.getString(_pinHashKey);
    if (storedHash == null) {
        return PinVerificationResult.error('PIN not configured');
    }

    final inputHash = _hashPin(pin);

    if (storedHash == inputHash) {
        _clearAttempts();
        return PinVerificationResult.success();
    } else {
        _incrementAttempts();
        final attemptsLeft = _getAttemptsLeft();

        if (attemptsLeft <= 0) {
            _setLockout();
            return PinVerificationResult.maxAttemptsReached();
        }

        return PinVerificationResult.incorrect(attemptsLeft);
    }
} catch (e) {
    return PinVerificationResult.error('Verification failed');
}
}

/// Отключает PIN-код
Future<bool> disablePin(String currentPin) async {

```

```

    if (!isPinEnabled()) {
        return true;
    }

    final verification = await verifyPin(currentPin);
    if (!verification.isSuccess) {
        return false;
    }

    try {
        await _prefs.remove(_pinHashKey);
        await _prefs.setBool(_pinEnabledKey, false);
        _clearAttempts();
        return true;
    } catch (e) {
        return false;
    }
}

/// Изменяет PIN-код
Future<bool> changePin(String oldPin, String newPin) async {
    if (!isPinEnabled()) {
        return false;
    }

    final verification = await verifyPin(oldPin);
    if (!verification.isSuccess) {
        return false;
    }

    return await setPin(newPin);
}

/// Получает количество оставшихся попыток
int getAttemptsLeft() {
    return _getAttemptsLeft();
}

/// Проверяет, заблокирован ли доступ
bool isLockedOut() {
    return _isLockedOut();
}

/// Получает время окончания блокировки

```

```

DateTime? getLockoutUntil() {
    return _getLockoutUntil();
}

// Приватные методы

bool _isValidPin(String pin) {
    return pin.length == 4 && pin.contains(RegExp(r'^\d+$'));
}

String _hashPin(String pin) {
    final bytes = utf8.encode(pin + _getSalt());
    final digest = sha256.convert(bytes);
    return digest.toString();
}

String _getSalt() {
    // В реальном приложении соль должна быть более сложной
    return 'financehand_salt_2024';
}

void _incrementAttempts() {
    final current = _prefs.getInt(_maxAttemptsKey) ?? 0;
    _prefs.setInt(_maxAttemptsKey, current + 1);
}

int _getAttemptsLeft() {
    final current = _prefs.getInt(_maxAttemptsKey) ?? 0;
    return (_maxAttempts - current).clamp(0, _maxAttempts);
}

void _clearAttempts() {
    _prefs.remove(_maxAttemptsKey);
    _prefs.remove(_lockoutTimeKey);
}

void _setLockout() {
    final lockoutUntil = DateTime.now().add(
        Duration(minutes: _lockoutDurationMinutes),
    );
    _prefs.setInt(_lockoutTimeKey, lockoutUntil.millisecondsSinceEpoch);
}

bool _isLockedOut() {

```

```

final lockoutUntil = _getLockoutUntil();
if (lockoutUntil == null) return false;

if (DateTime.now().isAfter(lockoutUntil)) {
    _clearAttempts();
    return false;
}
return true;
}

DateTime? _getLockoutUntil() {
    final timestamp = _prefs.getInt(_lockoutTimeKey);
    if (timestamp == null) return null;

    return DateTime.fromMillisecondsSinceEpoch(timestamp);
}
}

/// Результат проверки PIN-кода
class PinVerificationResult {
    final bool isSuccess;
    final bool isLockout;
    final bool isIncorrect;
    final bool isMaxAttemptsReached;
    final bool isInvalidFormat;
    final String? error;
    final int? attemptsLeft;
    final DateTime? lockoutUntil;

    PinVerificationResult._({
        required this.isSuccess,
        required this.isLockout,
        required this.isIncorrect,
        required this.isMaxAttemptsReached,
        required this.isInvalidFormat,
        this.error,
        this.attemptsLeft,
        this.lockoutUntil,
    });

    factory PinVerificationResult.success() {
        return PinVerificationResult._(
            isSuccess: true,
            isLockout: false,

```

```

        isIncorrect: false,
        isMaxAttemptsReached: false,
        isInvalidFormat: false,
    );
}

factory PinVerificationResult.incorrect(int attemptsLeft) {
    return PinVerificationResult._(
        isSuccess: false,
        isLockout: false,
        isIncorrect: true,
        isMaxAttemptsReached: false,
        isInvalidFormat: false,
        attemptsLeft: attemptsLeft,
    );
}

factory PinVerificationResult.lockout(DateTime lockoutUntil) {
    return PinVerificationResult._(
        isSuccess: false,
        isLockout: true,
        isIncorrect: false,
        isMaxAttemptsReached: false,
        isInvalidFormat: false,
        lockoutUntil: lockoutUntil,
    );
}

factory PinVerificationResult.maxAttemptsReached() {
    return PinVerificationResult._(
        isSuccess: false,
        isLockout: true,
        isIncorrect: false,
        isMaxAttemptsReached: true,
        isInvalidFormat: false,
    );
}

factory PinVerificationResult.invalidFormat() {
    return PinVerificationResult._(
        isSuccess: false,
        isLockout: false,
        isIncorrect: false,
        isMaxAttemptsReached: false,

```

```

        isInvalidFormat: true,
    );
}

factory PinVerificationResult.error(String error) {
    return PinVerificationResult._(
        isSuccess: false,
        isLockout: false,
        isIncorrect: false,
        isMaxAttemptsReached: false,
        isInvalidFormat: false,
        error: error,
    );
}

import 'package:flutter/services.dart';
import 'package:local_auth/local_auth.dart';
import 'package:shared_preferences/shared_preferences.dart';

class BiometricAuthException implements Exception {
    final String message;

    BiometricAuthException(this.message);

    @override
    String toString() => message;
}

class BiometricAuthService {
    static const String _biometricEnabledKey = 'biometric_enabled';
    final LocalAuthentication _localAuth;
    final SharedPreferences _prefs;

    BiometricAuthService(this._prefs, {LocalAuthentication? localAuth})
        : _localAuth = localAuth ?? LocalAuthentication();

    Future<bool> isDeviceSupported() async {
        return await _localAuth.isDeviceSupported();
    }

    Future<bool> canCheckBiometrics() async {
        return await _localAuth.canCheckBiometrics;
    }
}

```

```

Future<List<BiometricType>> getAvailableBiometrics() async {
  try {
    return await _localAuth.getAvailableBiometrics();
  } on PlatformException {
    return <BiometricType>[];
  }
}

bool isBiometricEnabled() {
  return _prefs.getBool(_biometricEnabledKey) ?? false;
}

Future<bool> enableBiometric({required String localizedReason}) async {
  final bool supported = await isDeviceSupported();
  final List<BiometricType> availableBiometrics = await
getAvailableBiometrics();

  if (!supported || availableBiometrics.isEmpty) {
    throw BiometricAuthException('Biometric authentication is not available on
this device.');
```

```

  }

  try {
    final bool didAuthenticate = await _localAuth.authenticate(
      localizedReason: localizedReason,
      options: const AuthenticationOptions(
        biometricOnly: false,
        stickyAuth: false,
        useErrorDialogs: true,
        sensitiveTransaction: true,
      ),
    );

    if (!didAuthenticate) {
      throw BiometricAuthException('Authentication was not successful.');
```

```

    }

    await _prefs.setBool(_biometricEnabledKey, true);
    return true;
  } on PlatformException catch (e) {
    throw BiometricAuthException(e.message ?? e.code);
  }
}

```

```

Future<void> disableBiometric() async {
  await _prefs.setBool(_biometricEnabledKey, false);
}

Future<bool> authenticate({required String localizedReason}) async {
  if (!isBiometricEnabled()) {
    throw BiometricAuthException('Biometric authentication is disabled.');
```

```

  }

  try {
    final bool didAuthenticate = await _localAuth.authenticate(
      localizedReason: localizedReason,
      options: const AuthenticationOptions(
        biometricOnly: false,
        stickyAuth: false,
        useErrorDialogs: true,
        sensitiveTransaction: true,
      ),
    );

    if (!didAuthenticate) {
      throw BiometricAuthException('Authentication was not successful.');
```

```

    }

    return true;
  } on PlatformException catch (e) {
    throw BiometricAuthException(e.message ?? e.code);
  }
}

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('financehand.db');
```

```

    return _database!;
}

Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(
        path,
        version: 5,
        onCreate: _createDB,
        onUpgrade: _onUpgrade,
    );
}

Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async {
    if (oldVersion < 2) {
        // Добавляем поле currency в transactions
        await db.execute('ALTER TABLE transactions ADD COLUMN currency
TEXT DEFAULT \'RUB\');
        print('Database upgraded to version 2: Added currency field');
    }

    if (oldVersion < 3) {
        // Создаем таблицу для повторяющихся транзакций
        await db.execute("""
CREATE TABLE recurring_transactions (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    amount REAL NOT NULL,
    description TEXT,
    category_id TEXT NOT NULL,
    transaction_type TEXT CHECK (transaction_type IN ('income', 'expense'))
NOT NULL,
    currency TEXT DEFAULT 'RUB',
    interval TEXT CHECK (interval IN ('daily', 'weekly', 'monthly', 'yearly',
'custom')) NOT NULL,
    custom_days INTEGER,
    start_date INTEGER NOT NULL,
    end_date INTEGER,
    last_executed INTEGER,
    next_execution INTEGER,
    is_active INTEGER DEFAULT 1,
    created_at INTEGER,

```

```

        updated_at INTEGER,
        FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE
        RESTRICT,
        FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
        CASCADE
    )
    "");
    print('Database upgraded to version 3: Added recurring_transactions table');
}

if (oldVersion < 4) {
    // Добавляем поле is_recurring в categories
    await db.execute('ALTER TABLE categories ADD COLUMN is_recurring
    INTEGER DEFAULT 0');
    print('Database upgraded to version 4: Added is_recurring field to categories');
}

if (oldVersion < 5) {
    // Добавляем поле recurring_id в transactions
    await db.execute('ALTER TABLE transactions ADD COLUMN recurring_id
    TEXT');
    print('Database upgraded to version 5: Added recurring_id field to
    transactions');
}
}

Future<void> _createDB(Database db, int version) async {
    // Users table
    await db.execute("""
    CREATE TABLE users (
        id TEXT PRIMARY KEY,
        username TEXT UNIQUE,
        email TEXT UNIQUE,
        password_hash TEXT,
        pin_hash TEXT,
        biometric_enabled INTEGER DEFAULT 0,
        currency TEXT DEFAULT 'USD',
        language TEXT DEFAULT 'ru',
        created_at INTEGER,
        updated_at INTEGER
    )
    """);

    // Categories table

```

```

await db.execute("""
CREATE TABLE categories (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  icon TEXT,
  color INTEGER,
  type TEXT CHECK (type IN ('income', 'expense')) NOT NULL,
  is_default INTEGER DEFAULT 0,
  user_id TEXT,
  is_recurring INTEGER DEFAULT 0,
  created_at INTEGER,
  updated_at INTEGER,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
CASCADE
)
""");

// Transactions table
await db.execute("""
CREATE TABLE transactions (
  id TEXT PRIMARY KEY,
  amount REAL NOT NULL,
  description TEXT,
  category_id TEXT NOT NULL,
  user_id TEXT NOT NULL,
  transaction_type TEXT CHECK (transaction_type IN ('income', 'expense'))
NOT NULL,
  currency TEXT DEFAULT 'RUB',
  date INTEGER NOT NULL,
  created_at INTEGER,
  updated_at INTEGER,
  recurring_id TEXT,
  FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE
RESTRICT,
  FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
CASCADE
)
""");

// Budgets table
await db.execute("""
CREATE TABLE budgets (
  id TEXT PRIMARY KEY,
  category_id TEXT,

```

```

        user_id TEXT NOT NULL,
        amount REAL NOT NULL,
        period TEXT CHECK (period IN ('weekly', 'monthly', 'yearly')) NOT NULL,
        start_date INTEGER NOT NULL,
        end_date INTEGER NOT NULL,
        created_at INTEGER,
        updated_at INTEGER,
        FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE
CASCADE,
        FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
CASCADE
    ) ");

```

```

// Recurring transactions table
await db.execute("
    CREATE TABLE recurring_transactions (
        id TEXT PRIMARY KEY,
        user_id TEXT NOT NULL,
        amount REAL NOT NULL,
        description TEXT,
        category_id TEXT NOT NULL,
        transaction_type TEXT CHECK (transaction_type IN ('income', 'expense'))
NOT NULL,
        currency TEXT DEFAULT 'RUB',
        interval TEXT CHECK (interval IN ('daily', 'weekly', 'monthly', 'yearly',
'custom')) NOT NULL,
        custom_days INTEGER,
        start_date INTEGER NOT NULL,
        end_date INTEGER,
        last_executed INTEGER,
        next_execution INTEGER,
        is_active INTEGER DEFAULT 1,
        created_at INTEGER,
        updated_at INTEGER,
        FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE
RESTRICT,
        FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
CASCADE
    )
");

```

```

// Create indexes for better performance
await db.execute('CREATE INDEX idx_transactions_user_date ON transactions
(user_id, date)');

```

```

    await db.execute('CREATE INDEX idx_transactions_category ON transactions
(category_id)');
    await db.execute('CREATE INDEX idx_budgets_user_period ON budgets
(user_id, period)');

    // Insert default categories
    await _insertDefaultCategories(db);
}

```

```

Future<void> _insertDefaultCategories(Database db) async {
    // Check if categories already exist
    final existingCategories = await db.query('categories', where: 'is_default = 1');
    if (existingCategories.isNotEmpty) {
        return; // Categories already exist
    }
}

```

```

final now = DateTime.now().millisecondsSinceEpoch;

```

```

// Default expense categories
final expenseCategories = [
    {'id': 'food', 'name': 'Еда', 'icon': '🍲', 'type': 'expense'},
    {'id': 'transport', 'name': 'Транспорт', 'icon': '🚗', 'type': 'expense'},
    {'id': 'shopping', 'name': 'Покупки', 'icon': '🛒', 'type': 'expense'},
    {'id': 'entertainment', 'name': 'Развлечения', 'icon': '🎬', 'type': 'expense'},
    {'id': 'bills', 'name': 'Счета', 'icon': '💡', 'type': 'expense'},
    {'id': 'health', 'name': 'Здоровье', 'icon': '🏥', 'type': 'expense'},
    {'id': 'education', 'name': 'Образование', 'icon': '📖', 'type': 'expense'},
    {'id': 'other_expense', 'name': 'Прочее', 'icon': '📦', 'type': 'expense'},
];

```

```

// Default income categories
final incomeCategories = [
    {'id': 'salary', 'name': 'Зарплата', 'icon': '💼', 'type': 'income'},
    {'id': 'freelance', 'name': 'Фриланс', 'icon': '💻', 'type': 'income'},
    {'id': 'investment', 'name': 'Инвестиции', 'icon': '📈', 'type': 'income'},
    {'id': 'gift', 'name': 'Подарки', 'icon': '🎁', 'type': 'income'},
    {'id': 'other_income', 'name': 'Прочее', 'icon': '💰', 'type': 'income'},
];

```

```

for (final category in [...expenseCategories, ...incomeCategories]) {
    await db.insert('categories', {
        'id': category['id'],

```

```

        'name': category['name'],
        'icon': category['icon'],
        'color': null,
        'type': category['type'],
        'is_default': 1,
        'user_id': null,
        'created_at': now,
        'updated_at': now,
    });
}
}

Future<void> ensureDefaultCategories() async {
    final db = await database;
    await _insertDefaultCategories(db);
}

Future<void> close() async {
    final db = await database;
    db.close();
}

class User {
    final String id;
    final String? username;
    final String? email;
    final bool biometricEnabled;
    final String currency;
    final String language;

    const User({
        required this.id,
        this.username,
        this.email,
        this.biometricEnabled = false,
        this.currency = 'USD',
        this.language = 'en',
    });
}
import '../data/models/budget_model.dart';

class Budget {
    final String id;
    final String? categoryId;

```

```

final String userId;
final double amount;
final BudgetPeriod period;
final DateTime startDate;
final DateTime endDate;
final DateTime createdAt;
final DateTime updatedAt;

const Budget({
  required this.id,
  this.categoryId,
  required this.userId,
  required this.amount,
  required this.period,
  required this.startDate,
  required this.endDate,
  required this.createdAt,
  required this.updatedAt,
});
}
import '../data/models/category_model.dart';

class Category {
  final String id;
  final String name;
  final String? icon;
  final int? color;
  final CategoryType type;
  final bool isDefault;
  final String? userId;
  final bool isRecurring;
  final DateTime createdAt;
  final DateTime updatedAt;

  const Category({
    required this.id,
    required this.name,
    this.icon,
    this.color,
    required this.type,
    this.isDefault = false,
    this.userId,
    this.isRecurring = false,
    required this.createdAt,

```

```

        required this.updatedAt,
    });
}

import 'package:equatable/equatable.dart';

class Currency extends Equatable {
    final String code;
    final String name;
    final String symbol;
    final double rate;
    final DateTime lastUpdated;

    const Currency({
        required this.code,
        required this.name,
        required this.symbol,
        required this.rate,
        required this.lastUpdated,
    });

    Currency copyWith({
        String? code,
        String? name,
        String? symbol,
        double? rate,
        DateTime? lastUpdated,
    }) {
        return Currency(
            code: code ?? this.code,
            name: name ?? this.name,
            symbol: symbol ?? this.symbol,
            rate: rate ?? this.rate,
            lastUpdated: lastUpdated ?? this.lastUpdated,
        );
    }

    @override
    List<Object?> get props => [code, name, symbol, rate, lastUpdated];
}

// Предопределенные валюты
class CurrencyConstants {
    static List<Currency> get supportedCurrencies {

```

```

final now = DateTime.now();
return [
  Currency(
    code: 'USD',
    name: 'US Dollar',
    symbol: '\$',
    rate: 1.0,
    lastUpdated: now, ),
  Currency(
    code: 'EUR',
    name: 'Euro',
    symbol: '€',
    rate: 0.85,
    lastUpdated: now, ),
  Currency(
    code: 'RUB',
    name: 'Russian Ruble',
    symbol: '₽',
    rate: 95.0,
    lastUpdated: now, ),
  Currency(
    code: 'BYN',
    name: 'Belarusian Ruble',
    symbol: 'Br',
    rate: 3.25,
    lastUpdated: now, ),
  Currency(
    code: 'GBP',
    name: 'British Pound',
    symbol: '£',
    rate: 0.73,
    lastUpdated: now, ),
  Currency(
    code: 'JPY',
    name: 'Japanese Yen',
    symbol: '¥',
    rate: 110.0,
    lastUpdated: now, ),
];
}
static const String defaultCurrency = 'USD';
}

import '../data/models/transaction_model.dart';

```

```

class Transaction {
  final String id;
  final double amount;
  final String? description;
  final String categoryId;
  final String userId;
  final TransactionType type;
  final String currency; // Валюта на момент создания транзакции
  final DateTime date;
  final DateTime createdAt;
  final DateTime updatedAt;
  final String? recurringId; // ID повторяющейся транзакции, если это
повторяющаяся

```

```

const Transaction({
  required this.id,
  required this.amount,
  this.description,
  required this.categoryId,
  required this.userId,
  required this.type,
  required this.currency,
  required this.date,
  required this.createdAt,
  required this.updatedAt,
  this.recurringId, });
}

```

```

import 'package:shared_preferences/shared_preferences.dart';
import '../features/budget/domain/entities/budget.dart';
import '../features/transactions/domain/entities/transaction.dart';
import '../features/transactions/data/models/transaction_model.dart';
import '../features/budget/data/models/budget_model.dart';
import '../main.dart' as main_app;

```

```

class BudgetAlertService {
  static const String _dismissedAlertsKey = 'dismissed_budget_alerts';

  /// Проверяет все бюджеты и возвращает список превышенных
  Future<List<BudgetAlert>> checkBudgetAlerts({
    required List<Budget> budgets,
    required List<Transaction> transactions,
  }) async {

```

```

final alerts = <BudgetAlert>[];
final dismissedAlerts = await getDismissedAlerts();

for (var budget in budgets) {
  final periodStart = _calculatePeriodStart(budget.period);
  final spent = _calculateSpentForBudget(transactions, budget, periodStart);
  final percentage = budget.amount > 0 ? spent / budget.amount : 0.0;

  // Создаем уникальный ID для алерта
  final alertId = '${budget.id}_${budget.period.name}';

  // Проверяем, не был ли алерт закрыт
  if (!dismissedAlerts.contains(alertId)) {
    if (percentage > 1.0) {
      alerts.add(BudgetAlert(
        id: alertId,
        budgetId: budget.id,
        categoryId: budget.categoryId,
        message: 'Превышение на ${_formatAmount(spent - budget.amount)}',
        type: BudgetAlertType.exceeded,
        spent: spent,
        budget: budget.amount,
        percentage: percentage,
      ));
    } else if (percentage > 0.9) {
      alerts.add(BudgetAlert(
        id: alertId,
        budgetId: budget.id,
        categoryId: budget.categoryId,
        message: 'Потрачено ${(percentage * 100).toStringAsFixed(0)}%
бюджета',
        type: BudgetAlertType.warning,
        spent: spent,
        budget: budget.amount,
        percentage: percentage,
      ));
    }
  }
}

return alerts;
}

DateTime _calculatePeriodStart(BudgetPeriod period) {

```

```

final now = DateTime.now();
switch (period) {
  case BudgetPeriod.weekly:
    return now.subtract(const Duration(days: 7));
  case BudgetPeriod.monthly:
    return DateTime(now.year, now.month, 1);
  case BudgetPeriod.yearly:
    return DateTime(now.year, 1, 1);
}
}

double _calculateSpentForBudget(
  List<Transaction> transactions,
  Budget budget,
  DateTime periodStart,
) {
  final currentCurrency = main_app.currencyService.getSelectedCurrency();
  return transactions
    .where((t) {
      final bool matchesCategory =
        budget.categoryId == null ? true : t.categoryId == budget.categoryId;
      final bool matchesType = t.type == TransactionType.expense;
      final bool matchesDate = (t.date.isAfter(periodStart) ||
        t.date.isAtSameMomentAs(periodStart)) &&
        (t.date.isBefore(budget.endDate) ||
        t.date.isAtSameMomentAs(budget.endDate));
      return matchesCategory && matchesType && matchesDate;
    })
    .fold(0.0, (sum, t) {
      final convertedAmount = _convertAmount(t.amount, t.currency,
currentCurrency);
      return sum + convertedAmount;
    });
}

double _convertAmount(double amount, String fromCurrency, String
toCurrency) {
  if (fromCurrency == toCurrency) return amount;

  const rates = {
    'USD': 1.0,
    'EUR': 0.85,
    'RUB': 95.0,
    'BYN': 3.25,

```

```

        'GBP': 0.73,
        'JPY': 110.0,
    };

    final fromRate = rates[fromCurrency] ?? 1.0;
    final toRate = rates[toCurrency] ?? 1.0;

    final amountInUSD = amount / fromRate;
    return amountInUSD * toRate;
}

String _formatAmount(double amount) {
    return '${amount.toStringAsFixed(2)}
    ${main_app.currencyService.getSelectedCurrency()}';
}

/// Получает список закрытых уведомлений
Future<List<String>> getDismissedAlerts() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getStringList(_dismissedAlertsKey) ?? [];
}

/// Закрывает уведомление
Future<void> dismissAlert(String alertId) async {
    final prefs = await SharedPreferences.getInstance();
    final dismissed = await getDismissedAlerts();
    if (!dismissed.contains(alertId)) {
        dismissed.add(alertId);
        await prefs.setStringList(_dismissedAlertsKey, dismissed);
    }
}

/// Очищает все закрытые уведомления (например, при начале нового
периода)
Future<void> clearDismissedAlerts() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_dismissedAlertsKey);
}

enum BudgetAlertType {
    warning,
    exceeded,
}

```

```

class BudgetAlert {
  final String id;
  final String budgetId;
  final String? categoryId;
  final String message;
  final BudgetAlertType type;
  final double spent;
  final double budget;
  final double percentage;

  const BudgetAlert({
    required this.id,
    required this.budgetId,
    this.categoryId,
    required this.message,
    required this.type,
    required this.spent,
    required this.budget,
    required this.percentage, });
}

import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

class CurrencyApiService {
  static const String _baseUrl = 'https://api.frankfurter.app';
  static const String _cacheKey = 'currency_rates_cache';
  static const int _cacheExpiryHours = 24;

  final SharedPreferences _prefs;

  CurrencyApiService(this._prefs);

  // Получение курсов валют
  Future<Map<String, double>> getExchangeRates({
    String baseCurrency = 'USD',
    List<String>? targetCurrencies, }) async {
    try {
      // Проверяем кэш
      final cachedRates = await _getCachedRates();
      if (cachedRates != null && !_isCacheExpired(cachedRates['lastUpdated'])) {
        return Map<String, double>.from(cachedRates['rates']);
      }
    }
  }
}

```

```

// Получаем курсы из основного API (исключаем BYN)
final currenciesToFetch = targetCurrencies ?? ['EUR', 'RUB', 'GBP', 'JPY'];
final rates = await _fetchRatesFromApi(baseCurrency, currenciesToFetch);

// Добавляем BYN через альтернативный API или статичный курс
if (targetCurrencies == null || targetCurrencies.contains('BYN')) {
  try {
    final bynRate = await _getBYNRate(baseCurrency);
    rates['BYN'] = bynRate;
  } catch (e) {
    print('Failed to fetch BYN rate, using default: $e');
    rates['BYN'] = _getDefaultRates()['BYN'] ?? 3.25;
  }
}

// Сохраняем в кэш
await _saveToCache(rates);

return rates;
} catch (e) {
  // В случае ошибки возвращаем кэшированные данные
  final cachedRates = await _getCachedRates();
  if (cachedRates != null) {
    return Map<String, double>.from(cachedRates['rates']);
  }

  // Если кэша нет, возвращаем статичные курсы
  return _getDefaultRates();
}

// Получение курса между двумя валютами
Future<double> getExchangeRate(String from, String to) async {
  if (from == to) return 1.0;

  final rates = await getExchangeRates(baseCurrency: from, targetCurrencies:
[to]);
  return rates[to] ?? 1.0;
}

// Конвертация суммы
Future<double> convertAmount(double amount, String from, String to) async {
  final rate = await getExchangeRate(from, to);

```

```

    return amount * rate;
}

// Получение списка доступных валют
Future<List<String>> getAvailableCurrencies() async {
  try {
    final response = await http.get(
      Uri.parse('${_baseUrl}/currencies'),
      headers: {'Accept': 'application/json'},
    );

    if (response.statusCode == 200) {
      final Map<String, dynamic> data = json.decode(response.body);
      return data.keys.toList();
    }
  } catch (e) {
    print('Error fetching currencies: $e');
  }

  // Возвращаем дефолтный список валют
  return ['USD', 'EUR', 'RUB', 'BYN', 'GBP', 'JPY', 'CHF', 'CAD', 'AUD'];
}

// Получение данных из API
Future<Map<String, double>> _fetchRatesFromApi(
  String baseCurrency,
  List<String>? targetCurrencies,
) async {
  final targetList = targetCurrencies ?? ['EUR', 'RUB', 'GBP', 'JPY'];
  final targets = targetList.join(',');

  final response = await http.get(
    Uri.parse('${_baseUrl}/latest?from=$baseCurrency&to=$targets'),
    headers: {'Accept': 'application/json'},
  );

  if (response.statusCode == 200) {
    final Map<String, dynamic> data = json.decode(response.body);
    final Map<String, dynamic> rates = data['rates'] ?? {};

    // Конвертируем в Map<String, double>
    return rates.map((key, value) => MapEntry(key, (value as num).toDouble()));
  } else {
    throw Exception('Failed to fetch exchange rates: ${response.statusCode}');
  }
}

```

```

}

// Получение кэшированных данных
Future<Map<String, dynamic>?> _getCachedRates() async {
  try {
    final cachedData = _prefs.getString(_cacheKey);

    if (cachedData != null) {
      return json.decode(cachedData);
    }
  } catch (e) {
    print('Error reading cached rates: $e');
  }

  return null;
}

// Сохранение в кэш
Future<void> _saveToCache(Map<String, double> rates) async {
  try {
    final cacheData = {
      'rates': rates,
      'lastUpdated': DateTime.now().toIso8601String(),
    };

    await _prefs.setString(_cacheKey, json.encode(cacheData));
  } catch (e) {
    print('Error saving cached rates: $e');
  }
}

// Проверка истечения кэша
bool _isCacheExpired(String lastUpdated) {
  try {
    final lastUpdate = DateTime.parse(lastUpdated);
    final now = DateTime.now();
    return now.difference(lastUpdate).inHours > _cacheExpiryHours;
  } catch (e) {
    return true; // Если ошибка парсинга, считаем кэш устаревшим
  }
}

// Дефолтные курсы (на случай недоступности API)
Map<String, double> _getDefaultRates() {

```

```

return {
    'EUR': 0.85,
    'RUB': 95.0,
    'BYN': 3.25,
    'GBP': 0.73,
    'JPY': 110.0,
    'CHF': 0.92,
    'CAD': 1.25,
    'AUD': 1.35,  };
}

// Получение курса BYN через альтернативный API
Future<double> _getBYNRate(String baseCurrency) async {
    if (baseCurrency == 'BYN') return 1.0;

    try {
        // Пробуем получить через альтернативный API
        // Используем Free Currency API как резерв для BYN
        final response = await http.get(
            Uri.parse('https://api.freecurrencyapi.com/v1/latest?apikey=fca_live_placeholder&
                currencies=BYN&base_currency=$baseCurrency'),
                headers: {'Accept': 'application/json'},
            ).timeout(const Duration(seconds: 5));

        if (response.statusCode == 200) {
            final Map<String, dynamic> data = json.decode(response.body);
            final rates = data['data'];
            if (rates != null && rates['BYN'] != null) {
                return (rates['BYN'] as num).toDouble();
            }
        }
    } catch (e) {
        print('Free Currency API failed for BYN: $e');
    }

    // Fallback: вычисляем через USD
    try {
        if (baseCurrency != 'USD') {
            // Получаем курс baseCurrency к USD
            final baseToUsd = await _fetchRatesFromApi(baseCurrency, ['USD']);
            final baseToUsdRate = baseToUsd['USD'] ?? 1.0;

            // Статичный курс BYN к USD (примерно 3.25)
            const usdToBynRate = 3.25;

```

```

        return baseToUsdRate * usdToBynRate;
    } else {
        return 3.25; // Статичный курс BYN к USD
    }
} catch (e) {
    print('Fallback calculation failed: $e');
    return 3.25; // Финальный fallback
}
}

// Очистка кэша
Future<void> clearCache() async {
    try {
        await _prefs.remove(_cacheKey);
    } catch (e) {
        print('Error clearing cache: $e');
    }
}

import 'package:shared_preferences/shared_preferences.dart';
import 'currency_api_service.dart';
import '../features/currency/domain/entities/currency.dart';

class CurrencyService {
    static const String _selectedCurrencyKey = 'selected_currency';
    static const String _currencyRatesKey = 'currency_rates';

    final CurrencyApiService _apiService;
    final SharedPreferences _prefs;

    CurrencyService(this._prefs, this._apiService);

    // Получение текущей выбранной валюты
    String getSelectedCurrency() {
        return _prefs.getString(_selectedCurrencyKey) ??
CurrencyConstants.defaultCurrency;
    }

    // Установка выбранной валюты
    Future<void> setSelectedCurrency(String currencyCode) async {
        await _prefs.setString(_selectedCurrencyKey, currencyCode);
    }

    // Получение курсов валют

```

```

Future<Map<String, double>> getExchangeRates() async {
    final baseCurrency = getSelectedCurrency();
    return await _apiService.getExchangeRates(baseCurrency: baseCurrency);
}

// Конвертация суммы в выбранную валюту
Future<double> convertToSelectedCurrency(double amount, String
fromCurrency) async {
    final selectedCurrency = getSelectedCurrency();
    if (fromCurrency == selectedCurrency) return amount;

    return await _apiService.convertAmount(amount, fromCurrency,
selectedCurrency);
}

// Конвертация из выбранной валюты в другую
Future<double> convertFromSelectedCurrency(double amount, String
toCurrency) async {
    final selectedCurrency = getSelectedCurrency();
    if (selectedCurrency == toCurrency) return amount;

    return await _apiService.convertAmount(amount, selectedCurrency,
toCurrency);
}

// Конвертация между любыми валютами
Future<double> convertAmount(double amount, String fromCurrency, String
toCurrency) async {
    if (fromCurrency == toCurrency) return amount;

    return await _apiService.convertAmount(amount, fromCurrency, toCurrency);
}

// Получение символа валюты
String getCurrencySymbol(String currencyCode) {
    final currency = CurrencyConstants.supportedCurrencies
        .firstWhere((c) => c.code == currencyCode, orElse: () =>
CurrencyConstants.supportedCurrencies.first);
    return currency.symbol;
}

// Получение названия валюты
String getCurrencyName(String currencyCode) {
    final currency = CurrencyConstants.supportedCurrencies

```

```

        .firstWhere((c) => c.code == currencyCode, orElse: () =>
CurrencyConstants.supportedCurrencies.first);
    return currency.name;
}

// Форматирование суммы с валютой
String formatAmount(double amount, {String? currencyCode}) {
    final code = currencyCode ?? getSelectedCurrency();
    final symbol = getCurrencySymbol(code);

    // Форматируем число с 2 знаками после запятой
    final formattedAmount = amount.toStringAsFixed(2);

    // Добавляем пробелы для больших чисел
    final parts = formattedAmount.split('.');
    final integerPart = parts[0];
    final decimalPart = parts.length > 1 ? parts[1] : "";

    String formattedInteger = integerPart;
    if (integerPart.length > 3) {
        final buffer = StringBuffer();
        for (int i = 0; i < integerPart.length; i++) {
            if (i > 0 && (integerPart.length - i) % 3 == 0) {
                buffer.write(' ');
            }
            buffer.write(integerPart[i]);
        }
        formattedInteger = buffer.toString();
    }

    return '$formattedInteger${decimalPart.isNotEmpty ? '.$decimalPart' : ""}
$symbol';
}

// Получение списка поддерживаемых валют
List<Currency> getSupportedCurrencies() {
    return CurrencyConstants.supportedCurrencies;
}

// Обновление курсов валют
Future<void> updateExchangeRates() async {
    try {
        final rates = await getExchangeRates();
        // Курсы автоматически кэшируются в CurrencyApiService

```

```

    } catch (e) {
      print('Failed to update exchange rates: $e');
    }
  }

  // Очистка кэша курсов
  Future<void> clearCache() async {
    await _apiService.clearCache();
  }

  // Проверка доступности API
  Future<bool> isApiAvailable() async {
    try {
      final rates = await _apiService.getExchangeRates();
      return rates.isNotEmpty;
    } catch (e) {
      return false;
    }
  }
}

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class LocaleService extends ChangeNotifier {
  static const String _localeKey = 'selected_locale';
  final SharedPreferences _prefs;
  Locale _locale = const Locale('ru', 'RU');

  LocaleService(this._prefs) {
    _loadLocale();
  }

  Locale get locale => _locale;

  String get languageName {
    switch (_locale.languageCode) {
      case 'en':
        return 'English';
      case 'ru':
        return 'Русский';
      default:
        return 'Русский';
    }
  }
}

```

```

    }

    Future<void> _loadLocale() async {
      final languageCode = _prefs.getString(_localeKey);
      if (languageCode != null) {
        _locale = Locale(languageCode, languageCode == 'ru' ? 'RU' : 'US');
        notifyListeners();
      }
    }

    Future<void> setLocale(Locale newLocale) async {
      _locale = newLocale;
      await _prefs.setString(_localeKey, newLocale.languageCode);
      notifyListeners();
    }

    Future<void> setLanguage(String languageCode) async {
      final countryCode = languageCode == 'ru' ? 'RU' : 'US';
      await setLocale(Locale(languageCode, countryCode));
    }
  }

import 'package:shared_preferences/shared_preferences.dart';

import
'../../features/recurring/data/datasources/recurring_transactions_local_datasource.dart';
import ' ../../features/recurring/data/models/recurring_transaction_model.dart';
import
'../../features/transactions/data/datasources/transactions_local_datasource.dart';
import ' ../../features/transactions/data/models/transaction_model.dart';
import ' ../database/database_helper.dart';

class RecurringCheckerService {
  static const int _catchUpDays = 90;

  final RecurringTransactionsLocalDataSource _recurringDs =
    RecurringTransactionsLocalDataSourceImpl(DatabaseHelper.instance);
  final TransactionsLocalDataSource _transactionsDs =
    TransactionsLocalDataSourceImpl(DatabaseHelper.instance);

  Future<void> run() async {

```

```

final prefs = await SharedPreferences.getInstance();
final userId = prefs.getString('current_user_id') ?? 'default_user';
await _processUser(userId);
}

Future<void> _processUser(String userId) async {
  final now = DateTime.now();
  final activeRecurring = await
_recurringDs.getActiveRecurringTransactions(userId);

  for (final recurringModel in activeRecurring) {
    final recurring = recurringModel.toEntity();
    if (!recurring.isActive) continue;
    if (recurring.endDate != null && now.isAfter(recurring.endDate!)) continue;

    final catchUpStart = now.subtract(const Duration(days: _catchUpDays));
    DateTime cursor = recurring.lastExecuted ?? recurring.startDate;
    if (cursor.isBefore(catchUpStart)) {
      cursor = catchUpStart;
    }

    // Страхуемся: если nextExecution известен и он раньше cursor, сдвигаем в
    его сторону
    if (recurring.nextExecution != null &&
recurring.nextExecution!.isAfter(cursor)) {
      cursor = recurring.nextExecution!;
    }

    while (!cursor.isAfter(now)) {
      if (recurring.endDate != null && cursor.isAfter(recurring.endDate!)) break;

      final exists = await _transactionExists(userId, recurring.id, cursor);
      if (!exists) {
        await _insertTransaction(recurringModel, cursor);
      }

      final next = _calculateNextDate(cursor, recurring.interval,
recurring.customDays);
      await _recurringDs.updateLastExecuted(recurring.id, cursor, next);
      cursor = next;
    }
  }
}

Future<bool> _transactionExists(String userId, String recurringId, DateTime

```

```

date) async {
  final all = await _transactionsDs.getTransactions(userId);
  return all.any((t) =>
    t.recurringId == recurringId &&
    t.date.year == date.year &&
    t.date.month == date.month &&
    t.date.day == date.day);
}

Future<void> _insertTransaction(RecurringTransactionModel recurring,
DateTime date) async {
  final now = DateTime.now();
  final tx = TransactionModel(
    id: now.millisecondsSinceEpoch.toString(),
    amount: recurring.amount,
    description: recurring.description,
    categoryId: recurring.categoryId,
    userId: recurring.userId,
    type: recurring.transactionType,
    currency: recurring.currency,
    date: date,
    createdAt: now,
    updatedAt: now,
    recurringId: recurring.id,
  );
  await _transactionsDs.addTransaction(tx);
}

DateTime _calculateNextDate(DateTime current, RecurringIntervalModel
interval, int? customDays) {
  switch (interval) {
    case RecurringIntervalModel.daily:
      return current.add(const Duration(days: 1));
    case RecurringIntervalModel.weekly:
      return current.add(const Duration(days: 7));
    case RecurringIntervalModel.monthly:
      return DateTime(current.year, current.month + 1, current.day);
    case RecurringIntervalModel.yearly:
      return DateTime(current.year + 1, current.month, current.day);
    case RecurringIntervalModel.custom:
      return current.add(Duration(days: customDays ?? 30)); }
  }
}

import 'package:shared_preferences/shared_preferences.dart';

```

```

import
'../features/recurring/data/datasources/recurring_transactions_local_datasource.da
rt';
import '../features/recurring/data/models/recurring_transaction_model.dart';
import '../features/recurring/domain/entities/recurring_transaction.dart';
import '../features/transactions/data/models/transaction_model.dart';
import
'../features/transactions/data/datasources/transactions_local_datasource.dart';

class RecurringTransactionService {
  static const String _lastCheckKey = 'recurring_last_check';

  final RecurringTransactionsLocalDataSource _recurringDataSource;
  final TransactionsLocalDataSource _transactionsDataSource;
  final SharedPreferences _prefs;

  RecurringTransactionService(
    this._recurringDataSource,
    this._transactionsDataSource,
    this._prefs,
  );

  /// Проверяет и выполняет все повторяющиеся транзакции, которые должны
  быть выполнены
  Future<int> executeRecurringTransactions(String userId) async {
    print('🔄 Checking recurring transactions for user: $userId');

    final now = DateTime.now();
    final activeRecurring = await
    _recurringDataSource.getActiveRecurringTransactions(userId);

    int executedCount = 0;

    for (var recurring in activeRecurring) {
      final entity = recurring.toEntity();

      if (entity.shouldExecute(now)) {
        print('✓ Executing: ${entity.description} ?? entity.transactionType}
        (${entity.interval.name}));

        // Создаем новую транзакцию
        await _createTransactionFromRecurring(entity);

        // Обновляем last_executed и next_execution

```

```

        final nextExec = entity.calculateNextExecution();
        await _recurringDataSource.updateLastExecuted(
            entity.id,
            now,
            nextExec,
        );
        executedCount++;
    }
}

// Сохраняем время последней проверки
await _prefs.setInt(_lastCheckKey, now.millisecondsSinceEpoch);

print('🎉 Executed $executedCount recurring transactions');
return executedCount;
}

Future<void> _createTransactionFromRecurring(RecurringTransaction recurring)
async {
    final now = DateTime.now();
    print('🔄 Creating transaction from recurring: ${recurring.id}');
    final transaction = TransactionModel(
        id: '${now.millisecondsSinceEpoch}_recurring_${recurring.id}',
        amount: recurring.amount,
        description: recurring.description,
        categoryId: recurring.categoryId,
        userId: recurring.userId,
        type: recurring.transactionType == 'income'
            ? TransactionType.income
            : TransactionType.expense,
        currency: recurring.currency,
        date: now,
        createdAt: now,
        updatedAt: now,
        recurringId: recurring.id, // ВАЖНО: устанавливаем recurringId!
    );
    print('📄 Transaction created with recurringId: ${transaction.recurringId}');
    await _transactionsDataSource.addTransaction(transaction);
}

/// Проверяет, нужно ли выполнить проверку (не чаще чем раз в час)
bool shouldCheck() {
    final lastCheck = _prefs.getInt(_lastCheckKey);
    if (lastCheck == null) return true;
}

```

```

final lastCheckDate = DateTime.fromMillisecondsSinceEpoch(lastCheck);
final now = DateTime.now();

// Проверяем раз в час
return now.difference(lastCheckDate).inHours >= 1;
}

/// Получение следующей даты выполнения для транзакции
DateTime calculateNextExecution(
  DateTime baseDate,
  String interval,
  int? customDays,
) {
  switch (interval) {
    case 'daily':
      return baseDate.add(const Duration(days: 1));
    case 'weekly':
      return baseDate.add(const Duration(days: 7));
    case 'monthly':
      return DateTime(
        baseDate.year,
        baseDate.month + 1,
        baseDate.day,
      );
    case 'yearly':
      return DateTime(
        baseDate.year + 1,
        baseDate.month,
        baseDate.day,
      );
    case 'custom':
      final days = customDays ?? 30;
      return baseDate.add(Duration(days: days));
    default:
      return baseDate.add(const Duration(days: 30));  }
  }
}

import 'dart:async';
import 'package:flutter/foundation.dart';

class TransactionsStreamService {
  static final TransactionsStreamService _instance =
    TransactionsStreamService._internal();

```

```

factory TransactionsStreamService() => _instance;
TransactionsStreamService._internal();

final StreamController<String> _deleteController =
StreamController<String>.broadcast();
final StreamController<String> _addController =
StreamController<String>.broadcast();
final StreamController<String> _updateController =
StreamController<String>.broadcast();

// Streams для отслеживания изменений
Stream<String> get deleteStream => _deleteController.stream;
Stream<String> get addStream => _addController.stream;
Stream<String> get updateStream => _updateController.stream;

// Методы для уведомления об изменениях
void notifyTransactionDeleted(String transactionId) {
  if (kDebugMode) {
    print('TransactionsStreamService: Transaction $transactionId deleted');
  }
  _deleteController.add(transactionId);
}

void notifyTransactionAdded(String transactionId) {
  if (kDebugMode) {
    print('TransactionsStreamService: Transaction $transactionId added');
  }
  _addController.add(transactionId);
}

void notifyTransactionUpdated(String transactionId) {
  if (kDebugMode) {
    print('TransactionsStreamService: Transaction $transactionId updated');
  }
  _updateController.add(transactionId); }

void dispose() {
  _deleteController.close();
  _addController.close();
  _updateController.close(); }
}

```

Обозначение					Наименование					Примечание		
					<u>Текстовые документы</u>							
БГУИР ДП 1-40 01 01 01 218 ПЗ					Пояснительная записка					126 с		
					Отзыв руководителя							
					Рецензия							
					Отчёт о проверке на заимствования							
					Справка о внедрении							
					<u>Графические документы</u>							
ГУИР.281071.001 ПЛ					Диаграмма вариантов использования					Формат А1		
					Плакат							
ГУИР.281071.002 ПЛ					Диаграмма деятельности					Формат А1		
					Плакат							
ГУИР.281071.003 ПЛ					Диаграмма классов программного средства					Формат А1		
					Плакат							
ГУИР.281071.004 СА					Алгоритм добавление транзакции и пересчет бюджета					Формат А1		
					Схема алгоритма							
ГУИР.281071.005 СА					Алгоритм агрегации расходов по категориям					Формат А1		
					Схема алгоритма							
ГУИР.281071.006 СА					Алгоритм безопасности					Формат А1		
					Схема алгоритма							